UNIVERSITATEA DE TIINE I TEHNOLOGIE
POLITEHNICA BUCURETI

Facultatea de Electronic, Telecomunicaii
i Tehnologia Informaiei

Programul de masterat:
Ingineria Informaiei i Sisteme de Calcul

Sistem E-Medicine pentru Analizarea i Monitorizarea Datelor Epidemiologice cu Modele Compartimentale din Familia SIR

LUCRARE DE DISERTAIE

Coordonator tiinific:
Prof. Radu Rdescu

Absolvent:
Gean Tudor-Bogdan

Bucureti, 2026

Rezumat

Aceast lucrare trateaz proiectarea unui sistem e-medicine pentru analiza datelor epidemiologice prin modele compartimentale, cu modelul SIDARTHE folosit ca direcie clinic principal de explorare. Spre deosebire de modelele SIR/SIS/SEIR/SIRS, care agreg dinamica în câteva stri canonice, SIDARTHE separ infeciile nedepistate, cazurile diagnosticate, formele simptomatice, formele severe, vindecrile i decesele. Aceast granularitate face modelele mai potrivite pentru discutarea efectelor testrii, diagnosticrii i severitii clinice asupra evoluiei unei epidemii.

Modelele SIR, SIS, SEIR i SIRS rmân prezente în lucrare ca repere secundare, folosite pentru comparaii i pentru verificarea comportamentului numeric al implementrilor. Sistemul construit separ explicit stocarea datelor, expunerea API-ului i execuia simulrilor. Datele epidemiologice sunt persistate într-o schem relaional compatibil PostgreSQL, rulat peste YugabyteDB, iar simulrile sunt executate asincron de runner-e specializate, coordonate prin Redis.

Lucrarea introduce un cadru explicit pentru compararea modelelor i calibrarea parametrilor. Modelele sunt comparate structural, prin compartimente i ipoteze, i numeric, prin operatori de observaie, MAE, RMSE i nRMSE. Calibrarea este formulat ca problem de optimizare pe parametrii fiecrui model, cu accent pe separarea dintre fereastra de antrenare i fereastra de test. În plus, este integrat o extensie predictiv bazat pe LSTM, cu runner dedicat, folosit ca model de serie temporal i ca posibil corecie a reziduurilor modelelor compartimentale.

Componenta web, alctuit dintr-un API Go i dou interfee Vue.js separate, ofer fluxurile necesare organizrii datelor pe studii epidemiologice, gestionrii rolurilor de acces, încrcrii seturilor de date, lansrii simulrilor, urmririi statusului i vizualizrii traiectoriilor compartimentale. Aplicaia principal este destinat cercettorilor i colaboratorilor dintr-un studiu, iar consola administrativ este rezervat utilizatorilor cu drepturi globale de super-admin. Întregul sistem este deployabil pe un cluster Kubernetes prin manifeste declarative, pe o platform cloud privat cu factor de replicare 3 pentru baza de date. Sistemul este validat empiric printr-o campanie de benchmark executat prin aplicaie (180 de rulri ODE cu cinci modele compartimentale, trei limbaje i patru orizonturi temporale) i prin calibrarea SIR pe valul 1 al pandemiei COVID-19 în Romania, Italia, Germania i Spania, cu analiza identifiabilitii practice prin scanarea peisajului de pierdere. Accentul lucrrii cade pe legtura dintre modelul matematic, implementarea numeric i arhitectura software care face posibil evaluarea reproductibil a rezultatelor pentru modele cu grade diferite de complexitate.

Cuvinte cheie: model SIDARTHE, modele SIR, epidemiologie computaional, e-medicine, calibrare de parametri, LSTM, RBAC, YugabyteDB, Kubernetes, Go, Vue.js, benchmark de performan.

Cuprins

Rezumat **i**

List de figuri **vii**

List de tabele **ix**

Lista algoritmilor **x**

1 Introducere **1**
1.1 Context i motivaie . . . 1
1.2 Lucrri i instrumente conexe . . . 1
1.3 Obiectivele lucrrii . . . 2
1.4 Contribuii originale . . . 2
1.5 Structura lucrrii . . . 2

2 Epidemiologie Computaional i Modelarea Matematic a Bolilor Infecioase **4**
2.1 Fundamente de epidemiologie . . . 4
2.1.1 Definiii i concepte fundamentale . . . 4
2.1.2 Parametrii fundamentali ai transmiterii . . . 4
2.2 Modele compartimentale în epidemiologie . . . 4
2.2.1 Cadrul general . . . 4
2.2.2 Modelul SIS . . . 5
2.2.3 Modelul SEIR . . . 5
2.2.4 Modelul SIRS . . . 5
2.3 Modelul SIDARTHE . . . 5
2.4 Modelul SIR ca reper secundar . . . 6
2.4.1 Ipoteze i derivarea ecuaiilor din principii prime . . . 6
2.4.2 Proprieti matematice fundamentale . . . 6
2.4.3 Analiza echilibrelor i stabilitate . . . 7
2.4.4 Numrul de reproducere de baz $\mathcal{R}_0$ . . . 7
2.4.5 Pragul de imunizare în turm (herd immunity) . . . 8
2.4.6 Ecuaia dimensiunii finale a epidemiei . . . 8
2.4.7 Extensii ale modelului SIR . . . 8
2.5 Metode Numerice pentru Ecuaii Difereniale Ordinare . . . 9
2.5.1 Clasificarea metodelor numerice . . . 9
2.5.2 Metoda Euler explicit . . . 9
2.5.3 Metoda Runge-Kutta de ordinul 4 (RK4) . . . 9
2.5.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince) . . . 10
2.5.5 Stabilitate numeric i probleme stiff . . . 10
2.6 Date Epidemiologice Surse i Preprocesare . . . 11
2.6.1 Surse de date epidemiologice . . . 11
2.6.2 Structura datelor i preprocesare . . . 11
2.6.3 Estimarea parametrilor din date reale . . . 11

3 Tehnologii i Instrumente Utilizate **12**
3.1 Limbaje de programare pentru calcul tiinific . . . 12
3.1.1 R . . . 12
3.1.2 Julia . . . 12
3.1.3 Python . . . 13
3.1.4 Analiz comparativ a limbajelor . . . 13
3.2 Baze de date distribuite YugabyteDB . . . 13
3.2.1 Concepte de baze de date distribuite . . . 13
3.2.2 Arhitectura YugabyteDB . . . 13
3.2.3 Sharding i replicare . . . 14
3.3 Orchestrarea containerelor Kubernetes . . . 14
3.3.1 Containere i nevoia de orchestrare . . . 14
3.3.2 Arhitectura Kubernetes . . . 14
3.3.3 Resurse Kubernetes relevante . . . 14
3.4 Backend API Go . . . 14
3.4.1 Caracteristici i filosofia limbajului Go . . . 14
3.4.2 Ecosistem pentru API-uri REST . . . 15
3.5 Frontend Vue.js . . . 15
3.5.1 Arhitectura Vue 3 i Composition API . . . 15
3.5.2 Ecosistem i biblioteci utilizate . . . 15

4 Arhitectura Sistemului E-Medicine **16**
4.1 Vedere de ansamblu . . . 16
4.2 Cerine funcionale i non-funcionale . . . 16
4.2.1 Cerine funcionale . . . 16
4.2.2 Cerine non-funcionale . . . 17
4.3 Arhitectura bazei de date distribuite . . . 18
4.3.1 Schema relaional . . . 18
4.3.2 Modelul datelor epidemiologice . . . 19
4.3.3 Strategia de partiionare i replicare în YugabyteDB . . . 20
4.4 Arhitectura API-ului backend Go . . . 21
4.4.1 Structura modular a proiectului . . . 21
4.4.2 Mecanismul de autentificare JWT . . . 21
4.4.3 RBAC pe studii . . . 21
4.4.4 Diagrama endpointurilor REST . . . 21
4.4.5 Integrarea cu runner-ele de algoritmi . . . 22
4.5 Arhitectura frontend Vue.js . . . 22
4.5.1 Structura componentelor . . . 22
4.5.2 Fluxul de date i state management . . . 23
4.6 Infrastructura Kubernetes . . . 23
4.6.1 Topologia cluster-ului . . . 23
4.6.2 Deployment YugabyteDB pe Kubernetes . . . 24
4.6.3 Stratul aplicativ: imagini container i registru . . . 24
4.6.4 Coad de mesaje i migraiile bazei de date . . . 24
4.6.5 Expunere extern i ingress . . . 25
4.7 Sistem de plug-in-uri pentru algoritmi . . . 25
4.7.1 Forma standard: *runner protocol* . . . 26
4.7.2 Provisioning Kubernetes . . . 26
4.7.3 Securitate i limitri . . . 27

5 Implementarea modelelor compartimentale **28**
5.1 Datele de intrare i iniializarea compartimentelor . . . 28
5.2 Contractul comun de simulare . . . 29
5.3 Implementarea modelului SIDARTHE . . . 29
5.4 Variantele SIR, SIS, SEIR i SIRS . . . 31
5.5 Persistena rezultatelor i exportul . . . 31
5.6 Metrici de performan . . . 31

6 Predicie Epidemiologic folosind Reele LSTM **32**
6.1 Rolul prediciei statistice în sistem . . . 32
6.2 Formularea problemei de predicie . . . 32
6.3 Arhitectura celulei LSTM . . . 32
6.4 Normalizarea i construcia exemplelor . . . 33
6.5 Antrenarea prin backpropagation through time . . . 33
6.6 Împrirea cronologic i evaluarea . . . 34
6.7 Integrarea în arhitectura aplicaiei . . . 34
6.8 Linii de baz i limitri . . . 35
6.9 Direcii de extindere . . . 36

7 Comparaia Modelelor, Analiz Numeric i Calibrare **37**
7.1 Rolul comparaiei între modele . . . 37
7.2 Comparaie structural . . . 37
7.3 Operatorul de observaie . . . 37
7.4 Metrici de eroare numeric . . . 37
7.5 Protocol de analiz comparativ . . . 38
7.6 Calibrarea parametrilor . . . 38
7.6.1 Aplicare pe Romania, valul 1 din 2020 . . . 39
7.7 Identificabilitate i limitri ale calibrrii . . . 39
7.7.1 Vizualizarea identifiabilitatii practice pentru SIR . . . 40
7.7.2 Generalizare pe mai multe regiuni . . . 40
7.8 Integrarea în sistemul software . . . 41
7.8.1 Costul software al calibrrii . . . 41

8 Evaluarea Software, Performanei i Scalabilitii **44**
8.1 Scopul evalurii . . . 44
8.2 Metrici colectate . . . 44
8.3 Matricea experimental . . . 44
8.4 Protocol de rulare . . . 44
8.5 Scalabilitate i complexitatea modelului . . . 46
8.6 Compararea implementrilor R, Julia i Python . . . 46
8.7 Indicatori agregai . . . 46
8.7.1 Rezultate msurate . . . 46
8.7.2 Costul componentei LSTM . . . 48
8.7.3 Variabilitatea cross-hardware . . . 48
8.8 Acuratee numeric i verificri de regresie . . . 49
8.9 Evaluarea API-ului i a bazei de date . . . 49
8.10 Interpretarea rezultatelor . . . 50

9 Aplicaia Web E-Medicine **52**
9.1 Rolul aplicaiei . . . 52
9.2 Backend Go . . . 52
9.3 Gestionarea dataseturilor . . . 52
9.4 Execuia simulrilor . . . 52
9.5 Editor de cod pentru cercettori . . . 53
9.5.1 Flux de lucru . . . 53
9.5.2 Contractul scriptului . . . 53
9.5.3 Execuie i separare de runner-e persistente . . . 53
9.5.4 Izolare între utilizatori i securitate . . . 54
9.6 Consola super-admin . . . 54
9.7 Consideraii de securitate . . . 54
9.7.1 Mecanisme implementate . . . 55
9.7.2 Limitri cunoscute i remedieri . . . 55
9.8 Validare funcional . . . 55

10 Concluzii i Direcii de Dezvoltare Viitoare **56**
10.1 Sinteza lucrrii . . . 56
10.1.1 Rspuns la obiectivele propuse . . . 56
10.2 Concluzii privind modelele . . . 56
10.3 Concluzii privind implementarea . . . 57
10.4 Concluzii experimentale . . . 57
10.5 Limitri . . . 58
10.6 Direcii viitoare . . . 58

Bibliografie **60**

A Cod Surs Implementarea R **62**

B Cod Surs Implementarea Julia **67**

C Cod Surs Implementarea Python **72**

D Cod Surs — Runner LSTM **77**

E Schema Bazei de Date **85**

F Configuraii Kubernetes **92**

G Documentaia API REST **107**

List de figuri

2.1 Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezint rata de noi infecii, iar γI reprezint rata de recuperare. 6

4.1 Arhitectura global a sistemului e-medicine. Sgeile indic direcia de flux a datelor: browserul acceseaz aplicaia prin Ingress, care ruteaz cererile ctre cele dou SPA-uri sau API; API-ul public job-uri în Redis i le consum runner-ele dedicate; atât API-ul, cât i runner-ele acceseaz acelai cluster YugabyteDB. . . . 17

4.2 Ciclul de via al unei simulri, de la cererea utilizatorului în UI pân la afiarea rezultatului. Timpul curge de sus în jos. Sgeile pline indic mesaje sincrone HTTP/queue/SQL; sgeile întrerupte indic polling-ul repetat al statusului pân la finalizare. API-ul rspunde `201 Created` *înainte* ca solver-ul s ruleze — întreaga execuie numeric se petrece asincron în runner, decuplat prin Redis. Aceasta este complementar Figurii 4.1, care arat topologia: cine vorbete cu cine; aici se vede *când*. 18

4.3 Diagrama entitate-relaie a bazei de date curente. PK-urile sunt marcate cu aldine i subliniere; FK-urile sunt marcate cu *FK*. Tabelele `datasets` i `simulation_runs` sunt scope-uite pe `study_id`, iar tabela `study_members` definete rolul fiecrui utilizator într-un studiu. 19

4.4 Topologia cluster-ului Kubernetes cu 3 noduri control plane HA i 3 noduri worker. Pod-urile YugabyteDB sunt distribuite uniform prin `podAntiAffinity`, garantând c pierderea unui worker afecteaz cel mult o replic `yb-master` i o replic `yb-tserver` — în limita toleranei quorum-ului Raft. 25

6.1 Iesirile runner-ului LSTM dedicat pe seria de cazuri active din Romania (4 ani, 22 ian. 2020 – 31 dec. 2023), cu antrenare pe primele 80% din puncte si evaluare pe ultimele 20%. Predictiile o-zi-inainte ($\hat{y}_t = f_\theta(y_{t-14:t-1})$) urmaresc seria observata atat pe fereastra de antrenare cat si pe cea de test, dar prognoza autoregresiva care reintroduce iesirile in fereastra (dincolo de ziua 1440) tinde rapid catre zero — o limitare cunoscuta a LSTM-urilor compacte fara mecanisme explicite pentru sezonalitate, asupra careia un strat ulterior cu informatie exogena (intervalul de raportare, calendarul scolar) ar putea ameliora. 35

7.1 Calibrare SIR pe Romania, valul 1 din 2020. Linia neagra arata seria observata; cele doua scenarii produc fituri vizual aproape identice pe fereastra de antrenare (MAE $\approx 1\,600$ cazuri pentru ambele), dar prognoza pe luna urmatoare diverge (MAE 13 031 pentru fitul liber vs 16 727 pentru cel constrans). Parametrii estimati difera substantial, ilustrand identifiabilitatea practica slaba: aceleasi date sunt explicate aproape la fel de bine de combinatii (β, γ) foarte diferite. 40

7.2 Peisajul de pierdere $\log_{10}\mathrm{MSE}(\beta, \gamma)$ pentru modelul SIR pe fereastra de antrenare. Punctele rosii marcheaza solutia libera $(0{,}368, 0{,}335)$ si solutia constransa $(0{,}104, 0{,}0714)$; linia rosie punctata indica restrictia clinica $\gamma = 1/14$. Valea de pierdere mica este alungita si aproape liniara, ceea ce inseamna ca multe combinatii (β, γ) produc o potrivire similara cu datele observate din valul 1 — semnatura tipica a identifiabilitatii practice slabe pentru SIR cu o singura serie observata. 41

7.3 Calibrare SIR cu $\gamma = 1/14$ pe valul 1 al pandemiei COVID-19, in patru tari europene cu scari de populatie diferite (atenie la scarile y distincte). Linia neagra este seria observata, linia rosie continua este fitul pe fereastra de antrenare (15 mar. – 15 aug. 2020), iar linia rosie intrerupta este prognoza pe luna urmatoare. Linia verticala marcheaza granita antrenare/test. Modelul reproduce calitativ valul 1 in toate cele patru cazuri; divergenta pe test este vizibila in Spania, unde valul 2 incepe sa creasca rapid in septembrie, evidentiind limitarea structurala a SIR cu coeficienti constanti. 42

8.1 Scalarea wall-time-ului median al solverului SIDARTHE in functie de orizontul $t_{\max}$, pentru cele trei limbaje. R prezinta o crestere aproape liniara cu $t_{\max}$ ($\sim$130$\times$ intre 90 si 10 000 de pasi), dominata de inserarile rand-cu-rand in baza de date prin `dbExecute()`; integrarea propriu-zisa cu `deSolve::ode` este sub-secundara. Python ramane stabil (4–8 ms), iar Julia este mai rapid in regimul amortizat dupa compilarea JIT, atingand 6 ms la 10 000 de pasi. 48

List de tabele

3.1 Comparaie a limbajelor de programare utilizate 14

4.1 Structura minim a fiierului CSV de date epidemiologice. 20
4.2 Endpointurile REST principale ale API-ului backend. 22
4.3 Ciclul de via al unui runner plug-in, de la upload pân la oprire. Strile sunt persistate in coloana `status` a tabelei `runners`; tranziiile sunt declanate de API (creare/tergere) i de wrapper-ul din pod (marcarea `ready`). 26

5.1 Maparea datelor observabile în condiii iniiale pentru modelele suportate. 29
5.2 Comparaie side-by-side a celor trei runner-e compartimentale, ca instane ale aceluiai contract de simulare. Diferenele in de ecosistemul numeric (biblioteca solver dominant, convenia de control al pasului), nu de modelul matematic implementat. Numrul de linii include parsarea mesajului, dispecerizarea pe model, scrierea în baza de date i colectarea metricilor. 31

6.1 Corespondena dintre formularea matematic a celulei LSTM i implementarea din `runner-lstm`. 34
6.2 Validarea LSTM fata de predictori naivi, eroare o-zi-inainte pe fereastra de test (ultimele 20% din seria Romania, 288 zile). Toate metodele folosesc doar valori observate trecute. *Skill* este reducerea relativa a MAE fata de persistenta (valori negative = mai slab decat persistenta). Pe aceasta serie, 85% din zilele de test sunt identice cu ziua precedenta (raportare saptamanala), ceea ce face persistenta extrem de greu de batut. 35

7.1 Comparaie structural între modelele utilizate în lucrare. 38
7.2 Locurile din analiza comparativ care necesit testare pentru obinerea rezultatelor. 39
7.3 Calibrare SIR pe Romania, valul 1 (15 martie – 15 august 2020 antrenare, 16 august – 15 septembrie 2020 test). Doua scenarii: *liber* (optimizeaza ambele rate, (β, γ)), *constrans* (foloseste $\gamma = 1/14$ din durata clinica medie a infectiei COVID-19, optimizeaza doar β). Comparatia ilustreaza tensiunea dintre minimizarea pierderii si interpretabilitatea epidemiologica a parametrilor. 40
7.4 Calibrare SIR cross-regiune pe valul 1 al pandemiei COVID-19 (15 martie – 15 august 2020 antrenare, 16 august – 15 septembrie 2020 test). In toate cele patru cazuri γ este fixat la valoarea de literatura 1/14 zile, iar β este optimizat prin Nelder-Mead. Valorile $\hat{\beta}$ si $\hat{\mathcal{R}}_0$ sunt comparabile direct intre tari, dar erorile absolute reflecta scarile diferite de populatie (column N) si de I_0. 41
7.5 Costul software al calibrarii SIR (Romania, valul 1; fereastra de 154 zile, restart-uri multiple Nelder–Mead pentru scenariile liber si constrans). Coloana *in-process* masoara direct costul procedurii din `calibrate.py`; coloana *naiv (ca job-uri)* proiecteaza costul daca fiecare simulare candidata ar fi un job separat prin aplicatie, folosind latenta end-to-end masurata a unui job SIR real (1047 ms/job); coloana *recomandat* corespunde calibrarii intr-un singur job, cu o singura simulare finala scrisa in baza de date. Numarul de simulari candidate este numarul real de evaluari ale functiei obiectiv. 43

8.1 Metricile colectate sau derivabile pentru evaluarea software. 45
8.2 Matrice minim pentru campania de benchmark. 45
8.3 Teste necesare pentru obinerea rezultatelor de evaluare software. 45
8.4 Wall-time median al solverului (ms), pe model si limbaj, pentru patru orizonturi $t_{\max}$. Rezultate din campania de benchmark prin aplicatie (cluster `kube-4kdhb`, 3 worker x 4 vCPU / 8 GiB), 3 repetari per combinatie, dataset Romania OWID, prima inregistrare ca conditie initiala. 47
8.5 Raport de viteza al solverului fata de Python (mediana wall-time Python / mediana limbaj). Valori > 1 inseamna mai rapid decat Python; valori < 1 inseamna mai lent. Pragul '$> x$' apare cand mediana sub-milisecundara face raportul exact instabil. 47

8.6 Memorie reziduala peak (MB), mediana pe toate combinatiile model x $t_{\max}$. 48
8.7 Costul de pornire JIT pentru runner-ul Julia: prima rulare include compilarea, repetarile ulterioare ruleaza din cod cache-uit. Valori la $t_{\max} = 90$. 48
8.8 Costul rularii pentru runner-ul LSTM dedicat (Python, hand-rolled NumPy), parametri impliciti (`lookback=14`, `hidden_size=12`, `epochs=150`, `learning_rate=0.01`), dataset Romania OWID (1 440 zile, 80% antrenare / 20% test cronologic). Forecast-ul autoregresiv pe orizontul $t_{\max}$ are cost neglijabil fata de antrenare. 49
8.9 Comparatia timpilor de executie pentru solverul Python (`scipy.solve_ivp` RK45) pe doua profile hardware distincte: cluster `kube-4kdhb` (worker amd64, OpenStack VM, 4 vCPU / 8 GiB, runner in container, wall-time include si scrierea rezultatelor in YugabyteDB) vs M1 Pro arm64 nativ (script standalone, fara container, fara DB). Workload non-trivial: set sintetic `multiwave` cu $I_0 = 100$, $N = 5 \cdot 10^7$. Coloana *speedup M1* este raportul medianelor; valori > 1 inseamna M1 mai rapid. 50
8.10 Verificare numerica de regresie cross-limbaj: abaterea maxima absoluta intre traiectoriile compartimentale produse in R, Julia si Python pentru aceiasi parametri si aceleasi conditii initiale, pe setul sintetic `multiwave` (populatia $N = 50 \cdot 10^6$, $t_{\max} = 365$ pasi zilnici). Modelele folosesc solvere diferite (R cu RK4 pas fix $h = 1$, Python cu RK45 adaptiv, Julia cu Tsit5), deci abaterile reflecta tolerantele numerice cumulate, nu erori de implementare. 50
8.11 Latenta API pe model, la orizontul de 365 puncte (1 an, pas zilnic). Pentru fiecare endpoint se raporteaza mediana (p_{50}); pentru *results*, endpointul cel mai greu, se raporteaza si percentila 95 (p_{95}) si dimensiunea raspunsului JSON. *create* raspunde asincron (inainte de rularea solver-ului); *results* si *export* includ deserializarea din YugabyteDB. Masurat prin port-forward la clusterul `kube-4kdhb`. . . . 51
8.12 Scalarea latentei de citire cu orizontul, pentru modelul `sidarthe` (8 compartimente). Pe masura ce numarul de puncte creste, latenta *results* si dimensiunea JSON cresc aproximativ liniar, in timp ce *export* CSV ramane mai ieftin (text plat, fara structura JSON). 51

F.1 Manifestele Kubernetes incluse în repository. 92

Lista algoritmilor

2.1 Metoda Euler pentru sistemul SIR 9
2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR 10
6.1 Antrenarea i evaluarea componentei LSTM 36
7.1 Calibrarea unui model compartimental pe date observate 39

Capitolul 1

Introducere

1.1 Context i motivaie

Bolile infecioase rmân una dintre formele prin care vulnerabilitatea unui sistem sanitar devine vizibil la scar social. Pandemia de grip din 1918, epidemia de SARS din 2003 i pandemia de COVID-19 declanat în 2019 arat c propagarea unui agent patogen nu poate fi înelegeas doar ca fenomen clinic, ci i ca proces populaional, dependent de contacte, mobilitate, raportare i capacitatea de rspuns instituional [1, 2].

Modelele matematice compartimentale ofer un limbaj riguros pentru descrierea acestor procese. Ele reduc populaia la stri epidemiologice relevante i descriu tranziiile dintre stri prin ecuaii difereniale. Modelul SIR (Susceptibili–Infectai–Recuperai), introdus de Kermack i McKendrick în 1927 [3], rmâne un reper important deoarece permite o analiz matematic transparent. Totui, pentru epidemii în care testarea, diagnosticul i severitatea clinic modific puternic raportarea, un model cu trei compartimente este prea agregat. Din acest motiv, lucrarea folosete SIDARTHE ca model clinic principal, pstrând modelele SIR, SIS, SEIR i SIRS ca repere secundare pentru comparaii i verificri.

În practica actual, problema nu se oprete la scrierea ecuaiilor. Datele provin din surse eterogene, sunt actualizate frecvent i conin corecii, goluri sau inconsistene administrative. În acelai timp, implementarea numeric poate fi realizat în limbaje diferite, fiecare cu avantaje i costuri proprii. Alegerea dintre R, Julia i Python nu ine doar de sintax, ci influeneaz timpul de execuie, consumul de memorie, modul de integrare cu infrastructura i reproductibilitatea analizei.

Pornind de la aceste observaii, lucrarea propune un sistem e-medicine în care modelul SIDARTHE este integrat într-un flux complet: ingestie de date epidemiologice, execuie asincron a simulrilor, stocare a rezultatelor i vizualizare în interfa web. Modelele SIR, SIS, SEIR i SIRS rmân utile ca repere mai simple, prin care se poate observa ce se pierde sau ce se câtig atunci când modelul introduce compartimente suplimentare.

1.2 Lucrri i instrumente conexe

Modelarea computaional a epidemiilor se sprijin pe un ecosistem matur de biblioteci de rezolvare a ecuaiilor difereniale. Lucrarea de fa folosete direct trei dintre ele, fiecare reprezentativ pentru limbajul su: `deSolve` în R [4], `DifferentialEquations.jl` în Julia [5] i `scipy.integrate` în Python [6]. Aceste biblioteci sunt crmizi de baz, optimizate i bine testate, dar rmân componente de calcul: ele rezolv un sistem ODE dat, fr a impune o structur de date epidemiologice, o disciplin de colectare a metricilor sau un cadru de comparaie între limbaje. În aceast lucrare ele sunt tratate exact ca atare — componente interschimbabile, evaluate sistematic în aceleai condiii.

La un nivel mai înalt exist instrumente dedicate modelrii epidemiologice, dar fiecare se specializeaz pe un ecosistem sau pe o paradigm. EpiModel ofer în R un cadru pentru modele compartimentale i pe reele de contact [7]; Covasim implementeaz în Python un model agent-based detaliat pentru COVID-19 [8]; iar microsimularea de tip CovidSim folosit în raportul Imperial College a influenat deciziile de politic sanitar din 2020 [9]. Aceste instrumente sunt puternice în domeniul lor, îns rmân legate de un singur limbaj i nu urmresc o comparaie cantitativ cross-limbaj a costului numeric, nici integrarea într-un sistem web colaborativ.

Pentru estimarea parametrilor, abordarea standard în literatura recent este calibrarea bayesian, cu instrumente precum Stan [10] sau NIMBLE [11], care ofer cuantificarea incertitudinii prin metode de tip MCMC/HMC. Lucrarea de fa folosete o calibrare determinist prin metoda Nelder–Mead, suficient pentru obiectivul comparativ urmrit; abordarea bayesian este menionat ca direcie viitoare, nu ca element exclus din motive metodologice.

În fine, dincolo de modelare exist platforme de agregare i diseminare a prognozelor, precum COVID-19 Forecast Hub [12], care combin predicii din mai multe modele independente în prognoze de ansamblu. Acestea rezolv o problem complementar — coordonarea i evaluarea prognozelor la scar — i nu un toolkit de modelare reutilizabil.

Raportat la aceste direcii, contribuia lucrrii se afl la intersecia lor: un benchmarking comparativ al aceluiai model implementat în R, Julia i Python, combinat cu o platform web integrat, cu control al accesului bazat pe roluri la nivel de studiu i cu o evaluare empiric sistematic a costului software. Aceast combinaie o deosebete atât de bibliotecile mono-limbaj de rezolvare a ODE-urilor,

cât i de platformele epidemiologice specializate care nu ofer aceast integrare de sistem.

1.3 Obiectivele lucrrii

Obiectivul principal este proiectarea unui sistem care leag explicit modelarea matematic de implementarea software. Modelul SIDARTHE este tratat ca model de lucru principal, deoarece separ cazurile nedepistate, diagnosticate, simptomatice, severe, vindecate i decedate. Familia SIR/SIS/SEIR/SIRS este analizat în continuare, dar rolul su devine acela de reper teoretic i computaional, util pentru înelegerea proprietilor de baz ale modelelor compartimentale.

Al doilea obiectiv este implementarea aceluiai contract de simulare în R, Julia i Python, folosind bibliotecile uzuale ale fiecrui ecosistem pentru rezolvarea ecuaiilor difereniale. Comparaia urmrete atât compatibilitatea rezultatelor, cât i costurile practice ale execuiei: timp wall-clock, timp CPU, memorie i integrare cu baza de date.

Al treilea obiectiv este definirea unei metodologii explicite de comparaie i calibrare. Modelele nu sunt evaluate doar vizual, ci prin operatori de observaie, metrici de eroare precum MAE i RMSE, separare între fereastra de calibrare i fereastra de test i raportarea costului software asociat fiecrei rulri.

Un obiectiv suplimentar este integrarea unei extensii predictive bazate pe LSTM. Aceasta este tratat ca model de serie temporal pentru predicii numerice pe termen scurt, implementat prin runner dedicat, i ca posibil corecie a reziduurilor produse de modelele compartimentale calibrate.

În paralel, lucrarea construiete infrastructura necesar rulrii acestor experimente într-un cadru accesibil prin web. API-ul Go, baza de date YugabyteDB, coada Redis i cele dou interfee Vue.js sunt tratate ca pri ale aceluiai instrument de cercetare, nu ca module izolate.

1.4 Contribuii originale

Contribuia lucrrii const în combinarea unei analize matematice clasice cu un sistem software funcional, capabil s ruleze modele compartimentale cu structuri de stare diferite în aceleai medii de execuie. Partea matematic fixeaz ipotezele i limitele modelelor, partea de implementare arat cum se schimb costul numeric între limbaje, iar partea de arhitectur ofer un cadru în care datele, simulrile i benchmark-urile pot fi gestionate unitar.

O a doua contribuie este metodologia de colectare a metricilor. Fiecare runner marcheaz explicit strile unei simulri, scrie rezultatele în aceeai schem de baz de date i salveaz metrici comparabile. Aceast disciplin de integrare este necesar pentru a evita comparaiile informale între limbaje, care pot fi uor influenate de diferene de I/O, iniializare sau serializare.

O alt contribuie este separarea între comparaia epidemiologic i comparaia software. Lucrarea definete operatori de observaie pentru a transforma strile interne ale modelelor în serii comparabile cu datele reale i introduce un cadru de calibrare care poate fi extins ulterior într-un job dedicat în aplicaie.

În fine, sistemul web rezultat permite folosirea modelelor într-un flux apropiat de un scenariu real de analiz colaborativ: crearea unui studiu, acordarea rolurilor, încrcarea unui set de date, alegerea modelului i a parametrilor, execuia asincron, vizualizarea traiectoriilor i compararea rulrilor.

O contribuie arhitectural suplimentar este **sistemul de plug-in-uri pentru algoritmi** (capitolul 4, seciunea 4.7): un super-admin poate înregistra într-un studiu fie un script Python (rulat de un container generic), fie o referin ctre o imagine container construit extern (în orice limbaj), iar API-ul provisioneaz automat un Deployment Kubernetes dedicat. Aceast extensie deschide aplicaia de la cinci modele compartimentale plus LSTM la *orice* algoritm care respect un contract minimal de I/O, fr modificri la API sau la schema bazei de date.

1.5 Structura lucrrii

Lucrarea este organizat în zece capitole.

Capitolul 2 (*Epidemiologie Computaional i Modelarea Matematic*) prezint fundamentele epidemiologiei matematice. Sunt introduse modelele compartimentale (SIR, SIS, SEIR, SIRS), apoi este prezentat modelul SIDARTHE ca model clinic principal. Modelul SIR este pstrat pentru analiza matematic de baz, deoarece permite discutarea clar a pozitivitii, conservrii populaiei i pragului epidemic $\mathcal{R}_0$. Capitolul include i o trecere în revist a metodelor numerice utilizate pentru rezolvarea sistemelor de ecuaii difereniale ordinare.

Capitolul 3 (*Tehnologii i Instrumente Utilizate*) explic de ce au fost alese limbajele R, Julia i Python, baza de date YugabyteDB, orchestrarea containerelor prin Kubernetes, backend-ul Go i frontend-urile Vue.js.

Capitolul 4 (*Arhitectura Sistemului E-Medicine*) descrie arhitectura global a sistemului, cerinele funcionale i non-funcionale, schema bazei de date distribuite, arhitectura API-ului backend i structura componentelor frontend.

Capitolul 5 (*Implementarea modelelor compartimentale*) detaliaz implementrile practice ale modelelor în fiecare limbaj de programare, inclusiv seturile de date utilizate, parametrii, schema generic de rezultate i interfaa de integrare cu API-ul Go.

Capitolul 6 (*Predicie Epidemiologic folosind Reele LSTM*) prezint rolul unei componente LSTM pentru prognoza seriilor temporale, formularea problemei de predicie, integrarea cu modelele compartimentale i evaluarea unui model hibrid compartimental–LSTM.

Capitolul 7 (*Comparaia Modelelor, Analiz Numeric i Calibrare*) introduce tabelul comparativ al modelelor,

operatorii de observaie, metricile MAE/RMSE i formularea calibrrii parametrilor ca problem de optimizare.

Capitolul 8 (*Evaluarea Software, Performanei i Scalabilitii*) prezint metodologia de benchmarking, matricea experimental, metricile colectate, verificrile de regresie numeric i evaluarea API-ului i a bazei de date.

Capitolul 9 (*Aplicaia Web E-Medicine*) detaliaz implementarea aplicaiei web, documentând backend-ul Go, aplicaia principal Vue.js i consola Vue.js pentru superadmini, cu descrierea funcionalitilor i a fluxurilor de utilizare.

Capitolul 10 (*Concluzii i Direcii Viitoare*) sintetizeaz contribuiile lucrrii, discut limitele sistemului i identific direcii de dezvoltare ulterioar.

Lucrarea se încheie cu **Bibliografie** i **Anexe** care conin codul surs complet al implementrilor, schema bazei de date, configuraiile Kubernetes i documentaia API-ului REST.

Capitolul 2

Epidemiologie Computaional i Modelarea Matematic a Bolilor Infecioase

2.1 Fundamente de epidemiologie

2.1.1 Definiii i concepte fundamentale

Epidemiologia este tiina care studiaz distribuia, determinanii i controlul strilor sau evenimentelor legate de sntate în populaii definite [1]. În sens mai specific, *epidemiologia infecioas* se ocup cu dinamica transmiterii bolilor cauzate de ageni patogeni biologici (virusuri, bacterii, parazii) în populaii umane sau animale.

Definiie 2.1 (Epidemic, pandemic, endemic)**.** O boal infecioas se numete **endemic** dac persist la nivel constant într-o regiune geografic definit. Se numete **epidemie** dac numrul de cazuri depete clar nivelul endemic ateptat, i **pandemie** dac afecteaz mai multe continente sau o mare parte a globului.

Definiie 2.2 (Inciden i prevalen)**. Incidena** reprezint numrul de cazuri noi ale unei boli aprute într-o populaie de risc, într-un interval de timp specificat. Se exprim ca rat:

$$\text{Inciden} = \frac{\text{cazuri noi în intervalul } [t_1, t_2]}{\text{populaie la risc} \times (t_2 - t_1)}. \tag{2.1}$$

Prevalena reprezint proporia indivizilor afectai la un moment dat:

$$\text{Prevalen} = \frac{\text{cazuri existente la momentul } t}{\text{populaie total la momentul } t}. \tag{2.2}$$

2.1.2 Parametrii fundamentali ai transmiterii

Definiie 2.3 (Intervalul serial i intervalul de generaie)**. Intervalul serial** este durata medie dintre apariia simptomelor la cazul primar i la cazul secundar generat de acesta. **Intervalul de generaie** este durata medie dintre infecia cazului primar i infecia unui caz secundar.

Definiie 2.4 (Perioada de incubaie)**. Perioada de incubaie** este intervalul de timp dintre momentul infectrii i apariia primelor simptome. Este distinct de *perioada de laten*, care reprezint intervalul dintre infecie i debutul infeciozitii.

Definiie 2.5 (Numrul de reproducere de baz $\mathcal{R}_0$)**.** Numrul de reproducere de baz $\mathcal{R}_0$ este definit ca numrul mediu de infecii secundare produse de un singur individ infectat introdus într-o populaie complet susceptibil, pe durata perioadei sale infecioase [13]. Este cel mai important parametru al epidemiologiei matematice, deoarece determin potenialul epidemic al unui agent patogen.

O consecin imediat a Teorem 2.5 este existena unui prag epidemic. Dac $\mathcal{R}_0 < 1$, fiecare caz infectat produce în medie mai puin de un caz nou, iar epidemia se stinge spontan. Dac $\mathcal{R}_0 > 1$, numrul de cazuri crete în faza iniial, de regul cu profil aproximativ exponenial. Cazul $\mathcal{R}_0 = 1$ separ cele dou regimuri i corespunde unei stri de echilibru instabil.

Numrul de reproducere *efectiv* (sau în timp real) $\mathcal{R}(t)$ generalizeaz $\mathcal{R}_0$ pentru o populaie parial imunizat:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N}, \tag{2.3}$$

unde $S(t)$ este numrul de indivizi susceptibili la momentul t i N este mrimea populaiei totale.

2.2 Modele compartimentale în epidemiologie

2.2.1 Cadrul general

Modelele compartimentale reprezint clasa fundamental de modele matematice în epidemiologie [2]. Principiul lor const în împrirea populaiei în *compartimente* care descriu stri epidemiologice distincte, iar tranziia dintre compartimente este guvernat de un sistem de ecuaii difereniale ordinare (EDO).

Fie N mrimea total a populaiei (presupus constant). Fiecare compartiment $X_i(t)$ reprezint numrul de indivizi în starea i la momentul t, cu $\sum_i X_i(t) = N$. Dinamica tranziiilor este descris de sistemul:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t, \boldsymbol{\theta}), \quad \mathbf{x}(0) = \mathbf{x}_0, \tag{2.4}$$

unde $\mathbf{x} = (X_1, \ldots, X_n)^\intercal$, $\boldsymbol{\theta}$ este vectorul parametrilor i $\mathbf{f}$ descrie fluxurile nete dintre compartimente.

2.2.2 Modelul SIS

Modelul **SIS** (Susceptibili-Infectai-Susceptibili) descrie infecii în care recuperarea nu confer imunitate de durat, permiând re-infectarea:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \gamma I, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.5}$$

Parametrul $\gamma > 0$ reprezint rata de recuperare ($1/\gamma$ este durata medie a perioadei infecioase). Pentru modelul SIS, $\mathcal{R}_0 = \beta/\gamma$. Dac $\mathcal{R}_0 \leq 1$, echilibrul fr boal $I^* = 0$ este stabil global; dac $\mathcal{R}_0 > 1$, sistemul admite un echilibru endemic stabil $I^* = N(1 - 1/\mathcal{R}_0) > 0$.

2.2.3 Modelul SEIR

Modelul **SEIR** introduce compartimentul E (Expui / în perioad de laten), care ine cont de faptul c indivizii infectai nu sunt imediat infecioi:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N}, \\ \frac{dE}{dt} = \quad \frac{\beta SI}{N} - \sigma E, \\ \frac{dI}{dt} = \quad \sigma E - \gamma I, \\ \frac{dR}{dt} = \quad \gamma I, \end{cases} \tag{2.6}$$

unde $\sigma = 1/T_E$ este rata de tranziie din starea expus în starea infecioas (T_E = durata medie de incubaie). Numrul de reproducere de baz pentru modelul SEIR este $\mathcal{R}_0 = \beta/\gamma$, identic cu cel al modelului SIR.

2.2.4 Modelul SIRS

Modelul **SIRS** generalizeaz modelul SIR prin imunitate temporar (indivizii recuperai pot deveni din nou susceptibili dup un interval de timp):

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \xi R, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} = \quad \gamma I - \xi R, \end{cases} \tag{2.7}$$

unde $\xi = 1/T_R$ este rata de pierdere a imunitii (T_R = durata medie a imunitii). Dac $\mathcal{R}_0 > 1$, modelul SIRS admite un echilibru endemic stabil.

2.3 Modelul SIDARTHE

Modelul SIDARTHE, propus pentru analiza epidemiei COVID-19 [14], separ dou dimensiuni care lipsesc din SIR: statutul de diagnosticare i severitatea clinic. Într-un sistem de raportare real, persoanele infectate dar nedepistate, persoanele diagnosticate, cazurile simptomatice i cazurile severe nu au aceeai contribuie la transmitere i nu sunt observate cu aceeai probabilitate. Sistemul implementat în aceast lucrare folosete varianta **SIDARTHE**, care include explicit compartimentul de decese.

Compartimentele modelului sunt S (susceptibili), I (infectai nedepistai), D (diagnosticai asimptomatici), A (simptomatici nedepistai), R (simptomatici diagnosticai), T (cazuri severe), H (vindecai) i E (extini / decedai). Compartimentul E este alimentat din T prin rata de mortalitate τ, ceea ce permite analiza separat a vindecrii i a deceselor.

$$\frac{dS}{dt} = -S \cdot \frac{\alpha I + \beta D + \gamma A + \delta R}{N}, \tag{2.8}$$

$$\frac{dI}{dt} = S \cdot \frac{\alpha I + \beta D + \gamma A + \delta R}{N} - (\varepsilon + \zeta + \lambda) I, \tag{2.9}$$

$$\frac{dD}{dt} = \varepsilon I - (\eta + \rho) D, \tag{2.10}$$

$$\frac{dA}{dt} = \zeta I - (\theta + \mu + \kappa) A, \tag{2.11}$$

$$\frac{dR}{dt} = \eta D + \theta A - (\nu + \xi) R, \tag{2.12}$$

$$\frac{dT}{dt} = \mu A + \nu R - (\sigma + \tau) T, \tag{2.13}$$

$$\frac{dH}{dt} = \lambda I + \rho D + \kappa A + \xi R + \sigma T, \tag{2.14}$$

$$\frac{dE}{dt} = \tau T. \tag{2.15}$$

Parametrii α, β, γ i δ controleaz contribuia la transmitere a compartimentelor infecioase. Parametrii ε, ζ i λ descriu ieirile din starea infectat nedepistat: diagnosticare, apariia simptomelor i vindecare fr diagnostic. Restul parametrilor descriu tranziiile ctre forme diagnosticate, severe sau vindecate. Spre deosebire de SIR, unde aproape toat interpretarea se concentreaz în raportul β/γ, SIDARTHE distribuie dinamica pe un numr mai mare de rate; de aceea, analiza numeric i sensibilitatea la parametri devin mai importante decât obinerea unei formule scalare simple.

Conservarea populaiei se verific prin însumarea ecuaiilor din (2.15). Toi termenii interni se anuleaz, iar relaia devine

$$\frac{d}{dt}(S + I + D + A + R + T + H + E) = 0. \tag{2.16}$$

Prin urmare, pentru condiii iniiale nenegative, suma compartimentelor rmâne egal cu populaia total N. Pozitivitatea soluiei se obine prin acelai argument folosit la modelele compartimentale clasice: pe frontiera fiecrui compartiment, derivata nu poate împinge soluia în regiunea negativ, deoarece termenii de ieire dispar când variabila respectiv este zero.

2.4 Modelul SIR ca reper secundar

2.4.1 Ipoteze i derivarea ecuaiilor din principii prime

Modelul SIR, formulat de Kermack i McKendrick [3], se bazeaz pe o serie de ipoteze simplificatoare care trebuie înelese explicit:

1. **Populaie închis i omogen:** Nu exist nateri, decese naturale sau migraii; $N = \text{const}$. Fiecare individ are aceeai probabilitate de a intra în contact cu orice alt individ (amestecare omogen).

2. **Transmitere prin aciune de mas:** Rata de contact eficient (productoare de infecie) este proporional cu produsul $S \cdot I$.

3. **Imunitate permanent:** Un individ recuperat nu poate fi re-infectat; compartimentul R este absorbant.

4. **Durata infecioas exponenial distribuit:** Rata de recuperare γ este constant, iar durata medie a perioadei infecioase este $1/\gamma$.

5. **Fr mortalitate indus de boal:** Sau, echivalent, mortalitatea specific bolii este neglijabil fa de dinamica transmiterii.

Derivarea termenului de infecie. Considerm c în unitatea de timp, un individ susceptibil are în medie c contacte cu ali indivizi alei uniform din populaie. Probabilitatea ca un contact arbitrar s fie cu un individ infectat este I/N, iar probabilitatea de transmitere la contact eficient este p. Astfel, rata de infecie a unui susceptibil este $c \cdot p \cdot I/N = \beta I/N$, unde $\beta = cp$ este parametrul de transmitere. Fluxul total de noi infecii pe unitate de timp este $(\beta I/N) \cdot S$.

Sistemul de ecuaii difereniale ordinare al modelului SIR este:

$$\boxed{\begin{aligned} \frac{dS}{dt} &= -\frac{\beta SI}{N}, \\ \frac{dI}{dt} &= \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, \end{aligned}} \tag{2.17}$$

cu condiii iniiale $S(0) = S_0 > 0$, $I(0) = I_0 > 0$, $R(0) = R_0 \geq 0$ i $S_0 + I_0 + R_0 = N$.

Diagrama de flux a compartimentelor este prezentat în Fig. 2.1.

2.4.2 Proprieti matematice fundamentale

Înainte de a analiza comportamentul calitativ al soluiilor, stabilim proprietile matematice eseniale: existena i unicitatea soluiei, conservarea populaiei totale i invariana domeniului pozitiv.

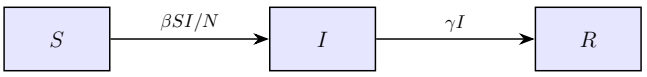


Figura 2.1: Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezint rata de noi infecii, iar γI reprezint rata de recuperare.

Teorem 2.6 (Existen i unicitate global)**.** *Fie $\beta, \gamma > 0$, $N > 0$ i $(S_0, I_0, R_0) \in \mathbb{R}^3$ cu $S_0, I_0 > 0$, $R_0 \geq 0$. Atunci sistemul* (2.17) *admite o unic soluie global $(S, I, R) \in C^1([0, \infty); \mathbb{R}^3)$.*

Demonstraie. Definim $\mathbf{x} = (S, I, R)^\intercal$ i $\mathbf{f}(\mathbf{x}) = (-\beta SI/N,\ \beta SI/N - \gamma I,\ \gamma I)^\intercal$. Funcia $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^3$ este de clas C^∞ (polinomul de gradul doi în S i I este neted), în particular continuu difereniabil. Prin *teorema Picard-Lindelöf* (Cauchy-Lipschitz), exist o soluie local unic pe un interval $[0, T_{\max})$.

Existena *global* ($T_{\max} = \infty$) se stabilete prin mrginirea soluiei. Deoarece $S(t), I(t), R(t) \geq 0$ (Teorema 2.8) i $S + I + R = N$ (Teorema 2.7), rezult c $\|\mathbf{x}(t)\| \leq N\sqrt{3}$ pentru orice $t \in [0, T_{\max})$. Soluia mrginit nu poate prezenta blow-up în timp finit, deci $T_{\max} = \infty$. □

Teorem 2.7 (Conservarea populaiei totale)**.** *Pentru orice soluie a sistemului* (2.17)*,*

$$S(t) + I(t) + R(t) = N \quad \forall\, t \geq 0. \tag{2.18}$$

Demonstraie. Calculm derivata sumei:

$$\begin{aligned} \frac{d}{dt}(S + I + R) &= \frac{dS}{dt} + \frac{dI}{dt} + \frac{dR}{dt} \\ &= -\frac{\beta SI}{N} + \left(\frac{\beta SI}{N} - \gamma I\right) + \gamma I \\ &= 0. \end{aligned}$$

Prin integrare, $S(t)+I(t)+R(t) = S_0+I_0+R_0 = N$. □

Teorem 2.8 (Pozitivitate i mrginire)**.** *Dac $S_0 > 0$, $I_0 > 0$, $R_0 \geq 0$, atunci:*

$$S(t) > 0, \quad I(t) > 0, \quad R(t) \geq 0, \quad \forall\, t > 0. \tag{2.19}$$

În particular, $S(t), I(t), R(t) \in [0, N]$ pentru orice $t \geq 0$.

Demonstraie. **(a) Pozitivitatea lui S:** Din ecuaia $dS/dt = -(\beta I/N)S$, aceasta este o ecuaie diferenial liniar omogen în S cu coeficient $-\beta I(t)/N$. Soluia este:

$$S(t) = S_0 \cdot \exp\left(-\frac{\beta}{N}\int_0^t I(s)\, ds\right). \tag{2.20}$$

Deoarece integrala este finit i $S_0 > 0$, rezult $S(t) > 0$ pentru orice $t \geq 0$ (presupunând $I(s) \geq 0$, demonstrat mai jos).

(b) Pozitivitatea lui I: Rescrierea ecuaiei a doua ca $dI/dt = (\beta S(t)/N - \gamma)\, I$ arat c aceasta este liniar în

I:

$$I(t) = I_0 \cdot \exp\left(\int_0^t \left[\frac{\beta S(\tau)}{N} - \gamma \right] d\tau \right) > 0, \qquad (2.21)$$

deoarece $I_0 > 0$ i exponeniala este strict pozitiv.

(c) Non-negativitatea lui R: Din $dR/dt = \gamma I(t) > 0$ i $R_0 \geq 0$, funcia R este strict cresctoare, deci $R(t) > R_0 \geq 0$.

Mrginirea urmeaz din Teorema 2.7: $S(t) \leq N$, $I(t) \leq N$, $R(t) \leq N$. □

Observaie 2.9. Teorema 2.8 implic c domeniul pozitiv

$$\Omega = \{(S, I, R) \in \mathbb{R}^3_{\geq 0} : S + I + R = N\} \qquad (2.22)$$

este *invariant pozitiv* fa de fluxul sistemului (2.17). Prin urmare, este suficient s studiem comportamentul soluiilor pe Ω, care este un simplex în $\mathbb{R}^3$.

Teorem 2.10 (Monotonia compartimentelor)**.** *De-a lungul oricrei soluii a sistemului* (2.17)*:*

(i) $S(t)$ *este strict descresctoare;*

(ii) $R(t)$ *este strict cresctoare;*

(iii) $I(t)$ *atinge un singur maxim în* $t^* = \inf\{t > 0 : S(t) = \gamma N/\beta\}$ *(dac* $S_0 > \gamma N/\beta$*); altfel este descresctoare.*

Demonstraie. (i) $dS/dt = -\beta SI/N < 0$ deoarece $S > 0$, $I > 0$. *(ii)* $dR/dt = \gamma I > 0$. *(iii)* $dI/dt = (\beta S/N - \gamma)I > 0 \Leftrightarrow S > \gamma N/\beta$. Deoarece S este strict descresctoare, exist cel mult un moment t^* când $S(t^*) = \gamma N/\beta$, care este maximul lui I. □

2.4.3 Analiza echilibrelor i stabilitate

Punctele de echilibru ale sistemului (2.17) sunt obinute din condiia $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Ecuaiile conduc la:

$$-\frac{\beta S^* I^*}{N} = 0, \quad \frac{\beta S^* I^*}{N} - \gamma I^* = 0, \quad \gamma I^* = 0. \qquad (2.23)$$

Din a treia ecuaie, $I^* = 0$. Substituind, obinem c orice punct de forma $(S^*, 0, N - S^*)$ cu $S^* \in [0, N]$ este echilibru. Mulimea echilibrelor este:

$$\mathcal{E} = \{(S^*, 0, N - S^*) : S^* \in [0, N]\} \subset \Omega. \qquad (2.24)$$

Observaie 2.11. Spre deosebire de modelele SIS i SIRS, modelul SIR nu admite echilibru endemic cu $I^* > 0$. Aceasta reflect faptul c, pe termen lung, epidemia se stinge întotdeauna (dei poate infecta o fracie mare din populaie înainte de stingere).

Analiza stabilitii locale. Deoarece $R = N - S - I$, sistemul se reduce la dou ecuaii (eliminând R):

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N}, \\ \frac{dI}{dt} = \dfrac{\beta SI}{N} - \gamma I. \end{cases} \qquad (2.25)$$

Jacobianul sistemului (2.25) evaluat la un echilibru $(S^*, 0)$ este:

$$J(S^*, 0) = \begin{pmatrix} 0 & -\beta S^*/N \\ 0 & \beta S^*/N - \gamma \end{pmatrix}. \qquad (2.26)$$

Valorile proprii sunt $\lambda_1 = 0$ i $\lambda_2 = \beta S^*/N - \gamma$.

Teorem 2.12 (Stabilitatea echilibrului fr boal)**.** *Echilibrul fr boal* $(S^*, 0)$ *este stabil nelinear dac* $\beta S^*/N < \gamma$*, adic dac* $S^*/N < 1/\mathcal{R}_0$*, i este instabil dac* $\beta S^*/N > \gamma$*.*

Demonstraie. Valorea proprie $\lambda_2 = \beta S^*/N - \gamma$ determin stabilitatea pe direcia I. Deoarece $\lambda_1 = 0$, analiza liniar este degenerat i trebuie completat cu argumente neliniare. Totui, din Teorema 2.10, $I(t)$ este descresctoare dac $S(t) < \gamma N/\beta$ pentru tot $t > 0$, ceea ce se întâmpl când $S_0 < \gamma N/\beta$, implicând stingerea epidemiei. □

2.4.4 Numrul de reproducere de baz $\mathcal{R}_0$

Definiie prin matricea generaiei urmtoare

Metoda *matricei generaiei urmtoare* (next-generation matrix), formalizat de Diekmann, Heesterbeek i Metz [13] i extins de van den Driessche i Watmough [15], furnizeaz o procedur sistematic pentru calculul $\mathcal{R}_0$ în modele compartimentale de complexitate arbitrar.

Fie $\mathbf{x}_I$ vectorul compartimentelor infectate. Scriem:

$$\frac{d\mathbf{x}_I}{dt} = \mathcal{F}(\mathbf{x}) - \mathcal{V}(\mathbf{x}), \qquad (2.27)$$

unde $\mathcal{F}_i$ reprezint rata de apariie a noilor infecii în compartimentul i, iar $\mathcal{V}_i$ reprezint rata net de ieire din compartimentul i prin toate celelalte procese.

Matricele Jacobiene evaluate la echilibrul fr boal $\mathbf{x}_0$ sunt:

$$F = \left[\frac{\partial \mathcal{F}_i}{\partial x_j} \right]_{\mathbf{x}_0}, \quad V = \left[\frac{\partial \mathcal{V}_i}{\partial x_j} \right]_{\mathbf{x}_0}. \qquad (2.28)$$

Matricea generaiei urmtoare este $K = FV^{-1}$, iar:

$$\mathcal{R}_0 = \rho(K), \qquad (2.29)$$

unde $\rho(\cdot)$ este raza spectral (cea mai mare valoare proprie în modul).

Calculul $\mathcal{R}_0$ pentru modelul SIR

Pentru modelul SIR, singurul compartiment infectat este $\{I\}$. La echilibrul fr boal $(S, I, R) = (N, 0, 0)$:

$$\mathcal{F}(I) = \frac{\beta SI}{N} \implies F = \left[\frac{\partial \mathcal{F}}{\partial I} \right]_{S=N} = \frac{\beta N}{N} = \beta. \qquad (2.30)$$

$$\mathcal{V}(I) = \gamma I \implies V = \gamma. \qquad (2.31)$$

Matricea K este scalarul:

$$K = FV^{-1} = \frac{\beta}{\gamma}. \qquad (2.32)$$

Prin urmare:

$$\boxed{\mathcal{R}_0 = \frac{\beta}{\gamma}.} \tag{2.33}$$

Interpretarea biologic este direct: $1/\gamma$ este durata medie a perioadei infecioase, iar β este numrul mediu de contacte eficiente pe unitate de timp; produsul lor d numrul de infecii secundare produse pe durata infeciozitii.

Teorem 2.13 (Pragul epidemic)**.** *Considerm o populaie iniial complet susceptibil ($S_0 \approx N$). Atunci:*

(i) Dac $\mathcal{R}_0 > 1$, numrul de infectai crete iniial exponenial i are loc o epidemie ($I(t)$ atinge un maxim $I_{\max} > I_0$).

(ii) Dac $\mathcal{R}_0 \leq 1$, $I(t)$ este imediat descresctoare i nu are loc nicio epidemie.

Demonstraie. Din $dI/dt|_{t=0} = I_0(\beta S_0/N - \gamma) \approx I_0(\beta - \gamma)$ (pentru $S_0 \approx N$), concluzia urmeaz din semnul expresiei $\beta - \gamma$:

$$\left.\frac{dI}{dt}\right|_{t=0} > 0 \iff \beta > \gamma \iff \frac{\beta}{\gamma} > 1 \iff \mathcal{R}_0 > 1. \tag{2.34}$$

□

2.4.5 Pragul de imunizare în turm (herd immunity)

Definiie 2.14 (Imunitate de grup)**.** Imunitatea de grup (*herd immunity*) se refer la protecia indirect a persoanelor susceptibile, generat atunci când o fracie suficient de mare din populaie este imunizat (fie natural, fie prin vaccinare), astfel încât transmiterea susinut a agentului patogen devine imposibil.

Epidemia se stinge când numrul de reproducere efectiv devine subunitar:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N} < 1 \iff \frac{S(t)}{N} < \frac{1}{\mathcal{R}_0}. \tag{2.35}$$

Dac o fracie p din populaie este imunizat (cu $S/N = 1 - p$), condiia de blocare a epidemiei devine:

$$(1-p) \cdot \mathcal{R}_0 < 1 \iff p > 1 - \frac{1}{\mathcal{R}_0}. \tag{2.36}$$

Definiie 2.15 (Pragul de imunizare în turm)**.** Fracia minim a populaiei care trebuie imunizat pentru a preveni propagarea epidemiei este:

$$p_c = 1 - \frac{1}{\mathcal{R}_0}. \tag{2.37}$$

Exemplu 2.16. Pentru rujeol, $\mathcal{R}_0 \approx 12$–18, deci $p_c \approx 92$–94%. Pentru COVID-19 (tulpina original), $\mathcal{R}_0 \approx 2$–3, deci $p_c \approx 50$–67%. Variantele ulterioare (Delta, Omicron) au $\mathcal{R}_0$ mai mare, crescând corespunztor pragul de imunizare.

2.4.6 Ecuaia dimensiunii finale a epidemiei

O caracteristic remarcabil a modelului SIR este c permite calculul analitic al *dimensiunii finale* a epidemiei — fracia total din populaie care este infectat în cursul epidemiei — fr a rezolva explicit sistemul diferenial.

Teorem 2.17 (Ecuaia dimensiunii finale)**.** *Fie $z = 1 - S(\infty)/N$ fracia din populaie infectat cumulativ pe durata epidemiei, presupunând $S_0 = N$ i $R_0 = 0$. Atunci z este unica rdcin pozitiv a ecuaiei transcendentale:*

$$z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.38}$$

Demonstraie. Împrind ecuaia pentru S la ecuaia pentru R:

$$\frac{dS}{dR} = \frac{-\beta SI/N}{\gamma I} = -\frac{\beta}{\gamma N} S = -\frac{\mathcal{R}_0}{N} S. \tag{2.39}$$

Aceast ecuaie separabil în S i R se integreaz direct:

$$\int_{S_0}^{S(t)} \frac{dS'}{S'} = -\frac{\mathcal{R}_0}{N} \int_0^{R(t)} dR' \implies \ln \frac{S(t)}{S_0} = -\frac{\mathcal{R}_0}{N} R(t). \tag{2.40}$$

Deci $S(t) = S_0 \exp(-\mathcal{R}_0 R(t)/N)$.

La $t \to \infty$, $I(\infty) = 0$ (demonstrat mai jos), deci $R(\infty) = N - S(\infty)$ (pentru $R_0 = 0$). Substituind cu $S_0 = N$:

$$S(\infty) = N e^{-\mathcal{R}_0 (N - S(\infty))/N}. \tag{2.41}$$

Notând $z = 1 - S(\infty)/N$ i împrind prin N:

$$1 - z = e^{-\mathcal{R}_0 z} \implies z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.42}$$

C $I(\infty) = 0$**:** Funcia $R(t)$ este monoton cresctoare i mrginit de N, deci $\lim_{t\to\infty} R(t)$ exist. Dac $I(\infty) = \ell > 0$, atunci $R'(t) = \gamma I(t) \geq \gamma \ell > 0$ pentru t suficient de mare, ceea ce implic $R(t) \to \infty$, contradicie. Deci $I(\infty) = 0$.

Unicitatea rdcinii pozitive: Fie $g(z) = 1 - e^{-\mathcal{R}_0 z} - z$. Avem $g(0) = 0$, $g'(0) = \mathcal{R}_0 - 1 > 0$ (dac $\mathcal{R}_0 > 1$), i $g(1) = 1 - e^{-\mathcal{R}_0} - 1 = -e^{-\mathcal{R}_0} < 0$. Deoarece $g''(z) = -\mathcal{R}_0^2 e^{-\mathcal{R}_0 z} < 0$, funcia g este concav, deci are exact o singur rdcin în $(0, 1)$. □

Observaie 2.18. Ecuaia (2.38) este *transcendental* i nu admite soluie în form închis pentru $\mathcal{R}_0 \neq 1$. Se rezolv numeric, de exemplu prin metoda Newton-Raphson sau prin iteraie de punct fix.

Corolar 2.19 (Rata de atac final)**.** *Dac $\mathcal{R}_0 > 1$, rata de atac final z^* satisface $z^* \in (0, 1)$ i crete monoton cu $\mathcal{R}_0$. Limita $\lim_{\mathcal{R}_0 \to \infty} z^* = 1$.*

2.4.7 Extensii ale modelului SIR

Modelul SIR de baz poate fi extins în mai multe direcii. Introducerea naterilor i mortalitii naturale permite studiul echilibrelor endemice pe termen lung. Structura

pe vârste poate fi inclus prin matrice de contact, iar propagarea geografic prin modele cu patch-uri sau ecuaii difereniale pariale. Pentru populaii mici sau pentru fazele incipiente ale unei epidemii, modelele stochastice devin mai potrivite decât formularea determinist. Interveniile precum vaccinarea, izolarea i tratamentul se pot introduce ca fluxuri suplimentare între compartimente.

2.5 Metode Numerice pentru Ecuaii Difereniale Ordinare

2.5.1 Clasificarea metodelor numerice

Soluia analitic a sistemului SIR nu exist în form închis (cu excepia cazurilor degenerate), ceea ce face indispensabil rezolvarea numeric. O metod numeric pentru problema cu valori iniiale (PVI):

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(t_0) = \mathbf{y}_0, \tag{2.43}$$

genereaz o secven de aproximaii $\mathbf{y}_n \approx \mathbf{y}(t_n)$ la punctele $t_n = t_0 + nh$.

Metodele numerice pot fi privite din mai multe unghiuri. Metodele explicite calculeaz direct $\mathbf{y}_{n+1}$ din valorile anterioare, în timp ce metodele implicite includ necunoscuta i în membrul drept, necesitând rezolvarea unui sistem la fiecare pas. Metodele un-pas folosesc doar starea curent, pe când metodele multi-pas folosesc i stri anterioare. Ordinul de convergen p determin comportamentul erorii globale, $\|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^p)$. În sfârit, metodele cu pas adaptiv ajusteaz h pe baza unei estimri a erorii locale, spre deosebire de schemele cu pas fix.

2.5.2 Metoda Euler explicit

Metoda Euler (1768) este cea mai simpl metod numeric un-pas, obinut prin truncherea seriei Taylor la ordinul întâi:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \cdot \mathbf{f}(t_n, \mathbf{y}_n). \tag{2.44}$$

Teorem 2.20 (Eroarea global a metodei Euler)**.** *Dac* $\mathbf{f}$ *este Lipschitz în* $\mathbf{y}$ *cu constanta* L *i* $\mathbf{y} \in C^2([t_0, T])$*, eroarea global a metodei Euler satisface:*

$$\max_{n:t_n \le T} \|\mathbf{y}(t_n) - \mathbf{y}_n\| \le \frac{h}{2L} \max_{t\in[t_0,T]} \|\mathbf{y}''(t)\| \cdot \left(e^{L(T-t_0)} - 1\right) = \mathcal{O}(h). \tag{2.45}$$

Eroarea local de trunchiere (pe un singur pas) este $\mathcal{O}(h^2)$, deci metoda are ordinul local 2 i ordinul global 1.

Aplicat sistemului SIR cu $\mathbf{y}_n = (S_n, I_n, R_n)^\intercal$, schema Euler este:

Stabilitate. Analiza stabilitii metodei Euler pe ecuaia test $y' = \lambda y$ ($\lambda \in \mathbb{C}$) conduce la factorul de amplificare $z = 1 + h\lambda$. Metoda este stabil dac $|1 + h\lambda| \le 1$, ceea ce definete domeniul de stabilitate (disc de raz 1 centrat în -1 în planul $h\lambda$). Aceast restricie limiteaza pasul de integrare h, în special pentru probleme *stiff*.

Algoritm 2.1 Metoda Euler pentru sistemul SIR

```
Input:  beta, gamma, N, S0, I0, R0, t0,
     tf, h
Output: Secventa (tn, Sn, In, Rn)
    pentru tn in [t0, tf]

(S, I, R) <- (S0, I0, R0);  t <- t0
while t < tf do
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    S  <- S + h * dS
    I  <- I + h * dI
    R  <- R + h * dR
    t  <- t + h
    Inregistreaza (t, S, I, R)
end while
```

2.5.3 Metoda Runge-Kutta de ordinul 4 (RK4)

Metodele Runge-Kutta sunt metode un-pas explicite care realizeaz mai multe evaluri ale funciei $\mathbf{f}$ la fiecare pas, atingând ordine superioare fr a recurge la derivate de ordin înalt ale soluiei.

Derivare i formul

Metoda RK4 clasic (cu 4 stadii) este dat de:

$$\begin{aligned}
\mathbf{k}_1 &= h \cdot \mathbf{f}(t_n, \mathbf{y}_n), \\
\mathbf{k}_2 &= h \cdot \mathbf{f}\left(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_1}{2}\right), \\
\mathbf{k}_3 &= h \cdot \mathbf{f}\left(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_2}{2}\right), \\
\mathbf{k}_4 &= h \cdot \mathbf{f}(t_n + h,\ \mathbf{y}_n + \mathbf{k}_3), \\
\mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right).
\end{aligned} \tag{2.46}$$

Coeficienii metodei sunt reprezentai compact prin *tabloul Butcher*:

$$\begin{array}{c|cccc}
0 & & & & \\
1/2 & 1/2 & & & \\
1/2 & 0 & 1/2 & & \\
1 & 0 & 0 & 1 & \\
\hline
 & 1/6 & 1/3 & 1/3 & 1/6
\end{array} \tag{2.47}$$

Teorem 2.21 (Ordinul metodei RK4)**.** *Metoda* (2.46) *are ordinul de convergen* $p = 4$*: dac* $\mathbf{f} \in C^4$ *i* $\mathbf{y} \in C^5([t_0, T])$*, atunci eroarea global satisface:*

$$\max_n \|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^4). \tag{2.48}$$

Schi de demonstraie. Eroarea local de trunchiere τ_n se

obine substituind soluia exact $\mathbf{y}(t_n)$ în schema (2.46) i expandând în serie Taylor. Se poate arta c primele 4 condiii de ordin (condiiile de arbore Butcher pentru $p = 4$) sunt satisfcute de coeficienii (2.47), implicând $\tau_n = \mathcal{O}(h^5)$. Eroarea global este $\mathcal{O}(h^4)$ prin acumularea $\sim 1/h$ de pai. Detalii complete în [16]. □

Algoritm 2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR

```
Input:  f, y0, t0, tf, h
Output: Secventa (tn, yn)

y <- y0;  t <- t0
while t < tf do
    k1 <- h * f(t,        y           )
    k2 <- h * f(t + h/2, y + k1/2     )
    k3 <- h * f(t + h/2, y + k2/2     )
    k4 <- h * f(t + h,    y + k3      )
    y  <- y + (k1 + 2*k2 + 2*k3 + k4) /
     6
    t  <- t + h
end while
```

2.5.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince)

Metodele cu pas fix impun utilizatorului alegerea unui h adecvat, care este o sarcin dificil în practic. Metodele *adaptive* ajusteaz automat pasul h pe baza unei estimri a erorii locale, asigurând c eroarea rmâne sub o toleran prestabilit ε.

Metode Runge-Kutta încorporate

O *pereche RK încorporat* de ordine p i $p+1$ calculeaz doi estimatori pentru $\mathbf{y}_{n+1}$ folosind *acelai* set de evaluri:

$$\hat{\mathbf{y}}_{n+1} = \mathbf{y}_n + h\sum_{i=1}^{s} b_i \mathbf{k}_i \quad \text{(ordinul } p\text{)}, \tag{2.49}$$

$$\tilde{\mathbf{y}}_{n+1} = \mathbf{y}_n + h\sum_{i=1}^{s} \tilde{b}_i \mathbf{k}_i \quad \text{(ordinul } p+1\text{)}, \tag{2.50}$$

cu $\mathbf{k}_i = \mathbf{f}(t_n + c_i h, \mathbf{y}_n + h\sum_{j<i} a_{ij}\mathbf{k}_j)$. Estimatorul de eroare local este:

$$\mathbf{e}_{n+1} = \tilde{\mathbf{y}}_{n+1} - \hat{\mathbf{y}}_{n+1} = h\sum_{i=1}^{s}(b_i - \tilde{b}_i)\mathbf{k}_i. \tag{2.51}$$

Metoda Dormand-Prince (DOPRI5 / RK45)

Perechea **Dormand-Prince** [17], implementat ca `ode45` în MATLAB, `RK45` în SciPy (Python), `Tsit5` în Julia i `lsoda`/`ode45` în R (pachetul `deSolve`), utilizeaz 6 evaluri de funcie i are ordinul principal 5 cu estimator de eroare de ordinul 4.

Tabloul Butcher complet al metodei Dormand-Prince este:

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	−56/15	32/9				
8/9	19372/6561	−25360/2187	64448/6561	−212/729			
1	9017/3168	−355/33	46732/5247	49/176	−5103/18656		
1	35/384	0	500/1113	125/192	−2187/6784	11/84	
$\hat{b}$	35/384	0	500/1113	125/192	−2187/6784	11/84	0
$\tilde{b}$	5179/57600	0	7571/16695	393/640	−92097/339200	187/2100	1/40

(2.52)

Control adaptiv al pasului

Pasul h este actualizat dup fiecare pas pe baza normei erorii:

$$\text{err} = \left\| \frac{\mathbf{e}_{n+1}}{\text{atol} + \text{rtol} \cdot |\mathbf{y}_{n+1}|} \right\|_{\infty}, \tag{2.53}$$

unde atol i rtol sunt tolerane absolute i relative. Pasul urmtor este calculat prin formula:

$$h_{\text{nou}} = h \cdot \min\Big(f_{\max},\ \max\Big(f_{\min},\ S \cdot \text{err}^{-1/(p+1)}\Big)\Big), \tag{2.54}$$

cu factorul de siguran $S = 0.9$ i limitele $f_{\min} = 0.1$, $f_{\max} = 10$.

2.5.5 Stabilitate numeric i probleme stiff

Definiie 2.22 (Problem stiff)**.** O problem cu valori iniiale (2.43) este numit *stiff* dac valorile proprii ale Jacobianului $\partial\mathbf{f}/\partial\mathbf{y}$ au componente cu parte real negativ de magnitudine mare, impunând pai de integrare h extrem de mici pentru metodele explicite din motive de stabilitate, chiar dac soluia variaz lent.

Aplicaie la modelul SIR. Jacobianul sistemului SIR (2.25) este:

$$J(S, I) = \begin{pmatrix} -\beta I/N & -\beta S/N \\ \beta I/N & \beta S/N - \gamma \end{pmatrix}. \tag{2.55}$$

Valorile proprii depind de (S, I). La debutul epidemiei ($S \approx N$, I mic), valorile proprii sunt aproximativ 0 i $\beta - \gamma$, deci problema nu este stiff. Modelul SIR de baz nu este stiff pentru parametri tipici epidemiologici. Totui, extensiile cu mai multe compartimente i rate foarte diferite pot produce probleme stiff, necesitând metode implicite (Runge-Kutta implicit, BDF).

Observaie 2.23. Metodele implicite (de exemplu, metoda Euler implicit, Crank-Nicolson, Radau) au domenii de stabilitate nelimitate i sunt preferate pentru probleme stiff, în ciuda costului computaional superior (rezolvarea unui sistem nonlinear la fiecare pas).

2.6 Date Epidemiologice Surse i Preprocesare

2.6.1 Surse de date epidemiologice

Implementarea i validarea modelelor epidemiologice computaionale necesit date empirice de calitate. Sursele folosite frecvent includ rapoartele Organizaiei Mondiale a Sntii, seriile de supraveghere publicate de ECDC, setul de date COVID-19 al Johns Hopkins University CSSE i coleciile armonizate de Our World in Data. Aceast lucrare folosete dou seturi OWID complementare: setul COVID-19 (sursa principal, cu acoperire global pe 2020–2023) i setul mpox (focar 2022–2024, cu o dinamic complet diferit ca scar, durat infecioas i raportare). Ambele ofer serii temporale publice, comparabile între ri i uor de prelucrat într-un format comun.

2.6.2 Structura datelor i preprocesare

Datele epidemiologice brute conin frecvent valori lips, raportri retrospective, efecte de weekend i schimbri administrative ale unitilor geografice. Înainte de utilizarea lor într-un model compartimental, seriile trebuie ordonate temporal, corectate pentru valori imposibile i aduse la o granularitate compatibil cu pasul de integrare ales. În caz contrar, erorile de raportare pot fi confundate cu variaii reale ale dinamicii infeciei.

2.6.3 Estimarea parametrilor din date reale

Parametrii β i γ ai modelului SIR pot fi estimai din date observate prin minimizarea erorii ptratice dintre seria raportat i soluia numeric, prin estimare de verosimilitate maxim cu un model de zgomot adecvat sau prin inferen bayesian. Alegerea metodei depinde de natura datelor: pentru inciden zilnic, modelele Poisson sau binomial negativ sunt mai potrivite decât o eroare gaussian simpl, în timp ce inferena bayesian ofer o descriere explicit a incertitudinii.

Estimatorul $\hat{\gamma}$ poate fi obinut direct dac durata medie a perioadei infecioase este disponibil din studii clinice: $\hat{\gamma} = 1/T_{\text{infecios}}$. Parametrul β este mai dificil de estimat i depinde de comportamentul social, intervenii i variante ale agentului patogen.

Capitolul 3

Tehnologii i Instrumente Utilizate

3.1 Limbaje de programare pentru calcul tiinific

Alegerea limbajului de programare pentru implementarea modelelor matematice nu este neutr. Performana, expresivitatea i ecosistemul de biblioteci influeneaz atât timpul de dezvoltare, cât i comportamentul numeric al aplicaiei. În aceast lucrare, R, Julia i Python sunt tratate ca ecosisteme de calcul tiinific, nu doar ca sintaxe diferite pentru aceleai formule.

3.1.1 R

Istoric i filosofie

R este un limbaj de programare i mediu software pentru calcul statistic i grafic, descendent direct al limbajului S (Bell Laboratories, 1976). A fost creat de Ross Ihaka i Robert Gentleman la Universitatea din Auckland în 1993, iar în 1997 a fost adoptat ca proiect open-source de *R Development Core Team* [18]. Filosofia R este centrat pe date: vectorii, matricile i cadrele de date (data frames) sunt ceteni de prim clas, iar gramatica limbajului este proiectat pentru manipularea i analiza datelor.

Caracteristici tehnice relevante

R este un limbaj *interpretat*, dinamic tipizat, cu suport pentru programare funcional, procedural i orientat pe obiecte prin sistemele S3, S4 i R6. Pentru calcul numeric, avantajul su principal vine din vectorizarea nativ: operaiile pe vectori i matrice sunt executate frecvent în cod C sau Fortran, evitând buclele interpretate. În acelai timp, modelul de memorie cu garbage collector i copiere la scriere poate introduce costuri vizibile atunci când sunt manipulate structuri mari. Posibilitatea de a apela cod nativ prin `.C()`, `.Call()` i `.Fortran()` explic de ce multe pachete R performante ascund implementri compilate în spatele unei interfee statistice convenabile.

Pachete relevante pentru modele compartimentale

Pentru modelele compartimentale implementate, pachetul relevant este `deSolve` [4], care ofer funcia `ode()` i acces la metode consacrate precum `lsoda`, `ode45` i `rk4`. În jurul lui se pot folosi pachete precum `ggplot2` pentru vizualizare, `data.table` pentru prelucrare tabular i `bench` sau `microbenchmark` pentru msurarea timpilor de execuie. În implementarea sistemului, runner-ul R folosete `deSolve` pentru integrarea numeric, iar vizualizarea este lsat aplicaiei web.

3.1.2 Julia

Istoric i filosofie

Julia a fost creat la MIT de Jeff Bezanson, Alan Edelman, Stefan Karpinski i Viral B. Shah, cu versiunea 1.0 lansat în august 2018 [19]. Motivaia proiectului a fost depirea *problemei celor dou limbaje* (two-language problem): cercettorii scriu prototipuri în R/Python (productivitate ridicat, vitez sczut) i re-implementeaz codul critic în C/Fortran (vitez ridicat, productivitate sczut). Julia vizeaz ambele obiective simultan.

Compilare Just-In-Time i sistemul de tipuri

Julia utilizeaz compilare **JIT** (Just-In-Time) bazat pe LLVM, cu o particularitate fundamental: specializarea pe tipuri concrete. Când o funcie este apelat pentru prima dat cu argumente de tipuri specifice, compilatorul genereaz cod main optimizat pentru acele tipuri exacte. Apelurile ulterioare cu aceleai tipuri ruleaz direct codul compilat, apropiind performana de C.

Mecanismul de *multiple dispatch* permite definirea mai multor metode pentru aceeai funcie, în funcie de tipurile argumentelor. Împreun cu sistemul de tipuri parametric, acesta permite compilatorului s genereze cod specializat pentru cazurile folosite efectiv. Macrocomenzile completeaz acest model prin posibilitatea de a transforma codul la momentul parsrii, o tehnic utilizat frecvent în ecosistemul tiinific Julia.

DifferentialEquations.jl

Ecosistemul **SciML** (Scientific Machine Learning) al Julia conine pachetul `DifferentialEquations.jl` [5], un framework open-source matur pentru rezolvarea ecuaiilor difereniale. Pachetul include solver-e pentru ecuaii difereniale ordinare, pariale, stochastice i cu întârziere. Pentru probleme nestiff, metode precum `Tsit5`

i `Vern7/9` sunt folosite frecvent, iar pentru probleme stiff exist familii precum `Rodas`. Ecosistemul SciML integreaz i difereniere automat, ceea ce îl face relevant pentru estimarea parametrilor i identificarea sistemelor dinamice.

3.1.3 Python

Istoric i filosofie

Python a fost creat de Guido van Rossum i lansat în 1991 [20]. Filosofia sa, rezumat în *Zen of Python* (`PEP 20`), accentueaz lizibilitatea i simplitatea codului. Python a devenit limbajul dominant în tiina datelor i machine learning datorit ecosistemului su extins i comunitii masive.

Caracteristici tehnice i ecosistem tiinific

Python este un limbaj interpretat, dinamic tipizat, multi-paradigm. Principala limitare pentru calcul intensiv este **GIL** (Global Interpreter Lock), care previne execuia paralel a thread-urilor Python pure. Performana este recuperat prin biblioteci care ruleaz cod nativ C sau Fortran în afara GIL. NumPy ofer calcul vectorizat pe array-uri dense, SciPy [6] adaug algoritmic tiinific i modulul `scipy.integrate`, iar Pandas este folosit pe scar larg pentru date tabulare. Pentru sistemul dezvoltat aici, componenta esenial este `solve_ivp`, care expune metode precum RK45, DOP853, Radau, BDF i LSODA.

Funciile de rezolvare a EDO în SciPy

Interfaa `solve_ivp` din `scipy.integrate` este apelat printr-o funcie RHS care descrie modelul selectat. Exemplul urmtor folosete cazul SIR, deoarece are doar trei variabile de stare; pentru SIDARTHE, aceeai interfa primete un vector de opt compartimente i un dicionar mai mare de parametri.

```
from scipy.integrate import solve_ivp
import numpy as np

def sir_model(t, y, beta, gamma, N):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt =  beta * S * I / N - gamma * I
    dRdt =  gamma * I
    return [dSdt, dIdt, dRdt]

# Parametri
N = 1_000_000; beta = 0.3; gamma = 0.1
S0, I0, R0 = N - 1, 1, 0
t_span = (0, 160); t_eval = np.linspace(0, 160, 1601)

sol = solve_ivp(
    fun=lambda t, y: sir_model(t, y, beta, gamma, N),
    t_span=t_span,
    y0=[S0, I0, R0],
    method='RK45',
    t_eval=t_eval,
    rtol=1e-8,
    atol=1e-10
)
```

Listing 3.1: Interfaa solve_ivp din SciPy pentru un sistem compartimental SIR

3.1.4 Analiz comparativ a limbajelor

Table 3.1 sintetizeaz principalele caracteristici ale celor trei limbaje din perspectiva utilizrii pentru implementarea modelelor compartimentale implementate în sistem.

3.2 Baze de date distribuite YugabyteDB

3.2.1 Concepte de baze de date distribuite

O *baz de date distribuit* este o colecie de date distribuite fizic pe mai multe noduri (maini sau locaii geografice), gestionat de un sistem software care ofer transparena localizrii: aplicaiile interacioneaz cu baza de date ca i cum ar fi centralizat.

Definiie 3.1 (Teorema CAP)**.** Teorema CAP (Brewer, 2000) afirm c un sistem distribuit nu poate garanta simultan mai mult de dou dintre urmtoarele trei proprieti: *consistency*, adic nodurile observ aceeai stare logic a datelor; *availability*, adic cererile valide primesc rspuns chiar în condiii de degradare; i *partition tolerance*, adic sistemul continu s funcioneze în prezena unor întreruperi de comunicare între noduri.

Bazele de date distribuite precum YugabyteDB, CockroachDB i Spanner opteaz pentru **CP** (consistency + partition tolerance), sacrificând disponibilitatea perfect în favoarea coerenei datelor.

3.2.2 Arhitectura YugabyteDB

YugabyteDB [21] este o baz de date distribuit open-source, construit pentru a combina caracteristicile PostgreSQL cu scalabilitatea orizontal i tolerana la erori a sistemelor NoSQL. Arhitectura sa se bazeaz pe doi piloni principali:

1. Stratul de stocare DocDB. DocDB este motorul de stocare distribuit al YugabyteDB, bazat pe arhitectura **LSM-tree** (Log-Structured Merge-tree) derivat din **RocksDB**. Datele sunt stocate în *tablete* — uniti de sharding distribuite pe noduri. Fiecare tablet este replicat pe cel puin trei noduri prin protocolul Raft. Scrierile trec printr-un lider, sunt replicate ctre nodurile urmritoare i sunt confirmate dup ce o majoritate de replici a persistat modificarea.

Tabela 3.1: Comparaie a limbajelor de programare utilizate

Criteriu	**R**	**Julia**	**Python**
Licen	Open-source	Open-source	Open-source
Paradigm	Func./OOP	Multi	Multi
Compilare	Interpretat	JIT (LLVM)	Interpretat
Solver EDO	deSolve	DiffEq.jl	SciPy
Performan rel.	Medie	Înalt	Medie
Maturitate ecos.	Înalt	Medie	Înalt
Utilizare acad.	Ridicat	Medie	Ridicat

2. Nivelurile de interfa API. Interfaa **YSQL** este compatibil cu PostgreSQL i suport SQL, tranzacii ACID distribuite, join-uri i indeci. Interfaa **YCQL** este compatibil cu Apache Cassandra i este orientat spre workload-uri wide-column. În aceast lucrare este folosit interfaa YSQL, pentru compatibilitatea cu driver-ele PostgreSQL disponibile în Go, R, Julia i Python.

3.2.3 Sharding i replicare

YugabyteDB suport sharding prin hash i sharding pe intervale. Hash sharding distribuie uniform cheia primar pe tablete i reduce riscul de noduri fierbini, dar este mai puin potrivit pentru scanri ordonate pe intervale. Range sharding pstreaz vecintatea valorilor i este util pentru serii temporale, îns poate concentra scrierile dac toate valorile noi ajung în acelai interval.

Observaie 3.2. Pentru datele epidemiologice cu interogri frecvente pe intervale de date (serii temporale), range sharding pe coloana `timestamp` sau `date` este strategia recomandat, împreun cu un index secundar pe codul bolii i/sau regiune geografic.

3.3 Orchestrarea containerelor Kubernetes

3.3.1 Containere i nevoia de orchestrare

Containerizarea (Docker) permite împachetarea unei aplicaii împreun cu toate dependenele sale într-o unitate portabil i reproductibil. Pe msur ce sistemele cresc în complexitate i numrul de containere depete câteva zeci, gestionarea manual devine infezabil: unde ruleaz fiecare container? Cum se repornete dac cade? Cum se scaleaz?

Kubernetes (K8s) [22, 23] este platforma open-source de orchestrare a containerelor, iniial dezvoltat la Google pe baza experienei cu sistemul intern Borg, i donat CNCF (Cloud Native Computing Foundation) în 2014.

3.3.2 Arhitectura Kubernetes

Un cluster Kubernetes este împrit între planul de control i nodurile worker. Planul de control expune API-ul clusterului prin `kube-apiserver`, pstreaz starea în `etcd`, programeaz pod-urile prin `kube-scheduler` i ruleaz controllerele care aduc starea curent spre starea dorit. Pe nodurile worker, `kubelet` gestioneaz ciclul de via al podurilor, `kube-proxy` configureaz accesul de reea la servicii, iar container runtime-ul pornete efectiv containerele.

3.3.3 Resurse Kubernetes relevante

Resursele folosite cel mai des sunt `Pod`, `Deployment`, `StatefulSet`, `Service`, `PersistentVolumeClaim`, `ConfigMap` i `Secret`. Pentru API i frontend-uri sunt suficiente Deployment-uri, deoarece instanele nu pstreaz stare local. Pentru YugabyteDB este necesar un StatefulSet, care ofer identitate stabil i volume persistente pentru fiecare replic.

3.4 Backend API Go

3.4.1 Caracteristici i filosofia limbajului Go

Go (cunoscut i ca Golang) este un limbaj de programare open-source creat la Google de Robert Griesemer, Rob Pike i Ken Thompson, cu prima versiune stabil în 2012 [24]. Go a fost proiectat pentru productivitate la scar mare, cu accent pe simplitate, compilare rapid i concuren structurat.

Pentru un API backend, Go are câteva proprieti utile: compileaz rapid la cod main, produce binare statice uor de containerizat, ofer concuren prin goroutine i canale, iar biblioteca standard include un pachet HTTP suficient de solid pentru servicii REST. Interfeele structurale reduc cuplarea dintre module, iar garbage collectorul concurrent menine latene acceptabile pentru un API care nu face calcul numeric intensiv în procesul HTTP.

3.4.2 Ecosistem pentru API-uri REST

Dei `net/http` din biblioteca standard este suficient pentru API-uri simple, proiectul folosete routerul `chi`, o bibliotec idiomatic i redus ca suprafa, care ofer rutare ierarhic, middleware-uri compozabile i integrare direct cu tipurile standard `http.Handler`. Aceast alegere evit un framework mai greu i se potrivete cu dimensiunea backend-ului implementat.

3.5 Frontend Vue.js

3.5.1 Arhitectura Vue 3 i Composition API

Vue.js [25] este un framework JavaScript progresiv pentru construirea interfeelor utilizator, creat de Evan You în 2014. Vue 3 (2020) introduce **Composition API**, o alternativ la Options API care permite organizarea logicii prin funcionalitate (nu prin tip — data, methods, computed) i reutilizarea logicii prin *composables*.

Arhitectura Vue 3 combin un sistem reactiv bazat pe `Proxy`, un Virtual DOM pentru reconcilierea arborelui de interfa i Single File Components, în care template-ul, logica TypeScript i stilurile CSS sunt grupate în fiiere `.vue`. Pentru aplicaia de fa, aceast structur permite pstrarea componentelor de formular, navigare i grafic în uniti uor de urmrit.

3.5.2 Ecosistem i biblioteci utilizate

În implementare sunt folosite Vue Router pentru rutare client-side, Pinia pentru starea global, Axios pentru apelurile REST i Chart.js, prin `vue-chartjs`, pentru reprezentarea curbelor compartimentale ale modelului selectat. Componenta de grafic construiete dinamic seriile în funcie de cheile prezente în obiectul JSONB `compartments`, astfel încât acelai cod afieaz atât traiectoriile $S(t)$, $I(t)$, $R(t)$ ale modelului SIR, cât i cele opt traiectorii ale modelului SIDARTHE. Aceste biblioteci acoper nevoile aplicaiei fr a introduce un framework frontend suplimentar peste Vue.

Capitolul 4

Arhitectura Sistemului E-Medicine

4.1 Vedere de ansamblu

Sistemul e-medicine propus în aceast lucrare este o aplicaie distribuit, organizat în componente independente, care permite utilizatorilor s organizeze date epidemiologice pe studii, s încarce seturi de date reale, s declaneze simulri pentru SIDARTHE, SIR, SIS, SEIR i SIRS în trei limbaje de programare distincte — R, Julia i Python — i s compare rezultatele atât din punct de vedere numeric, cât i al performanei computaionale. Arhitectura global este organizat în patru straturi funcionale, ilustrate în Fig. 4.1.

Stratul de prezentare const din dou aplicaii web single-page (SPA) implementate în Vue.js i servite prin containere Nginx: aplicaia principal pentru utilizatorii unui studiu i consola separat pentru super-admini. **Stratul API** este un server REST implementat în Go, responsabil cu autentificarea prin JWT, autorizarea pe baza rolurilor din studii, orchestrarea job-urilor de simulare i medierea accesului la baza de date. **Stratul de calcul** cuprinde runner-ele compartimentale R, Julia i Python, plus runner-ul LSTM dedicat prediciei pe serii temporale; acestea comunic asincron cu API-ul prin cozi Redis separate. **Stratul de stocare** este asigurat de YugabyteDB, care persist atât datele epidemiologice de intrare, cât i rezultatele simulrilor. În mediul local, componentele sunt pornite prin Docker Compose; pentru un deployment de producie, aceeai separare a responsabilitilor permite descrierea resurselor în Kubernetes.

Fluxul principal de date începe cu autentificarea utilizatorului i obinerea unui token JWT. Utilizatorul selecteaz un studiu în care este membru, iar toate operaiile ulterioare sunt evaluate în contextul acelui `study_id`. În funcie de rolul su (`admin`, `researcher`, `contributor` sau `viewer`), el poate încrca un fiier CSV cu date epidemiologice, poate porni simulri sau poate administra membrii studiului. API-ul valideaz structura fiierului, stocheaz metadatele în `datasets`, înregistrrile brute în `epidemiological_records` i pstreaz toate resursele scope-uite pe studiu. La iniierea unei simulri, utilizatorul specific modelul, limbajul dorit, parametrii, populaia N i orizontul $t_{\max}$; API-ul creeaz o înregistrare în `simulation_runs` cu statusul `pending` i public un mesaj JSON în coada Redis corespunztoare limbajului ales. Runner-ul consum job-ul, execut simularea cerut, scrie traiectoriile compartimentale în `simulation_results` i actualizeaz statusul la `completed`. Frontend-ul interogheaz periodic API-ul i afieaz graficele imediat ce simularea este finalizat.

Figura 4.2 prezint aceeai secven sub forma unei diagrame temporale, în care timpul curge de sus în jos, evideniind decuplarea între rspunsul sincron al API-ului i execuia asincron din runner.

4.2 Cerine funcionale i nonfuncionale

4.2.1 Cerine funcionale

Prima cerin funcional privete **autentificarea i autorizarea**: utilizatorii trebuie s se poat înregistra i autentifica, iar toate operaiile asupra datelor i simulrilor sunt protejate prin token JWT cu durat de valabilitate configurabil. Autorizarea nu este doar global, ci i contextual: resursele sunt grupate în studii, iar accesul depinde de rolul utilizatorului în studiul curent. Pe lâng aceste roluri exist flag-ul `is_super_admin`, folosit pentru consola administrativ i pentru vizibilitate la nivelul tuturor studiilor.

Gestionarea studiilor epidemiologice permite crearea unui context de lucru cu nume, cod de boal i descriere. Creatorul devine automat `admin` al studiului i poate aduga membri cu rolurile `researcher`, `contributor` sau `viewer`. Aceast separare permite ca aceiai utilizatori s participe în mai multe studii cu drepturi diferite.

Gestionarea seturilor de date epidemiologice presupune c un utilizator cu rol `admin` sau `contributor` poate încrca fiiere CSV ce conin serii temporale epidemiologice — date calendaristice, numere de cazuri, populaie, regiune geografic. Sistemul valideaz structura, calculeaz numrul de înregistrri i stocheaz datele persistent în baza de date, asociindu-le cu studiul curent.

Configurarea i declanarea simulrilor permite utilizatorului cu rol `admin` sau `researcher` s selecteze un set de date din studiul curent, s aleag limbajul de execuie (R, Julia sau Python) i s introduc tipul modelului, parametrii i dimensiunea populaiei N. Sistemul execut simularea asincron, printr-un runner dedicat, i notific

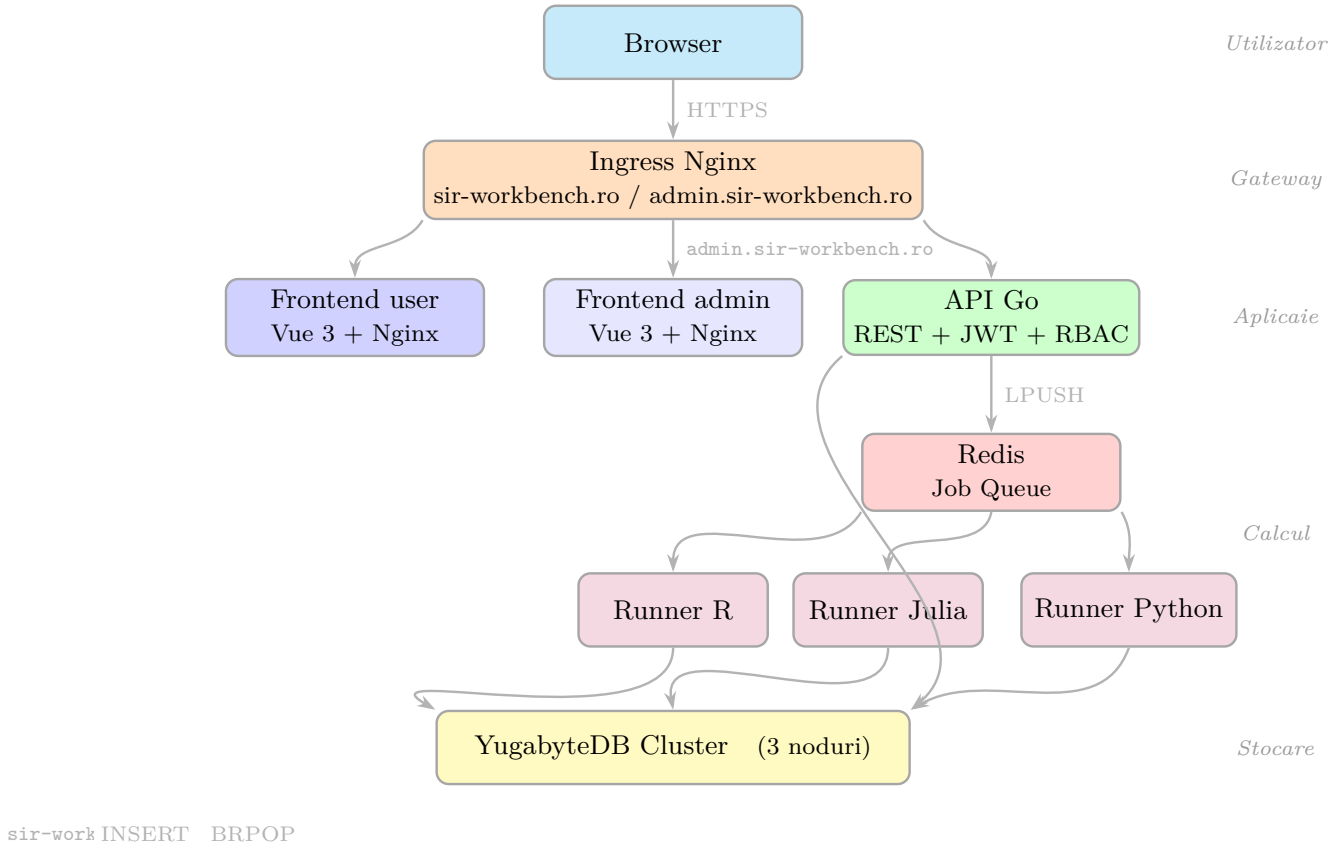


Figura 4.1: Arhitectura global a sistemului e-medicine. Sgeile indic direcia de flux a datelor: browserul acceseaz aplicaia prin Ingress, care ruteaz cererile ctre cele dou SPA-uri sau API; API-ul public job-uri în Redis i le consum runner-ele dedicate; atât API-ul, cât i runner-ele acceseaz acelai cluster YugabyteDB.

finalizarea fr a bloca interfaa.

Vizualizarea rezultatelor presupune afiarea curbelor compartimentale ca grafice interactive, indiferent dac simularea are dou compartimente, ca SIS, trei compartimente, ca SIR/SIRS, patru compartimente, ca SEIR, sau opt compartimente, ca SIDARTHE.

Compararea inter-limbaj permite utilizatorului s selecteze mai multe simulri efectuate pe acelai set de date, cu aceiai parametri, în limbaje diferite, i s compare vizual atât forma curbelor, cât i metricile de performan (timp de execuie, memorie utilizat).

Exportul rezultatelor ofer posibilitatea de a descrca seria temporal a unei simulri în format CSV.

Monitorizarea benchmark-urilor înseamn c sistemul colecteaz automat timpii de execuie (wall-clock i CPU) i consumul de memorie pentru fiecare rulare i le expune printr-un endpoint dedicat, accesibil în interfaa de comparare.

Administrarea global este separat de interfaa principal. Utilizatorii marcai `is_super_admin=true` pot accesa consola `admin.sir-workbench.ro`, unde vd statistici globale, lista utilizatorilor cu promovarea sau revocarea drepturilor de super-admin, lista tuturor studiilor i pagina de detaliu pentru orice studiu, cu membrii, seturile de date, simulrile (cu badge de status) i benchmark-urile agregate. Dashboard-ul include distribuiile agregate între studii (simulri pe status, model i limbaj; seturi de date i studii pe boal; top studii dup volum), iar o pagin dedicat afieaz feed-ul de activitate recent (simulri i încrcri de seturi de date, indiferent de studiu). Toate paginile administrative sunt strict de citire; mutaiile rmân în fluxul aplicaiei principale, controlate prin RBAC pe studiu.

4.2.2 Cerine non-funcionale

Din punct de vedere al performanei, sistemul trebuie s pstreze scurte operaiile sincrone ale API-ului. Crearea unei simulri nu execut modelul în procesul HTTP, ci doar valideaz cererea, persist metadatele i public job-ul în Redis. Durata simulrii este tratat separat, la nivel de runner.

Disponibilitatea depinde de modul de deployment. În varianta Docker Compose, sistemul este potrivit pentru dezvoltare i testare local. Într-un deployment Kubernetes, API-ul poate fi replicat, iar YugabyteDB poate rula cu factor de replicare 3, tolerând pierderea unui nod fr pierdere de date confirmate.

Scalabilitatea este obinut prin design-ul stateless al API-ului i prin arhitectura bazat pe coad: numrul de replici ale oricrui runner poate fi crescut orizontal prin simpla modificare a câmpului `replicas` în Deployment-ul Kubernetes, fr modificri de cod; YugabyteDB suport adugarea de noduri fr downtime.

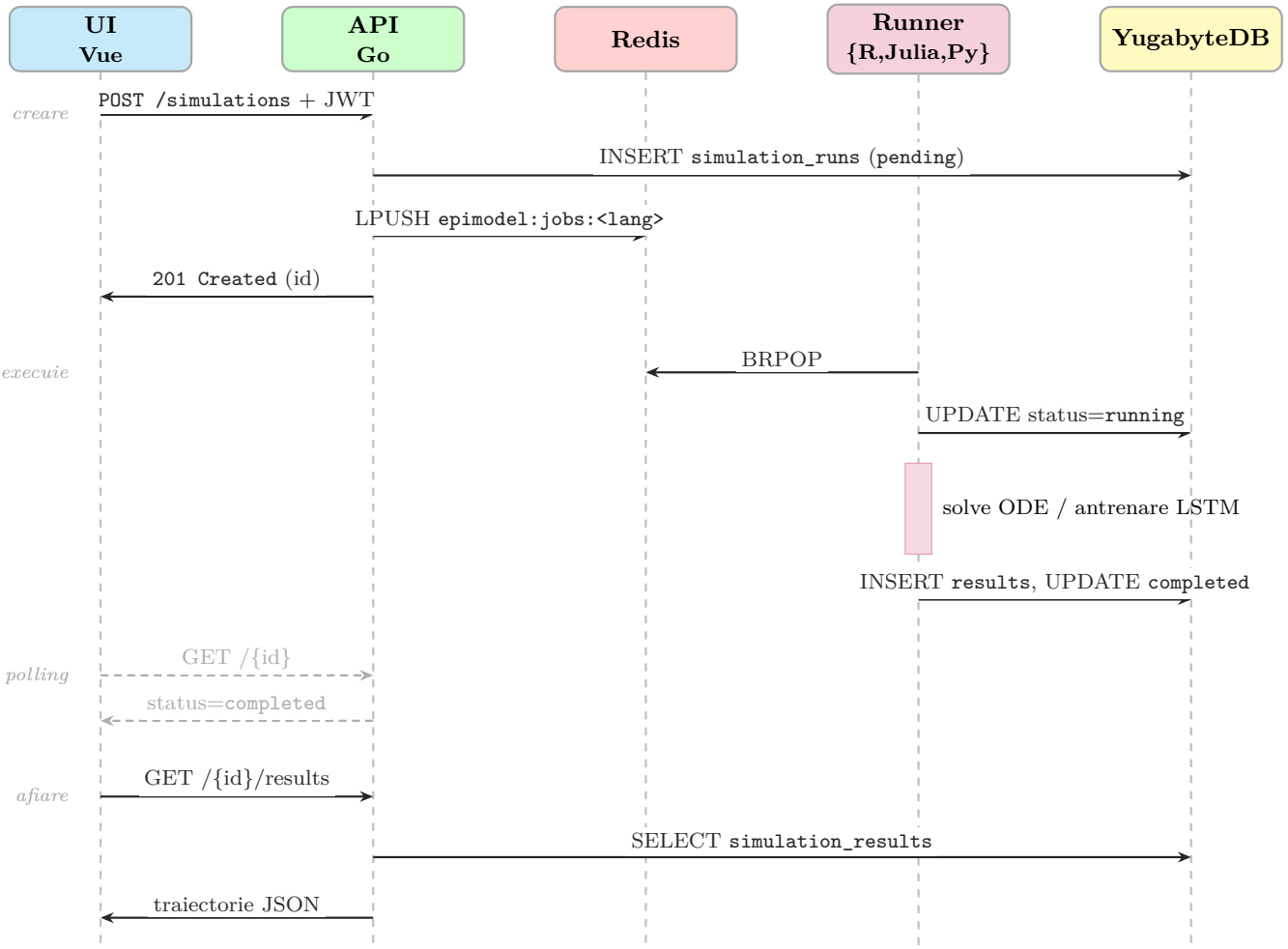


Figura 4.2: Ciclul de via al unei simulri, de la cererea utilizatorului în UI pân la afiarea rezultatului. Timpul curge de sus în jos. Sgeile pline indic mesaje sincrone HTTP/queue/SQL; sgeile întrerupte indic polling-ul repetat al statusului pân la finalizare. API-ul rspunde `201 Created` *înainte* ca solver-ul s ruleze — întreaga execuie numeric se petrece asincron în runner, decuplat prin Redis. Aceasta este complementar Figurii 4.1, care arat topologia: cine vorbete cu cine; aici se vede *când*.

Securitatea implic protejarea tuturor endpointurilor sensibile prin JWT, gestionarea secretelor (chei JWT, parole DB) prin obiecte Kubernetes `Secret` — montate ca variabile de mediu, nu incluse în imaginile Docker — i aplicarea principiului privilegiilor minime la nivelul ServiceAccount-urilor Kubernetes.

Extensibilitatea garanteaz c adugarea unui nou limbaj sau a unui nou model compatibil necesit doar crearea unui nou container runner care respect interfaa de comunicare cu coada Redis, fr modificarea API-ului sau a schemei bazei de date.

Portabilitatea este asigurat prin containerizarea (Docker) a tuturor componentelor i descrierea lor declarativ prin manifeste Kubernetes, permiând deployment pe orice furnizor de cloud sau infrastructur on-premises care suport Kubernetes 1.28+.

4.3 Arhitectura bazei de date distribuite

4.3.1 Schema relaional

Baza de date a sistemului este structurat în jurul conceptului de studiu. Schema include tabele pentru identitate, membership, resurse epidemiologice, simulri i benchmark-uri, iar relaiile principale sunt ilustrate în Fig. 4.3. În forma curent, tabelele aplicative principale sunt: `users`, `studies`, `study_members`, `datasets`, `epidemiological_records`, `simulation_runs`, `simulation_results` i `benchmarks`. Resursele de lucru (`datasets` i `simulation_runs`) conin explicit `study_id`, astfel încât autorizarea i interogrile s fie fcute în contextul unui studiu.

Tabelul `users` stocheaz datele de autentificare ale utilizatorilor; parola nu este reinut în clar, ci ca hash bcrypt cu factor de cost 12. Câmpul `is_super_admin` acord acces la consola administrativ i la vizibilitatea global asupra studiilor. Tabelul `studies` definete unitatea de

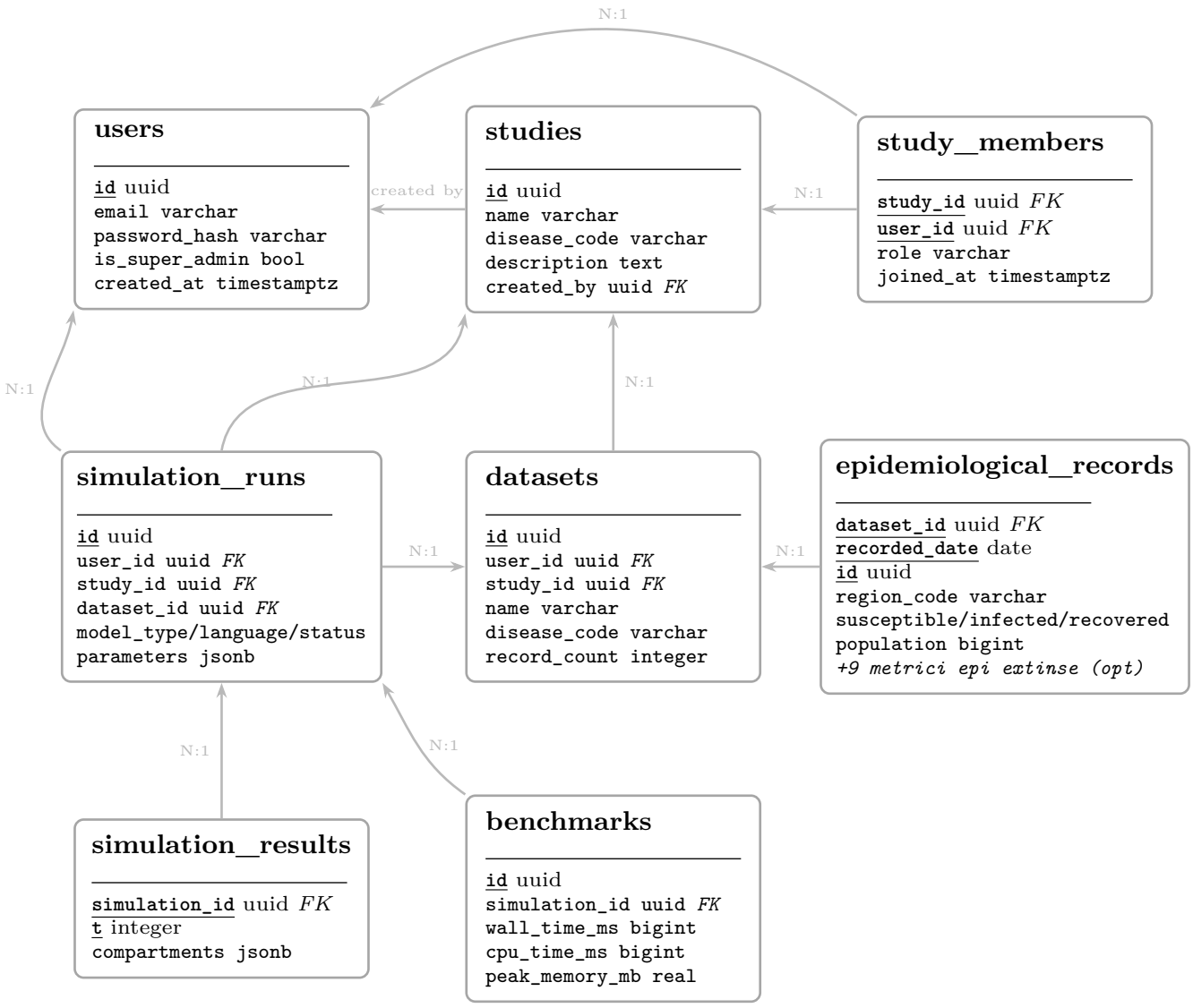


Figura 4.3: Diagrama entitate-relaie a bazei de date curente. PK-urile sunt marcate cu aldine i subliniere; FK-urile sunt marcate cu *FK*. Tabelele `datasets` i `simulation_runs` sunt scope-uite pe `study_id`, iar tabela `study_members` definete rolul fiecrui utilizator într-un studiu.

izolare logic a datelor, iar `study_members` leag utilizatorii de studii prin unul dintre rolurile `admin`, `researcher`, `contributor` sau `viewer`. Tabelul `datasets` conine metadatele seturilor de date epidemiologice încrcate: autorul, studiul, codul bolii, data încrcrii i numrul total de înregistrri. Tabelul `epidemiological_records` stocheaz valorile brute ale seriei temporale; câmpurile `susceptible`, `infected` i `recovered` conin valorile observate sau valorile iniiale folosite la calibrarea parametrilor, iar `region_code` urmeaz standardul ISO 3166-1 alpha-3 în seturile de date preprocesate din OWID. Pe lâng acestea, schema include i 9 coloane epidemiologice opionale (`deaths`, `new_cases`, `new_deaths`, `hospitalized`, `icu`, `vaccinated`, `tested`, `stringency_index`, `reproduction_rate`), populate atunci când sursa de date le ofer; aceste metrici nu sunt folosite de runner-ele built-in, dar sunt expuse integral scripturilor user. Tabelul `simulation_runs` reprezint job-ul de simulare i include studiul, setul de date, tipul modelului (`sidarthe`, `sir`, `sis`, `seir`, `sirs`, `lstm` sau `script`), parametrii în format JSONB, populaia, orizontul temporal, limbajul ales, statusul execuiei (`pending`, `running`, `completed`, `failed`), câmpul `error_message` i timestampurile de creare i finalizare. Tabelul `simulation_results` stocheaz seria temporal de ieire: pentru fiecare pas de timp $t \in \{0, 1, \ldots, t_{\max}\}$, câmpul `compartments` conine un obiect JSONB cu valorile compartimentelor modelului. Tabelul `benchmarks` reine metricile de performan colectate automat la finalul fiecrei rulri: timpul wall-clock, timpul CPU i consumul maxim de memorie.

4.3.2 Modelul datelor epidemiologice

Datele epidemiologice sunt ingerate în sistem prin fiiere CSV cu structura minim prezentat în Table 4.1.

Câmpurile `susceptible` i `recovered` sunt opionale; dac lipsesc, API-ul le estimeaz prin $S(0) = N - I(0)$

Tabela 4.1: Structura minim a fiierului CSV de date epidemiologice.

Câmp	Tip	Descriere
`date`	`DATE`	Data înregistrrii (format ISO 8601, obligatoriu)
`region`	`VARCHAR`	Cod regiune ISO 3166-2 (obligatoriu)
`infected`	`BIGINT`	Numr de cazuri active (obligatoriu)
`population`	`BIGINT`	Populaia total (obligatoriu)
`recovered`	`BIGINT`	Cazuri recuperate (opional)
`susceptible`	`BIGINT`	Populaia susceptibil estimat (opional)
Metrici epidemiologice extinse (toate opionale, propagate scripturilor user)		
`deaths`	`BIGINT`	Decese cumulate
`new_cases`	`BIGINT`	Cazuri noi în intervalul de raportare
`new_deaths`	`BIGINT`	Decese noi în intervalul de raportare
`hospitalized`	`BIGINT`	Pacieni spitalizai la moment
`icu`	`BIGINT`	Pacieni în terapie intensiv la moment
`vaccinated`	`BIGINT`	Persoane vaccinate (cel puin o doz), cumulat
`tested`	`BIGINT`	Teste cumulate
`stringency_index`	`REAL`	Oxford Stringency Index, $0 \ldots 100$
`reproduction_rate`	`REAL`	R_t observat, când e disponibil

i $R(0) = 0$. Granularitatea temporal recomandat este zilnic; datele sptmânale sunt acceptate, dar reduc precizia estimrii parametrilor. La încrcarea unui set de date, API-ul verific c toate câmpurile obligatorii sunt prezente i au tipul corect, c relaia $I(t) + R(t) \leq N$ este respectat pentru fiecare înregistrare, c seria temporal este ordonat cronologic i c fiecare cod de regiune este nevid. Validarea formal fa de un registru ISO i analiza golurilor calendaristice nu sunt incluse în implementarea curent a API-ului; ele sunt tratate în etapa de preprocesare pentru seturile de date OWID incluse în repository.

Metricile epidemiologice extinse (decese, spitalizri, ATI, vaccinri, teste, indicele Oxford de stringen, R_t observat) sunt opionale la nivel de schem i de upload: dac header-ul CSV include coloana corespunztoare iar valoarea este nevid, parser-ul Go o convertete la tipul potrivit (`BIGINT` sau `REAL`) i o trece prin `COPY FROM` în `epidemiological_records`; altfel, câmpul rmâne `NULL`. Runner-ele built-in (SIR/SIDARTHE/LSTM) ignor aceste coloane — ele consum doar prima înregistrare pentru condiia iniial —, îns executorul de scripturi (Chapter 9, Section 9.5) le expune integral funciei `main` prin lista `records`, ceea ce permite cercettorului s scrie modele mai bogate (calibrri care folosesc decesele, ajustri la R_t observat) fr modificri de schem sau de cod în API.

4.3.3 Strategia de partiionare i replicare în YugabyteDB

YugabyteDB utilizeaz un model de stocare bazat pe *tablets* (echivalentul shard-urilor), gestionate prin protocolul de consens Raft [26]. Clusterul este configurat cu factor de replicare $RF = 3$, garantând c pierderea oricrui nod unic nu afecteaz disponibilitatea sau consistena datelor.

Tabelul `epidemiological_records` este proiectat pentru *range sharding* pe cheia compus `(dataset_id, recorded_date)`. Aceast strategie optimizeaz interogrile de tip *range scan* pe interval temporal, care reprezint cel mai frecvent pattern de acces — de exemplu, extragerea seriei temporale complete a unui set de date. În mediul de producie, partiionarea iniial se declar explicit prin clauza `SPLIT AT VALUES`, prezentat mai jos; migraia de baz aplic schema fr presplitting, lsând YugabyteDB s partitioneze automat pe msura creterii datelor.

```
CREATE TABLE epidemiological_records (
    id                  UUID DEFAULT
    gen_random_uuid(),
    dataset_id          UUID NOT NULL
    REFERENCES datasets(id),
    recorded_date       DATE NOT NULL,
    region_code         VARCHAR(10) NOT
    NULL,
    susceptible         BIGINT,
    infected            BIGINT NOT NULL,
    recovered           BIGINT,
    population          BIGINT NOT NULL,
    -- Metrici epi extinse (opt;
    populate cand sursa le ofera).
    deaths              BIGINT,
    new_cases           BIGINT,
    new_deaths          BIGINT,
    hospitalized        BIGINT,
    icu                 BIGINT,
    vaccinated          BIGINT,
    tested              BIGINT,
    stringency_index    REAL,
    reproduction_rate   REAL,
    PRIMARY KEY (dataset_id,
    recorded_date, id)
) SPLIT AT VALUES (
    ('
    00000000-0000-0000-0000-000000000000
    ', '2020-01-01'),
    ('
```

```
    00000000-0000-0000-0000-000000000000', '2021-01-01'),
    ('00000000-0000-0000-0000-000000000000', '2022-01-01')
);
```

Listing 4.1: DDL de producie al tabelului `epidemiological_records` cu range sharding presplit pe cheie temporal.

Tabelul `simulation_results` utilizeaz, în schimb, hash sharding pe `simulation_id`, distribuind uniform rândurile între tablets, deoarece accesul la rezultate este întotdeauna filtrat dup un `simulation_id` specific i nu necesit range scans inter-simulri.

Fiecare tablet are un lider Raft ales dintre cele trei replici; scrierile trec prin lider (*strong consistency*) i sunt replicate sincron pe celelalte dou noduri înainte de confirmare. Parametrul `ysql_num_shards_per_tserver` este setat la 4 (fa de valoarea implicit 8) pentru a reduce supraîncrcarea managementului de tablets pe un cluster de dimensiuni mici.

4.4 Arhitectura API-ului backend Go

4.4.1 Structura modular a proiectului

Proiectul backend este organizat conform conveniilor comunitii Go (*Standard Go Project Layout*) [27]:

```
emedicine-api/
+-- cmd/
|   `-- server/
|       `-- main.go          # Entry point: initializare, injectie dependinte
+-- internal/
|   +-- api/
|   |   +-- handlers/       # Auth, users, studies, datasets, sims, admin
|   |   +-- middleware/     # JWT si RBAC pe studii
|   |   `-- router/         # Configurare rute (chi router)
|   +-- db/
|   |   `-- repository/     # Interfete si implementari repository pattern
|   +-- runner/
|   |   `-- dispatcher.go   # Publicare job-uri in Redis
|   `-- models/             # Struct-uri de domeniu (User, Study, Dataset, Run)
```

Listing 4.2: Structura de directoare a proiectului backend Go.

Arhitectura intern folosete o separare pragmatic între handler-ele HTTP, middleware-uri i repository-uri. **Handler layer** parseaz cererea, valideaz input-ul, apeleaz repository-urile i serializeaz rspunsul JSON. Regulile transversale sunt extrase în middleware-uri: validarea JWT introduce `user_id` în context, iar middleware-ul RBAC introduce `study_id` i rolul curent dup verificarea membership-ului. **Repository layer** concentreaz interogrile SQL i operaiile tranzacionale peste `pgx`. Publicarea job-urilor în Redis este izolat în pachetul `runner`, printr-un dispatcher injectat explicit în handler-ul de simulri.

4.4.2 Mecanismul de autentificare JWT

Autentificarea se bazeaz pe JSON Web Tokens (RFC 7519) [28], utilizând algoritmul de semntur `HS256` (HMAC-SHA256). Cheia secret este generat la instalare i stocat ca Kubernetes Secret, montat în containerul API ca variabil de mediu — nu este inclus în imaginea Docker. La autentificare, clientul trimite credenialele la `POST /api/auth/login`; API-ul verific hash-ul bcrypt al parolei din baza de date i, în caz de succes, returneaz un JWT cu payload `{sub: user_id, exp: now+24h, iat: now}`. Clientul include token-ul în fiecare cerere ulterioar prin header-ul `Authorization: Bearer <token>`, iar middleware-ul JWT îl valideaz — verificând semntura i expierea — fr a accesa baza de date. Absena oricrei stri pe server este esenial pentru scalabilitatea orizontal: oricare replic a API-ului poate valida orice token, fr sincronizare între instane.

4.4.3 RBAC pe studii

Dup autentificare, rutele de lucru sunt evaluate în contextul unui parametru `{studyId}`. Middleware-ul `RequireStudyMembership` verific dac utilizatorul curent este membru al studiului i dac rolul su este inclus în lista permis pentru operaia respectiv. Pentru citire este suficient orice rol valid; pentru încrcarea sau tergerea dataseturilor sunt acceptate `admin` i `contributor`; pentru lansarea simulrilor sunt acceptate `admin` i `researcher`; pentru administrarea studiului este necesar rolul `admin`. Utilizatorii cu `is_super_admin=true` trec automat verificarea de membership i sunt tratai ca administratori în orice studiu, ceea ce permite operaii de suport i audit global.

4.4.4 Diagrama endpointurilor REST

Table 4.2 prezint endpointurile REST expuse de API.

Rspunsurile API urmeaz o convenie simplificat inspirat din *JSON:API*: obiectul de nivel superior conine fie câmpul `data` (la succes), fie `error` cu un cod numeric i un mesaj uman-citibil (la eec), permiând tratarea uniform a erorilor în clientul frontend.

Tabela 4.2: Endpointurile REST principale ale API-ului backend.

Metod	Cale	Descriere	Auth
POST	/api/auth/register	Înregistrare utilizator nou	Nu
POST	/api/auth/login	Autentificare, returneaz JWT	Nu
GET	/api/health	Health check pentru API, DB i Redis	Nu
GET	/api/me	Profilul curent i membershipurile în studii	DA
GET	/api/studies	Lista studiilor vizibile utilizatorului curent	DA
POST	/api/studies	Creare studiu; creatorul devine admin	DA
GET	/api/studies/{studyId}	Detalii studiu	DA
PATCH	/api/studies/{studyId}	Actualizare metadata studiu	Admin studiu
DELETE	/api/studies/{studyId}	tergere studiu i resurse asociate	Admin studiu
GET	/api/studies/{studyId}/members	Lista membrilor studiului	Membru
POST	/api/studies/{studyId}/members	Adugare membru prin email i rol	Admin studiu
PATCH	/api/studies/{studyId}/members/{userId}	Modificare rol membru	Admin studiu
DELETE	/api/studies/{studyId}/members/{userId}	Eliminare membru	Admin studiu
GET	/api/studies/{studyId}/datasets	Lista dataseturilor din studiu	Membru
POST	/api/studies/{studyId}/datasets	Încrcare CSV nou	Admin/Contributor
GET	/api/studies/{studyId}/datasets/{id}	Detalii set de date	Membru
DELETE	/api/studies/{studyId}/datasets/{id}	tergere set de date	Admin/Contributor
GET	/api/studies/{studyId}/simulations	Lista simulrilor din studiu	Membru
POST	/api/studies/{studyId}/simulations	Iniiere simulare nou	Admin/Researcher
GET	/api/studies/{studyId}/simulations/{id}	Status i metadate simulare	Membru
GET	/api/studies/{studyId}/simulations/{id}/results	Seria temporal	Membru
GET	/api/studies/{studyId}/simulations/{id}/export	Export CSV rezultate	Membru
GET	/api/studies/{studyId}/benchmarks	Benchmark-uri agregate per model i limbaj	Membru
GET	/api/admin/stats	Statistici globale pentru consola admin	Super-admin
GET	/api/admin/users	Lista utilizatorilor	Super-admin
PATCH	/api/admin/users/{userId}/super-admin	Promovare/revocare super-admin	Super-admin
GET	/api/admin/studies	Lista tuturor studiilor	Super-admin
GET	/api/admin/breakdown	Distribuii agregate între studii (status, model, limbaj, boal, top studii)	Super-admin
GET	/api/admin/recent-simulations	Ultimele N simulri cu studiu i user	Super-admin
GET	/api/admin/recent-datasets	Ultimele N seturi de date încrcate	Super-admin

4.4.5 Integrarea cu runner-ele de algoritmi

Comunicarea dintre API-ul Go i runner-e este asincron, mediat de Redis printr-un model *producer–consumer*. Când utilizatorul iniiaz o simulare sau o predicie, API-ul public un obiect JSON într-o list Redis dedicat runner-ului. Pentru modelele compartimentale se folosete convenia `epimodel:jobs:<language>` (ex. `epimodel:jobs:r`), iar pentru LSTM se folosete coada separat `epimodel:jobs:lstm`:

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id":    "7b4e1a02-...",
  "model_type":    "sidarthe",
  "parameters":    {"alpha": 0.57, "beta": 0.011, "gamma": 0.456,
                    "sigma": 0.017, "tau": 0.010},
  "population_n":  1000000,
  "t_max":         365,
  "initial":       {"s": 998999.0, "i": 1001.0, "d": 0.0,
                    "a": 0.0, "r": 0.0, "t": 0.0, "h": 0.0, "e": 0.0},
  "db_dsn":        "postgres://emedicine:***@yb-tserver-service.yugabyte.svc.cluster.local:5433/emedicine"
}
```

Listing 4.3: Exemplu mesaj JSON publicat în coada Redis la iniierea unei simulri.

Fiecare runner execut o bucl `BRPOP` blocant pe coada sa dedicat. La primirea unui job, runner-ul actualizeaz statusul simulrii la `running`, execut modelul cerut prin `model_type` (detaliat în Chapter 5), insereaz rezultatele în `simulation_results` prin inserri batch de $t_{\max} + 1$ rânduri, înregistreaz metricile de performan în `benchmarks`, iar în final actualizeaz statusul la `completed` — sau la `failed`, cu mesajul de eroare stocat, dac a aprut o excepie. Existena cozilor separate per executor (`epimodel:jobs:r`, `epimodel:jobs:julia`, `epimodel:jobs:python` i `epimodel:jobs:lstm`) permite scalarea independent a fiecrui runner în funcie de cerere, fr a modifica logica principal a API-ului.

4.5 Arhitectura frontend Vue.js

4.5.1 Structura componentelor

Stratul frontend este împrit în dou aplicaii Vue.js 3 cu Composition API, Vite i TypeScript. Aplicaia `frontend` este interfaa de lucru pentru studiile epidemiologice, iar `frontend-admin` este consola separat pentru superadmini. Cele dou build-uri sunt independente, dar folosesc acelai API i acelai mecanism JWT.

```
frontend/
```

```
‘-- src/
    +-- pages/
    |   +-- StudiesPage.vue
    |   +-- CreateStudyPage.vue
    |   +-- StudyManagementPage.vue
    |   +-- DashboardPage.vue
    |   +-- DatasetsPage.vue
    |   +-- NewSimulationPage.vue
    |   +-- SimulationResultPage.vue
    |   +-- ComparePage.vue
    |   ‘-- BenchmarksPage.vue
    +-- stores/
    |   +-- auth.store.ts
    |   +-- studies.store.ts
    |   +-- datasets.store.ts
    |   +-- simulations.store.ts
    |   ‘-- benchmarks.store.ts
    ‘-- api/client.ts

frontend-admin/
‘-- src/
    +-- pages/
    |   +-- LoginPage.vue
    |   +-- DashboardPage.vue
    |   +-- UsersPage.vue
    |   +-- StudiesPage.vue
    |   +-- StudyDetailPage.vue
    |   ‘-- ActivityPage.vue
    +-- stores/
    |   +-- auth.store.ts
    |   ‘-- admin.store.ts
    ‘-- api/client.ts
```

Listing 4.4: Structura simplificat a celor dou aplicaii Vue.js.

4.5.2 Fluxul de date i state management

Gestionarea strii globale este realizat cu Pinia [29], biblioteca oficial de state management pentru Vue.js 3. Fiecare store encapsuleaz starea, aciunile (apeluri API asincrone) i getters (date derivate) pentru un domeniu specific.

La autentificare, `authStore` primete JWT-ul de la API i îl stocheaz în `localStorage` pentru persisten între sesiuni. Aplicaia principal folosete cheia `emedicine.token`, iar consola admin folosete `emedicine-admin.token`; astfel, sesiunile celor dou interfee rmân independente. Un interceptor Axios adaug automat header-ul `Authorization: Bearer` la fiecare cerere ulterioar, iar navigation guards Vue Router redirecioneaz utilizatorii neautentificai spre pagina de login.

În aplicaia principal, `studiesStore` pstreaz lista de membershipuri, studiul curent i rolul utilizatorului. Rutele care necesit date epidemiologice au meta-câmpul `requiresStudy`; dac nu exist un studiu selectat, utilizatorul este trimis ctre pagina de studii. Drepturile derivate `canWriteDatasets`, `canWriteSimulations` i `canManageStudy` controleaz vizibilitatea aciunilor în interfa i oglindesc regulile aplicate de backend.

Fluxul de simulare începe din pagina `NewSimulationPage`, care apeleaz `simulationsStore.create()` cu parametrii formularului i cu `studyId`-ul curent inclus în URL-ul API. Store-ul primete înapoi `simulation_id` i componenta `SimulationStatus` pornete un polling la interval de 2 secunde pe ruta scope-uit pe studiu, oprindu-se când statusul devine `completed` sau `failed`. La finalizare, `SimulationResultPage` încarc seria temporal i o paseaz componentei `CompartmentChart`, care randeaz graficul interactiv pe baza cheilor prezente în obiectul `compartments`.

Pagina `ComparePage` permite selectarea a 2–3 simulri finalizate. Pentru fiecare rulare selectat, store-ul încarc metadatele i seria temporal prin cereri paralele, iar `CompartmentChart` le suprapune în acelai grafic, difereniindu-le prin stil de linie (continuu, întrerupt, punctat).

Consola `frontend-admin` folosete un store separat, `adminStore`, care apeleaz exclusiv rutele `/api/admin/*`. La login, aplicaia refuz utilizatorii care nu au `is_super_admin=true`, chiar dac autentificarea API ar fi valid. Pe lâng statisticile globale i lista utilizatorilor cu promovarea/revocarea drepturilor de super-admin, store-ul mai expune trei fluxuri de citire între studii: distribuiile agregate pentru dashboard (`breakdown`), simulrile recente (`recentSimulations`) i seturile de date recente (`recentDatasets`). Pagina de detaliu pentru un studiu (`StudyDetailPage`) reutilizeaz direct endpointurile `/api/studies/{studyId}/*`, deoarece superadminul trece automat prin middleware-ul RBAC, fr a fi necesare endpointuri admin dedicate.

4.6 Infrastructura Kubernetes

Sistemul ruleaz pe un cluster Kubernetes versiunea v1.31, gzduit pe o platform cloud privat; detaliile specifice provider-ului IaaS rmân în afara domeniului acestei lucrri. Ciclul de via al nodurilor (creare, redimensionare, rolling updates) este gestionat declarativ prin Cluster API, care permite descrierea topologiei prin obiecte Kubernetes într-un cluster de management separat. Aceast abordare ofer reproducibilitate i uniformitate între medii: aceleai manifeste aplicative pot fi deployate pe orice cluster K8s conform standardelor CNCF, indiferent de provider-ul de infrastructur.

4.6.1 Topologia cluster-ului

Clusterul este compus din ase noduri (Fig. 4.4): trei noduri în rolul de control plane (2 vCPU, 4 GiB RAM fiecare) care gzduiesc componentele standard — `kube-apiserver`, `etcd` stacked, `kube-scheduler`, `kube-controller-manager` — i trei noduri *worker* (4 vCPU, 8 GiB RAM fiecare) pe care ruleaz pod-urile aplicaiei, baza de date i componentele add-on. Toate nodurile ruleaz Ubuntu 22.04 LTS cu `containerd` 1.7 ca

runtime de container.

Interfaa de reea este implementat prin Cilium CNI, care ofer reeaua de pod, DNS intern (prin `CoreDNS`) i posibilitatea de a aplica politici de reea (NetworkPolicy, CiliumNetworkPolicy) la nivel de namespace. Stocarea persistent este asigurat de un driver CSI de tip block storage cu `reclaimPolicy: Delete` i expansion de volum activat, ceea ce permite scalarea în sus a volumelor YugabyteDB fr migrare manual. `metrics-server` este instalat pentru a permite atât utilitarul `kubectl top`, cât i autoscalarea bazat pe metrici CPU/memorie (Horizontal Pod Autoscaler).

Toate resursele aplicaiei sunt grupate în namespaceul `epimodel`, ceea ce permite aplicarea politicilor de reea (NetworkPolicy) i controlul accesului bazat pe roluri (RBAC) la nivel de namespace. Add-on-urile sunt separate în namespace-urile lor proprii (`ingress-nginx`, `kube-system`) conform conveniei standard.

4.6.2 Deployment YugabyteDB pe Kubernetes

YugabyteDB este deploiat prin Helm chart-ul oficial. Cu chart-ul vine un set complet de manifeste — dou `StatefulSet`-uri, dou Service-uri *headless* pentru DNS intern, ConfigMap-uri pentru `gflags` i volumele persistente — pe care îl putem suprascrie printr-un singur fiier de `values` custom adaptat resurselor disponibile:

```
helm repo add yugabytedb https://charts
    .yugabyte.com
helm install yugabyte yugabytedb/
    yugabyte \
  --namespace epimodel \
  -f code/k8s/10-yugabyte-values.yaml \
  --wait --timeout 10m
```

Listing 4.5: Instalarea YugabyteDB cu Helm şi fişierul de `values` custom.

Fiierul `10-yugabyte-values.yaml` declar factor de replicare 3 pentru ambele componente, sizing adaptat workerilor (1 GiB request / 1.5 GiB limit pentru `yb-master`; 2 GiB request / 3 GiB limit pentru `yb-tserver`), un singur volum persistent per replic (5 GiB master, 10 GiB tserver), i gflag-uri specifice pentru clustere mici (`default_memory_limit_to_ram_ratio: 0.6`, `ysql_num_shards_per_tserver: 2`). Chart-ul creeaz:

- dou `StatefulSet`-uri (`yb-master` i `yb-tserver`) cu câte 3 replici fiecare;
- ase `PersistentVolumeClaim`-uri (3×5 GiB pentru masteri + 3×10 GiB pentru tserveri = 45 GiB cumulat);
- dou `Service`-uri headless (`yb-masters` i `yb-tservers`) pentru descoperirea peer-urilor prin DNS intern stabil (de exemplu `yb-master-0.yb-masters.epimodel.svc.cluster.local`), necesar protocolului Raft;
- un `Service ClusterIP` numit `yb-tserver-service` pe portul 5433 (YSQL), folosit de API-ul Go i de runnere pentru conexiunea la baza de date.

4.6.3 Stratul aplicativ: imagini container i registru

Cele apte componente proprii ale sistemului — API Go, frontend-ul principal, frontend-ul admin, cele trei runnere compartimentale R / Python / Julia i runnerul LSTM — sunt împachetate ca imagini Docker construite pentru arhitectura `linux/amd64` i publicate într-un registru de imagini compatibil OCI (GitHub Container Registry în cazul de fa). Pod-urile cluster-ului fac `pull` autentificat prin `imagePullSecrets` de tip `docker-registry`, configurat o singur dat per namespace.

Imaginea API-ului este construit cu un *multi-stage build*: primul stage compileaz binarul Go static (`CGO_ENABLED=0`), iar al doilea copiaz doar binarul rezultat într-o imagine `distroless/static` (`nonroot`), obinând o imagine final de aproximativ 18 MiB, fr shell i fr pachete de sistem inutile. Cele dou frontend-uri Vue sunt compilate în fiiere statice (`npm run build`) i servite prin containere Nginx configurate ca SPA: toate rutele necunoscute sunt redirecionate la `index.html`, permiând Vue Router s gestioneze navigarea pe partea de client. Runner-ele utilizeaz imagini de baz specifice fiecrui limbaj (Python 3.12 slim, R 4.4 base, Julia 1.11), cu dependenele instalate la build time pentru a evita orice descrcare la pornire.

API-ul este deployat ca `Deployment` cu 2 replici i un `Service ClusterIP` pe portul 8080; probele `readinessProbe` i `livenessProbe` interogheaz `GET /api/health`, care verific intern conexiunile ctre YugabyteDB i Redis. Runner-ele funcioneaz ca `Deployment`-uri cu o replic fiecare, fr Service expus — comunicarea se face exclusiv prin coada Redis, ceea ce le face independente de adresele lor de reea i uor de scalat orizontal prin `kubectl scale` sau HPA pe metrici personalizate (de exemplu lungimea cozii Redis).

4.6.4 Coad de mesaje i migraiile bazei de date

Redis este deployat ca `Deployment` cu o singur replic i volum efemer (`emptyDir`); datele cozilor nu sunt persistente între restarturi, ceea ce este acceptabil pentru job-uri de simulare care dureaz secunde sau minute, nu zile. În scenarii cu cerine stricte de durabilitate (de exemplu retenia istoric a job-urilor pentru audit), Redis poate fi convertit într-un `StatefulSet` cu `PersistentVolumeClaim` fr modificri la nivelul aplicaiei.

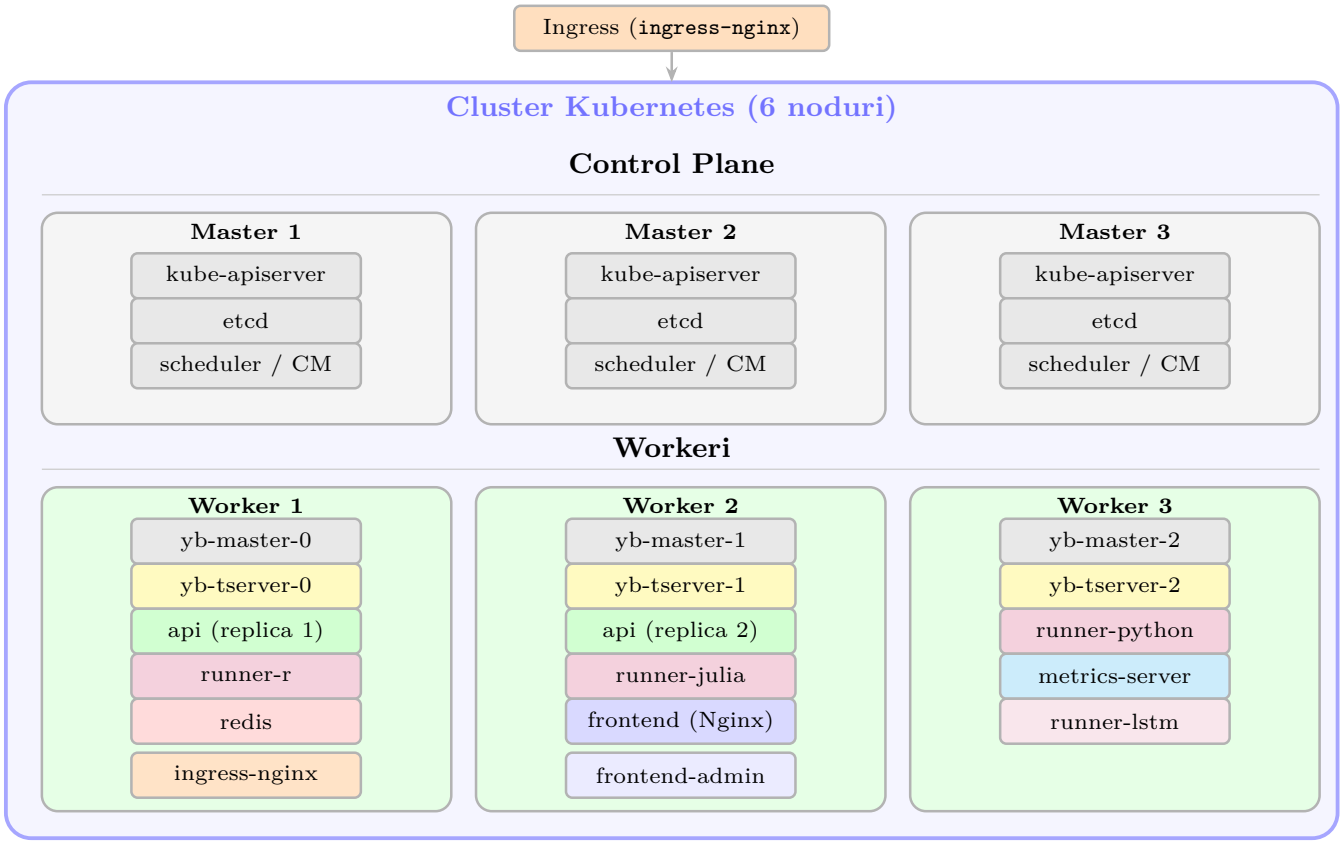


Figura 4.4: Topologia cluster-ului Kubernetes cu 3 noduri control plane HA i 3 noduri worker. Pod-urile YugabyteDB sunt distribuite uniform prin `podAntiAffinity`, garantând c pierderea unui worker afecteaz cel mult o replic `yb-master` i o replic `yb-tserver` — în limita toleranei quorum-ului Raft.

Migraiile SQL sunt aplicate printr-un `Job` Kubernetes care folosete clientul `psql` în locul utilitarului `golang-migrate`. Aceast decizie este motivat de faptul c YugabyteDB nu implementeaz înc *advisory locks* PostgreSQL, pe care multe utilitare de migrare le folosesc pentru a se asigura c o singur instan aplic schema la un moment dat. Job-ul nostru itereaz fiierele `*.up.sql` în ordine alfabetic i le aplic cu `psql -v ON_ERROR_STOP=1`, ceea ce ofer acelai efect de *all-or-nothing* fr a depinde de blocaje distribuite. Prima migrare creeaz extensia `pgcrypto`, necesar pentru funcia `gen_random_uuid()` pe baza PostgreSQL 11 a YugabyteDB 2024.1.

4.6.5 Expunere extern i ingress

Ingress controller-ul `ingress-nginx` este instalat în namespace-ul `ingress-nginx` prin Helm chart-ul oficial, înregistrând un singur `IngressClass` (`nginx`) marcat ca *default* pe cluster. Un singur obiect `Ingress` (`epimodel-ingress`) ruteaz cererile externe pentru dou host-uri. Pe `sir-workbench.ro`, prefixul `/api` ajunge la `Service api`, iar restul traficului ajunge la `Service frontend`. Pe `admin.sir-workbench.ro`, acelai prefix `/api` ajunge la API, iar restul traficului ajunge la `Service frontend-admin`. Aceast separare pstreaz consola administrativ într-un build distinct, fr a duplica backend-ul.

Modul de expunere a ingress-controller-ului depinde de capabilitile provider-ului cloud. Pe un cluster cu `LoadBalancer Service` funcional, controller-ul primete un IP public alocat automat i poate fi accesat direct prin DNS. Pe medii fr provider de `LoadBalancer` disponibil — sau pentru sesiuni de demonstraie controlat — ingress-controller-ul poate fi expus indirect prin `kubectl port-forward` la portul 80 intern al Service-ului. În mediul de producie, terminarea TLS se configureaz la nivel de Ingress prin `cert-manager` cu certificate Let's Encrypt; pentru disertaie, traficul rmâne necriptat pe legtura local `port-forward`, urmând ca activarea TLS s fie un pas operaional ulterior.

4.7 Sistem de plug-in-uri pentru algoritmi

Runner-ele R, Julia i Python descrise pân acum acoper familia modelelor compartimentale (SIR/SIS/SEIR/SIRS/SIDARTHE) i componenta LSTM, dar lista algoritmilor relevani pentru analiza unei serii epidemiologice este deschis: un cercettor poate dori s compare cu o regresie statistic, cu un model AR/ARIMA, cu o variant bayesian sau cu un model agent-based implementat în orice limbaj. Pentru a evita modificarea API-ului, a schemei de baz de date i a deployment-urilor Kubernetes

la fiecare astfel de adugare, sistemul ofer dou ci distincte i complementare de extensibilitate, separate explicit ca responsabiliti i permisiuni:

- **Runner-e bazate pe imagine container** (aceast seciune): un super-admin înregistreaz în studiu o referin ctre o imagine construit extern, iar API-ul provisioneaz automat un Deployment Kubernetes dedicat. Algoritmul este *persistent* (pod always-on, coad proprie) i poate fi rulat de orice membru al studiului. Potrivit pentru algoritmi "oficiali", reutilizabili, în orice limbaj.
- **Editor de cod în browser pentru cercettori** (descris în capitolul 9, seciunea 9.5): un cercettor scrie un script Python într-un editor Monaco, îl salveaz ca draft personal i îl ruleaz ad-hoc peste un dataset. Algoritmul este *efemer* (execuie per cerere, fr pod dedicat) i este executat de un executor partajat `runner-script`. Potrivit pentru explorare, prototipare i analize one-off, fr ceremonia înregistrrii unui runner.

Aceast diviziune are o motivaie practic: înregistrarea unui runner-container implic provisionarea unei resurse Kubernetes permanente i consum o slot de pod în cluster, ceea ce justific restricia la super-admin. Scrierea unui script de exploare nu trebuie s aib acelai cost organizaional, deci este liber oricrui cercettor cu drept de scriere în studiu. Cele dou ci se folosesc de aceleai tabele `simulation_runs`/`simulation_results` i de acelai grafic în interfa — difer doar lifecycle-ul resursei.

4.7.1 Forma standard: *runner protocol*

Forma pe care trebuie s o respecte o imagine container pentru a se comporta ca runner valid este deliberat compact, ca s poat fi implementat în orice limbaj:

1. La pornire, imaginea citete variabilele de mediu `RUNNER_QUEUE` (coada Redis de ascultat), `DATABASE_URL` (conexiunea la YugabyteDB) i `REDIS_URL`.
2. Pentru fiecare mesaj scos cu `BRPOP` din `RUNNER_QUEUE`, se ateapt un JSON cu cheile `simulation_id`, `dataset_id`, `parameters`, `population_n` i `t_max`.
3. Imaginea citete înregistrrile dataset-ului din tabela `epidemiological_records` (`WHERE dataset_id = $1`), execut algoritmul i scrie rezultatele în `simulation_results` (câte un rând per pas temporal, cu `compartments` ca JSONB) i metricile în `benchmarks`.
4. În final, marcheaz `simulation_runs.status = 'completed'` (sau `'failed'` cu `error_message`).

Tipul rezultatului ateptat per pas temporal este acelai ca pentru runner-ele built-in (`{"t": int, "compartments": {...}}`); aceasta înseamn c graficul de traiectorii compartimentale poate afia rezultatele oricrui plug-in fr modificri de cod. Imaginea `runner-custom` furnizat cu sistemul (folosit i ca executor pentru editor) respect acest protocol i poate servi atât ca implementare de referin, cât i ca baz pentru imagini derivate (`FROM ghcr.io/.../emedicine-runner-custom`). Validarea referinei de imagine la upload este o verificare de sanitate (`^[a-zA-Z0-9][a-zA-Z0-9._/:@-]{1,254}$`); Kubernetes respinge la momentul `pull` referinele care nu pot fi rezolvate de nodurile cluster-ului.

4.7.2 Provisioning Kubernetes

La înregistrarea unui runner, API-ul folosete clientul oficial `client-go` pentru a crea un Deployment cu o replic, imagine egal cu referina dat de super-admin, variabilele de mediu populate din Secret/ConfigMap (`DATABASE_URL`, `REDIS_URL`) i `RUNNER_QUEUE` setat la o coad unic derivat din numele runner-ului (`epimodel:jobs:custom-<nume>`). Un `ServiceAccount` dedicat (`api`) este legat de un `Role` care permite verbele `get`, `list`, `create` i `delete` doar pe resursa `deployments` din namespace-ul aplicaiei — nimic mai mult, ceea ce respect principiul privilegiului minim. Tabelul 4.3 rezum tranziiile de stare ale unui runner pe acest traseu.

Tabela 4.3: Ciclul de via al unui runner plug-in, de la upload pân la oprire. Strile sunt persistate in coloana `status` a tabelei `runners`; tranziiile sunt declanate de API (creare/tergere) i de wrapper-ul din pod (marcarea `ready`).

Stare	Trigger / descriere
`pending`	INSERT iniial în DB; rspunsul HTTP returneaz imediat.
`building`	API a creat Deployment-ul K8s prin `client-go`; pod-ul este *Pending*.
`ready`	Wrapper-ul din pod a citit manifestul din DB, a compilat scriptul i ascult pe coad.
`failed`	Eroare la crearea Deployment-ului sau la compilarea scriptului; `status_message` conine detalii.
`stopped`	Oprire explicit; Deployment-ul este ters, dar rândul rmâne pentru audit (opional).

Imaginea fiind construit extern (în CI-ul propriu al cercettorului sau local cu `docker build`), API-ul nu execut nicio operaie de build in-cluster. O extensie natural ar fi un pipeline opional bazat pe `Kaniko` care ar putea construi imagini din scripturi cu `requirements.txt` declarat, dar pentru scopul tezei aceast responsabilitate este lsat în sarcina cercettorului, exact aa cum este lsat construirea oricrei imagini reutilizabile.

4.7.3 Securitate i limitri

Runner-ele plug-in ruleaz cod arbitrar furnizat de utilizator, deci modelul de securitate este conservator. **Acces restrâns la creare:** numai un super-admin poate înregistra sau terge un runner — API-ul aplic aceast restricie prin lanul de middleware `RequireStudyMembership` + `RequireSuperAdmin`, iar pentru un utilizator obinuit cu rol în studiu cererea returneaz `403 super admin required`. **Scope-uire la studiu:** fiecare runner are coloana `study_id` NOT NULL cu cheie strin la `studies`, iar atât listarea cât i tergerea sunt scope-uite (`Get(sid, id)`), exact ca pentru simulri i seturi de date. Un runner înregistrat într-un studiu nu apare în alt studiu i nu poate fi referit cross-studiu. **Nume DNS-safe:** numele runner-ului este validat prin expresia regulat `^[a-z][a-z0-9-]{1,62}[a-z0-9]$`, deoarece ajunge atât în numele cozii Redis cât i în numele Deployment-ului K8s. **Limite de resurse:** fiecare Deployment generat are atribuite explicit `requests` (`100m` CPU, `128Mi` memorie) i `limits` (`500m` CPU, `512Mi` memorie), prevenind ca o imagine prost configurat s afecteze celelalte runner-e sau API-ul.

Întrucât imaginea provine dintr-un registry arbitrar, super-adminul îi asum coninutul ei, exact cum îi asum orice alt imagine din infrastructur. Nu exist înc un mecanism de *allowlist* de registre permise; o îmbuntire natural ar fi restricionarea la registre cunoscute (de exemplu doar `ghcr.io/organizatia/...`) i verificarea opional a semnturilor prin `cosign`. De asemenea, nu exist un mecanism formal de sandboxing suplimentar (de exemplu `gVisor` sau `Kata Containers`) pentru a izola i mai puternic runner-ele fa de nodul Kubernetes; containerul are acces la reeaua cluster-ului (inclusiv DB-ul, prin DSN-ul motenit), ceea ce este intenionat pentru a putea citi dataset-ul, dar înseamn c nu este o sandbox universal. În scenariul vizat de aceast lucrare — platform intern pentru cercettori cunoscui, super-admin curatând înregistrrile — modelul este adecvat; pentru deployment public ar fi necesare politici de reea (`NetworkPolicy`) restrictive i posibil un sandbox runtime.

Modelul de securitate al *editor-ului din browser* este complementar i descris la momentul potrivit (seciunea 9.5): scripturile ruleaz într-un executor partajat `runner-script` cu aceleai limite de resurse, dar permisiunea de a rula scripturi este deschis oricrui cercettor cu drept de scriere de simulri în studiu, fr gating super-admin — compromisul fiind natura ad-hoc a execuiei i absena unei resurse Kubernetes permanente create per script.

Capitolul 5

Implementarea modelelor compartimentale

5.1 Datele de intrare i iniializarea compartimentelor

Datele epidemiologice utilizate de sistem provin din setul public *Our World in Data* (OWID) COVID-19 [30]. Fierele brute nu conin direct toate compartimentele necesare modelului SIDARTHE, ci serii temporale agregate, cum ar fi cazurile noi, cazurile cumulate, decesele cumulate i populaia. Din acest motiv, etapa de preprocesare nu pretinde c recupereaz starea epidemiologic complet a populaiei. Ea construiete o stare iniial compatibil cu sistemul software, folosind datele observabile drept punct de pornire pentru explorarea numeric a scenariilor.

Scriptul `scripts/preprocess_owid.py` reindexeaz fiecare ar pe un calendar zilnic continuu, completeaz cumulativii prin forward-fill i înlocuiete valorile negative din `new_cases` cu zero, deoarece acestea apar de obicei ca ajustri retrospective ale raportrilor. Numrul de infectai activi este aproximat printr-o fereastr mobil de 14 zile:

$$I(t) = \sum_{\tau=t-\Delta}^{t} \text{new_cases}(\tau), \qquad \Delta = 14. \tag{5.1}$$

Compartimentul *removed* este aproximat prin

$$R_{\text{removed}}(t) = \text{total_cases}(t) - I(t), \tag{5.2}$$

iar susceptibilii se obin din bilanul populaiei,

$$S(t) = N - I(t) - R_{\text{removed}}(t). \tag{5.3}$$

Pentru modelele SIR, SIS, SEIR i SIRS, aceste trei valori sunt mapate direct sau printr-o transformare simpl, în funcie de existena compartimentului *removed*. Pentru SIDARTHE, aceeai informaie este distribuit într-o stare iniial clinic mai bogat: cazurile active observabile sunt plasate în compartimentul I, persoanele ieite din infecie sunt plasate în H, iar compartimentele clinice neobservabile pornesc de la zero. Aceast alegere este deliberat conservatoare. Ea nu afirm c distribuia clinic real este cunoscut, ci ofer o condiie iniial reproductibil, potrivit pentru compararea implementrilor i pentru analiza sensibilitii la parametri.

Pentru ca benchmark-urile s fie evaluate pe regimuri dinamice variate, sistemul suport trei familii de seturi de date care produc toate aceeai schem CSV de baz (`date, region, infected, population, recovered, susceptible`). La aceasta se adaug, opional, nou coloane epidemiologice extinse propagate de `preprocess_owid.py` direct din sursa OWID atunci când exist în fiierul brut: `deaths`, `new_cases`, `new_deaths`, `hospitalized`, `icu`, `vaccinated`, `tested`, `stringency_index` (Oxford COVID-19 Government Response Tracker) i `reproduction_rate` (R_t observat). Aceste coloane sunt ignorate de runner-ele built-in (care nu au nevoie decât de prima înregistrare pentru condiia iniial), dar sunt expuse integral scripturilor scrise în editorul din UI prin lista `records`, ceea ce permite scenarii mai bogate de analiz (calibrri care fiteaz simultan cazurile i decesele, factoring în R_t observat, ajustri la interveniile non-farmaceutice prin Stringency Index) fr modificri de schem sau de cod în API.

OWID COVID-19 extins. Lista de ri procesate prin `preprocess_owid.py` a fost extins de la nucleul european la 64 de ri de pe toate continentele (Europa, America, Asia, Africa, Oceania), pentru a acoperi scri de populaie de la ∼5 M (Noua Zeeland) la ∼1.4 B (China, India) i tipare epidemice diferite (suprimare timpurie, valuri masive necontrolate, ieire abrupt din zero-COVID).

OWID mpox. O a doua boal real este integrat prin `preprocess_mpox.py`, care consum setul oficial OWID mpox [30]. Spre deosebire de COVID, fiierul brut nu raporteaz populaia direct; aceasta este dedus ca median a raportului total_cases/total_cases_per_million peste rândurile cu raportare nenul. Fereastra infecioas folosit este de 28 de zile (perioada infecioas a mpox este semnificativ mai lung decât la COVID), iar rile sunt selectate automat dup un prag minim de cazuri totale, rezultând 27 de ri care acoper atât focarul european/american din 2022, cât i focarul din Africa Central din 2024.

Generator sintetic de stres. Pentru a împinge limitele sistemului — în special antrenarea LSTM pe serii lungi i volumul de inserri în baza de date — scriptul `generate_synthetic.py` simuleaz înainte un model

Tabela 5.1: Maparea datelor observabile în condiii iniiale pentru modelele suportate.

Model	Compartimente iniiale	Interpretare
SIS	$S_0 + R_{\text{removed},0}$, I_0	recuperaii redevin susceptibili
SIR i SIRS	S_0, I_0, $R_{\text{removed},0}$	stare agregat cu imunitate permanent sau temporar
SEIR	S_0, $E_0 = 0$, I_0, $R_{\text{removed},0}$	latena pornete neobservat
SIDARTHE	S_0, I_0, $D_0 = A_0 = R_0 = T_0 = 0$, H_0, $E_0 = 0$	model clinic cu decese explicite

SIRS cu forare sezonier:

$$\beta(t) = \beta_0 \left(1 + a \sin(2\pi t/T)\right), \tag{5.4}$$

producând valuri recurente pe orizonturi configurabile (implicit 20 000 de zile, ∼55 de ani simulai). Generatorul ofer trei profile predefinite (`multiwave`, `sharp`, `slow`) cu valori distincte pentru $\mathcal{R}_0$ de baz, durata infecioas i amplitudinea sezonier, iar conservarea $S + I + R = N$ este impus exact dup rotunjire. Logica comun de derivare, validare i scriere CSV este extras în modulul `epi_common.py`, partajat între cele trei surse.

5.2 Contractul comun de simulare

Extinderea aplicaiei la SIDARTHE i la modelele SIR/SIS/SEIR/SIRS a impus generalizarea contractului intern dintre API i runnere. Versiunea iniial a sistemului transmitea doar parametrii β i γ i presupunea implicit c rezultatul conine curbele S, I i R. Versiunea curent folosete câmpul `model_type`, un obiect `parameters` i un obiect `initial`. În acest fel, acelai flux poate executa toate modelele suportate fr a modifica interfaa de cozi sau mecanismul de benchmark.

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id": "7b4e1a02-...",
  "model_type": "sidarthe",
  "parameters": {
    "alpha": 0.57, "beta": 0.011, "
    gamma": 0.456, "delta": 0.011,
    "epsilon": 0.171, "zeta": 0.125, "
    lambda": 0.034,
    "eta": 0.125, "rho": 0.034, "theta"
    : 0.370,
    "mu": 0.017, "kappa": 0.017, "nu":
    0.027,
    "xi": 0.017, "sigma": 0.017, "tau":
     0.010
  },
  "initial": {
    "s": 19000000, "i": 1200, "d": 0, "
    a": 0,
    "r": 0, "t": 0, "h": 18000, "e": 0
  },
  "population_n": 19019200,
  "t_max": 160,
  "db_dsn": "postgres://..."
}
```

Listing 5.1: Structura job-ului publicat în Redis pentru o simulare SIDARTHE.

Coada Redis a fost redenumit din `sir:jobs:<language>` în `epimodel:jobs:<language>`. Schimbarea este mic în cod, dar important conceptual: runner-ul nu mai este o implementare exclusiv SIR, ci un executor pentru modele compartimentale compatibile cu schema comun. Pentru LSTM exist o coad separat, `epimodel:jobs:lstm`, consumat de runner-ul `runner-lstm`. Limbajul nu trebuie transmis în payload pentru modelele compartimentale, deoarece este deja codificat în numele cozii Redis (`epimodel:jobs:r`, `epimodel:jobs:julia` sau `epimodel:jobs:python`); pentru LSTM, rutarea este determinat de `model_type`. Rezultatele sunt stocate în `simulation_results` ca obiect JSONB `compartments`, nu ca trei coloane fixe. Pentru o rulare SIDARTHE, un punct temporal conine cheile `s`, `i`, `d`, `a`, `r`, `t`, `h` i `e`; pentru SIR, acelai câmp conine doar `s`, `i` i `r`; pentru SEIR, ordinea este `s`, `e`, `i`, `r`; pentru LSTM, câmpul conine serii predictive precum `observed`, `fitted` i `predicted`.

5.3 Implementarea modelului SIDARTHE

Sistemul include varianta clinic `sidarthe`. Modelul folosete compartimentele S, I, D, A, R, T, H i E, iar ieirea din T este împrit între vindecare (σT) i deces (τT). Aceast alegere elimin ambiguitatea dintre vindecri i decese i pstreaz explicit bilanul populaiei în toate rezultatele.

În Python, nucleul modelului este o funcie RHS compatibil cu `scipy.integrate.solve_ivp`. Ea primete vectorul de stare, dicionarul de parametri i populaia total:

```
def sidarthe_rhs(t, y, params, n):
    s, i, d, a, r, threatened, h,
    extinct = y
    force = (
        params["alpha"] * i +
        params["beta"] * d +
        params["gamma"] * a +
        params["delta"] * r
    ) / n
```

```
    new_infections = s * force

    return [
        -new_infections,
        new_infections - (params["epsilon"] + params["zeta"] + params["lambda"]) * i,
        params["epsilon"] * i - (params["eta"] + params["rho"]) * d,
        params["zeta"] * i - (params["theta"] + params["mu"] + params["kappa"]) * a,
        params["eta"] * d + params["theta"] * a - (params["nu"] + params["xi"]) * r,
        params["mu"] * a + params["nu"] * r - (params["sigma"] + params["tau"]) * threatened,
        params["lambda"] * i + params["rho"] * d + params["kappa"] * a
            + params["xi"] * r + params["sigma"] * threatened,
        params["tau"] * threatened,
    ]
```

Listing 5.2: Funcia RHS SIDARTHE în runner-ul Python.

În ultimele trei ecuaii se vede separarea cazurilor severe în vindecare i deces:

```
dT = params["mu"] * a + params["nu"] * r \
     - (params["sigma"] + params["tau"]) * threatened
dH = params["lambda"] * i + params["rho"] * d + params["kappa"] * a \
     + params["xi"] * r + params["sigma"] * threatened
dE = params["tau"] * threatened
```

Listing 5.3: Ieirea din compartimentul sever în SIDARTHE.

Aceeai dinamic este exprimat în R i în Julia, cu idiomurile specifice fiecrui ecosistem. În R, funcia returneaz o list de derivate, iar variabilele sunt aduse în scop prin `with(as.list(...))`, ceea ce face codul foarte apropiat de notaia matematic:

```
sidarthe_rhs <- function(t, state, parms) {
  with(as.list(c(state, parms)), {
    force <- (alpha*S_I + beta*D + gamma*A + delta*R) / N
    new_infections <- S * force
    dS <- -new_infections
    dI <- new_infections - (epsilon + zeta + lambda) * S_I
    dD <- epsilon * S_I - (eta + rho) * D
    dA <- zeta * S_I - (theta + mu + kappa) * A
    dR <- eta * D + theta * A - (nu + xi) * R
    dT <- mu * A + nu * R - (sigma + tau) * T
    dH <- lambda*S_I + rho*D + kappa*A + xi*R + sigma*T
    dE <- tau * T
    list(c(dS, dI, dD, dA, dR, dT, dH, dE))
  })
}
```

Listing 5.4: Funcia RHS SIDARTHE în runner-ul R (`deSolve`).

În Julia, convenia *in-place* din `DifferentialEquations.jl` cere ca funcia s scrie derivatele direct într-un vector `du` pre-alocat, ceea ce permite solverului s evite alocrile repetate de memorie:

```
function sidarthe_rhs!(du, u, p, t)
    S, I, D, A, R, T, H, E = u
    N = p["N"]
    force = (p["alpha"]*I + p["beta"]*D
           + p["gamma"]*A + p["delta"]*R) / N
    new_inf = S * force
    du[1] = -new_inf
    du[2] = new_inf - (p["epsilon"] + p["zeta"]
                     + p["lambda"]) * I
    du[3] = p["epsilon"]*I - (p["eta"] + p["rho"]) * D
    du[4] = p["zeta"]*I - (p["theta"] + p["mu"]
                         + p["kappa"]) * A
    du[5] = p["eta"]*D + p["theta"]*A
            - (p["nu"] + p["xi"]) * R
    du[6] = p["mu"]*A + p["nu"]*R
            - (p["sigma"] + p["tau"]) * T
    du[7] = p["lambda"]*I + p["rho"]*D + p["kappa"]*A
            + p["xi"]*R + p["sigma"]*T
    du[8] = p["tau"] * T
end
```

Listing 5.5: Funcia RHS SIDARTHE în runner-ul Julia (`DifferentialEquations.jl`, convenie in-place).

Cele trei snippet-uri reprezint exact *acelai* sistem ODE; diferenele in de ecosistemul numeric: R ofer o sintax apropiat de notaia matematic, Julia urmrete performana prin alocare zero a vectorului de derivate, iar Python ofer integrare direct cu infrastructura software i cu bibliotecile tiinifice uzuale. Tabelul 5.2 sintetizeaz diferenele de implementare pentru cele trei runner-e ODE.

Trei observaii merit notate. Prima: pentru R s-a ales `rk4` cu pas fix $h = 1$ în loc de un solver adaptiv ca `lsoda`, pentru a pstra predictibilitatea timpului de execuie în benchmark-uri — compromisul este o precizie mai mic decât în Julia sau Python, vizibil în regre-

Tabela 5.2: Comparaie side-by-side a celor trei runner-e compartimentale, ca instane ale aceluiai contract de simulare. Diferenele in de ecosistemul numeric (biblioteca solver dominant, convenia de control al pasului), nu de modelul matematic implementat. Numrul de linii include parsarea mesajului, dispecerizarea pe model, scrierea în baza de date i colectarea metricilor.

Limbaj	Bibliotec & metod	Pas	Linii
`r`	`deSolve::ode`, RK4 clasic	fix	274
`julia`	`DiffEq.jl`, `Tsit5` (Tsitouras 5(4))	adaptiv	236
`python`	`scipy.solve_ivp`, `RK45` (Dormand-Prince)	adaptiv	259

sia cross-limbaj (Tabelul 8.10), dar funcional acceptabil. A doua: Julia i Python folosesc tolerane identice (rtol = 10^{-8}, atol = 10^{-10}), ceea ce face traiectoriile lor numerice quasi-identice. A treia: lungimea similar a celor trei runner-e (sub 300 de linii fiecare) arat c abstractizarea contractului comun — mesaj din Redis, dispecerizare pe model, scriere în DB — este suficient de simpl încât adugarea unui al patrulea limbaj s fie un efort redus.

5.4 Variantele SIR, SIS, SEIR i SIRS

Modelele din familia SIR rmân în sistem pentru comparaii, verificri de regresie i explorarea ipotezelor epidemiologice de baz. Ele folosesc acelai câmp `parameters`, dar cu seturi diferite de rate: `sir` i `sis` folosesc `beta`, `gamma`; `seir` adaug `sigma` pentru tranziia expui–infecioi; iar `sirs` adaug `xi` pentru pierderea imunitii. Condiiile iniiale sunt transmise prin acelai obiect `initial`, iar rezultatele sunt stocate tot în `compartments`. Aceast simetrie simplific interfaa web: graficul citete cheile disponibile în fiecare punct temporal i construiete curbele fr s presupun apriori numrul de compartimente.

5.5 Persistena rezultatelor i exportul

Tabela `simulation_runs` reine metadatele execuiei: utilizatorul, studiul, setul de date, modelul, limbajul, populaia, orizontul temporal i parametrii sub form JSONB. Câmpul `study_id` pstreaz izolarea logic dintre studiile epidemiologice i permite ca aceeai tabel s conin simulri aparinând unor echipe diferite. Tabela `simulation_results` reine câte un rând pentru fiecare zi simulat, identificat prin perechea `(simulation_id, t)`. În locul coloanelor fixe `s`, `i`, `r`, rândul conine obiectul JSONB `compartments`. Aceast decizie evit migraii repetate ale schemei atunci când se adaug un model cu alt structur de stare.

```
CREATE TABLE simulation_results (
    simulation_id UUID NOT NULL
    REFERENCES simulation_runs(id) ON
    DELETE CASCADE,
    t              INTEGER NOT NULL,
    compartments   JSONB NOT NULL,
    PRIMARY KEY (simulation_id, t),
    CHECK (t >= 0),
    CHECK (jsonb_typeof(compartments) =
    'object')
);
```

Listing 5.6: Schema generic pentru rezultatele simulrilor.

Exportul CSV este generat dinamic. API-ul inspecteaz cheile prezente în rezultate, le ordoneaz în secvena epidemiologic potrivit i scrie doar coloanele existente pentru modelul respectiv. Prin urmare, o simulare SIR se export cu coloanele `t,s,i,r`, o simulare SEIR cu `t,s,e,i,r`, iar o simulare SIDARTHE cu `t,s,i,d,a,r,t,h,e`. O predicie LSTM se export cu coloanele `t,observed,fitted,predicted`. Aceast convenie pstreaz fiierele lizibile i evit completarea artificial a unor compartimente care nu exist în model.

5.6 Metrici de performan

Toate runnerele colecteaz aceleai trei metrici: timpul wall-clock, timpul CPU i vârful de memorie rezident. Python folosete `time.perf_counter()`, `time.process_time()` i `resource.getrusage`; R folosete `proc.time()` i `/proc/self/status`; Julia folosete `time_ns()` i `Sys.maxrss()`. Valorile sunt scrise în tabela `benchmarks` dup inserarea traiectoriilor, în aceeai tranzacie logic în care rularea este marcat ca `completed`. Dac integrarea numeric sau persistarea eueaz, runner-ul marcheaz rularea ca `failed` i pstreaz mesajul de eroare, astfel încât interfaa web s poat diferenia între o simulare aflat înc în lucru i una care a euat definitiv.

Capitolul 6

Predicie Epidemiologic folosind Reele LSTM

6.1 Rolul prediciei statistice în sistem

Modelele compartimentale prezentate în capitolul 5 ofer interpretabilitate: fiecare compartiment are o semnificaie epidemiologic, iar parametrii descriu rate de tranziie între stri. Aceast proprietate este esenial pentru o lucrare centrat pe familia SIR i pe extensia SIDARTHE. Seriile epidemiologice reale conin îns tipare pe care ecuaiile difereniale nu le modeleaz explicit: întârzieri de raportare, efecte de weekend, ajustri administrative ale datelor i schimbri brute de comportament. Aceste tipare apar ca zgomot structurat peste dinamica de transmitere.

Din acest motiv, sistemul include o component bazat pe o reea LSTM (*Long Short-Term Memory*), tratat ca instrument de predicie statistic pe serii temporale. Ea nu înlocuiete modelele SIR, SIS, SEIR, SIRS sau SIDARTHE, ci adaug o linie de comparaie complet empiric: în loc s presupun o structur de compartimente, înva direct dependena dintre valorile recente ale seriei i valoarea urmtoare. Capitolul descrie modelul implementat efectiv în runner-ul `runner-lstm`, nu o variant idealizat. Implementarea curent este *univariat* — folosete o singur serie, numrul de infectai activi — i *independent* — nu este cuplat cu un model compartimental. Extensiile naturale spre variante multivariate i hibride sunt discutate separat, în seciunea 6.9, i sunt marcate explicit ca neimplementate.

6.2 Formularea problemei de predicie

Fie $y_0, y_1, \ldots, y_{n-1}$ seria observat, unde y_t este numrul de infectai activi în ziua t, derivat din datele OWID conform preprocesrii descrise în seciunea 5.1. Predicia este formulat ca regresie pe ferestre glisante: dintr-o fereastr de lungime ℓ (parametrul `lookback`), reeaua estimeaz valoarea imediat urmtoare,

$$\hat{y}_t = f_\theta(y_{t-\ell}, y_{t-\ell+1}, \ldots, y_{t-1}), \tag{6.1}$$

unde θ reunete toi parametrii reelei. Funcia f_θ produce o predicie pe un singur pas. Predicia pe orizont mai lung se obine *autoregresiv*: ieirea $\hat{y}_t$ este reintrodus în fereastr, iar procedeul se repet pentru fiecare zi din orizontul cerut h (parametrul `forecast_horizon`, egal cu `t_max`).

Aceast formulare are dou consecine importante. În primul rând, ordinea temporal trebuie pstrat strict: o împrire aleatoare a exemplelor ar permite ca informaie din viitor s ajung în antrenare, producând o evaluare optimist nerealist. În al doilea rând, normalizarea seriei trebuie estimat numai pe poriunea de antrenare, pentru a nu introduce, prin medie i abatere standard, informaie din fereastra de test. Ambele cerine sunt tratate explicit în seciunile 6.4 i 6.6.

6.3 Arhitectura celulei LSTM

O reea recurent simpl sufer de atenuarea gradientului pe secvene lungi, ceea ce o împiedic s învee dependene pe termen lung. Celula LSTM rezolv aceast limitare introducând o stare intern c_k protejat de trei pori multiplicative [31]. Implementarea din `runner-lstm` construiete celula manual, în NumPy, fr bibliotec de învare automat, astfel încât fiecare ecuaie s fie vizibil i verificabil în cod.

Se noteaz cu H dimensiunea strii ascunse (parametrul `hidden_size`). La fiecare pas k din fereastr, $1 \le k \le \ell$, intrarea scalar x_k este concatenat cu starea ascuns anterioar $h_{k-1} \in \mathbb{R}^H$ într-un vector

$$z_k = \begin{bmatrix} h_{k-1} \\ x_k \end{bmatrix} \in \mathbb{R}^{H+1}. \tag{6.2}$$

Din z_k se calculeaz cele trei pori i candidatul de stare:

$$f_k = \sigma(W_f\, z_k + b_f)\,, \qquad \text{poarta de uitare} \tag{6.3}$$

$$i_k = \sigma(W_i\, z_k + b_i)\,, \qquad \text{poarta de intrare} \tag{6.4}$$

$$o_k = \sigma(W_o\, z_k + b_o)\,, \qquad \text{poarta de ieire} \tag{6.5}$$

$$\tilde{c}_k = \tanh(W_g\, z_k + b_g)\,, \qquad \text{candidat de stare} \tag{6.6}$$

unde σ este funcia sigmoid, iar $W_f, W_i, W_o, W_g \in \mathbb{R}^{H\times(H+1)}$ i $b_f, b_i, b_o, b_g \in \mathbb{R}^H$. Starea celulei i starea

ascuns se actualizeaz prin

$$c_k = f_k \odot c_{k-1} + i_k \odot \tilde{c}_k, \tag{6.7}$$
$$h_k = o_k \odot \tanh(c_k), \tag{6.8}$$

unde $\odot$ este produsul Hadamard. Strile iniiale sunt nule, $h_0 = c_0 = \mathbf{0}$. Poarta de uitare f_k controleaz cât din starea anterioar se pstreaz, poarta de intrare i_k controleaz cât din candidatul $\tilde{c}_k$ se adaug, iar poarta de ieire o_k filtreaz starea expus ca h_k. Bias-ul b_f este iniializat la o valoare pozitiv mic, ceea ce menine poarta de uitare aproape deschis la începutul antrenrii i uureaz propagarea gradientului.

Dup parcurgerea întregii ferestre, predicia pe un pas se obine printr-un strat liniar de ieire aplicat ultimei stri ascunse:

$$\hat{y} = W_y\, h_\ell + b_y, \qquad W_y \in \mathbb{R}^{1\times H},\ b_y \in \mathbb{R}. \tag{6.9}$$

Tabelul 6.1 pune în coresponden fiecare element matematic cu reprezentarea sa în clasa `LSTMRegressor`. Listingul 6.1 reproduce pasul înainte din implementare, în care se recunosc direct ecuaiile (6.7)–(6.9).

```
for value in seq:
    x = np.array([[float(value)]])
    z = np.vstack([h_prev, x])
    f = _sigmoid(self.wf @ z + self.bf)
    i = _sigmoid(self.wi @ z + self.bi)
    o = _sigmoid(self.wo @ z + self.bo)
    g = np.tanh(self.wg @ z + self.bg)
    c = f * c_prev + i * g
    h = o * np.tanh(c)
y_hat = float((self.wy @ h + self.by)
    [0, 0])
```

Listing 6.1: Pasul înainte al celulei LSTM în `runner-lstm`.

6.4 Normalizarea i construcia exemplelor

Seriile epidemiologice au scri foarte diferite între regiuni, iar valorile absolute pot ajunge la zeci sau sute de mii de cazuri. Pentru ca funciile σ i tanh s lucreze într-un domeniu util, seria este standardizat prin transformarea z-score:

$$\tilde{y}_t = \frac{y_t - \mu}{\varsigma}, \tag{6.10}$$

unde media μ i abaterea standard ς se calculeaz *exclusiv* pe poriunea de antrenare a seriei. Dac abaterea standard este sub un prag numeric, este înlocuit cu 1, pentru a evita împrirea la zero pe serii aproape constante. Prediciile reelei sunt readuse la scara original prin transformarea invers $\hat{y}_t = \tilde{y}_t\,\varsigma + \mu$, urmat de o trunchiere la valori nenegative, deoarece numrul de infectai nu poate fi negativ.

Din seria normalizat se construiesc exemple de antrenare prin ferestre glisante: pentru fiecare indice $t \geq \ell$ din poriunea de antrenare, intrarea este vectorul $(\tilde{y}_{t-\ell}, \ldots, \tilde{y}_{t-1})$, iar inta este $\tilde{y}_t$. Se obine astfel un set de perechi intrare–int pe care reeaua le folosete la antrenare.

6.5 Antrenarea prin backpropagation through time

Antrenarea minimizeaz eroarea ptratic dintre predicie i int. Pentru un singur exemplu cu inta y, funcia de pierdere este

$$L = (\hat{y} - y)^2, \qquad \frac{\partial L}{\partial \hat{y}} = 2\,(\hat{y} - y). \tag{6.11}$$

Deoarece predicia depinde de toi paii din fereastr, gradientul trebuie propagat înapoi prin întreaga secven recurent. Acest procedeu este cunoscut ca *backpropagation through time* (BPTT) [32].

Gradientul stratului de ieire se obine direct din ecuaia (6.9):

$$\frac{\partial L}{\partial W_y} = 2(\hat{y} - y)\, h_\ell^\top, \qquad \frac{\partial L}{\partial b_y} = 2(\hat{y} - y). \tag{6.12}$$

De aici pornete recurena înapoi. Se noteaz cu δh_k gradientul acumulat al strii ascunse la pasul k i cu δc_k gradientul strii celulei. La pasul final, $\delta h_\ell = W_y^\top\, 2(\hat{y} - y)$. Pentru $k = \ell, \ell-1, \ldots, 1$ se aplic, în ordine,

$$\delta o_k = \delta h_k \odot \tanh(c_k) \odot o_k \odot (1 - o_k), \tag{6.13}$$
$$\delta c_k = \delta h_k \odot o_k \odot \big(1 - \tanh^2(c_k)\big) + \delta c_k^{+}, \tag{6.14}$$
$$\delta f_k = \delta c_k \odot c_{k-1} \odot f_k \odot (1 - f_k), \tag{6.15}$$
$$\delta i_k = \delta c_k \odot \tilde{c}_k \odot i_k \odot (1 - i_k), \tag{6.16}$$
$$\delta \tilde{c}_k = \delta c_k \odot i_k \odot \big(1 - \tilde{c}_k^2\big), \tag{6.17}$$

unde δc_k^{+} este contribuia propagat de la pasul $k+1$ i este nul la pasul final. Termenii δo_k, δf_k, δi_k i $\delta \tilde{c}_k$ includ deja derivata neliniaritii corespunztoare. Cu acetia se acumuleaz gradienii ponderilor,

$$\frac{\partial L}{\partial W_\bullet} \mathrel{+}= \delta_{\bullet,k}\, z_k^\top, \qquad \frac{\partial L}{\partial b_\bullet} \mathrel{+}= \delta_{\bullet,k}, \tag{6.18}$$

pentru $\bullet \in \{f, i, o, g\}$, iar gradientul se propag mai departe ctre pasul anterior:

$$\delta z_k = W_f^\top \delta f_k + W_i^\top \delta i_k + W_o^\top \delta o_k + W_g^\top \delta \tilde{c}_k, \tag{6.19}$$
$$\delta h_{k-1} = [\delta z_k]_{1:H}, \qquad \delta c_{k-1}^{+} = \delta c_k \odot f_k. \tag{6.20}$$

Componenta $[\delta z_k]_{1:H}$ reprezint primele H elemente ale vectorului δz_k, adic partea care corespunde strii ascunse din concatenarea z_k; ultima component, asociat intrrii x_k, nu este propagat deoarece intrarea este un dat al problemei.

Înainte de actualizare, fiecare gradient este limitat element cu element la intervalul $[-5, 5]$. Aceast tiere previne exploziile de gradient, frecvente la reelele recurente

Tabela 6.1: Corespondena dintre formularea matematic a celulei LSTM i implementarea din `runner-lstm`.

Element	Notaie	Reprezentare în cod
Porile i candidatul de stare	W_f, W_i, W_o, W_g	matrice `wf, wi, wo, wg` de form $(H, H{+}1)$
Bias-urile porilor	b_f, b_i, b_o, b_g	vectori `bf, bi, bo, bg`; `bf` iniializat la 0,5
Stratul de ieire	W_y, b_y	`wy` de form $(1, H)$ i scalarul `by`
Vectorul concatenat	z_k	`z = vstack([h_prev, x])`
Starea celulei i starea ascuns	c_k, h_k	variabilele `c` i `h`, reportate între pai
Memoria pasului înainte	—	dicionarul `cache`, folosit la backpropagation

antrenate pe secvene cu variaii mari. Actualizarea parametrilor folosete coborârea pe gradient stochastic, cu rata de învare η (parametrul `learning_rate`):

$$\theta \leftarrow \theta - \eta \, \nabla_\theta L. \tag{6.21}$$

Antrenarea parcurge setul de exemple de un numr fixat de ori (parametrul `epochs`); în fiecare epoc, exemplele sunt prezentate într-o ordine permutat aleator, astfel încât actualizrile succesive s nu fie corelate cu ordinea calendaristic.

6.6 Împrirea cronologic i evaluarea

Pentru ca rezultatele s msoare capacitatea de generalizare, seria este împrit cronologic: ultima fraciune din puncte (în implementare, 20%) formeaz fereastra de test, iar restul formeaz poriunea de antrenare. Împrirea respect o constrângere de mrime — poriunea de antrenare trebuie s conin cel puin un exemplu complet de lungime ℓ — altfel rularea eueaz cu un mesaj explicit. Normalizarea (6.10) i exemplele de antrenare se construiesc numai din poriunea de antrenare.

Dup antrenare, reeaua produce predicii pe un pas pentru întreaga serie. În interiorul poriunii de antrenare acestea sunt predicii *in-sample*; pe fereastra de test sunt predicii *out-of-sample*, deoarece intele respective nu au fost vzute la antrenare. Pe fereastra de test se calculeaz metricile de eroare definite în seciunea 7.4: eroarea medie absolut MAE, eroarea medie ptratic radical RMSE i varianta normalizat la populaia regiunii nRMSE $=$ RMSE $/N$. Folosirea acelorai definiii ca în analiza comparativ din capitolul 7 permite aezarea prediciei LSTM pe aceeai scar cu modelele compartimentale. Listing-ul 6.2 arat orchestrarea acestor pai în funcia `train_model`.

```
n = len(series)
train_end = split_index(n, lookback)
# normalizarea se calibreaza doar pe
    antrenare
mean, std = fit_scaler(series[:
    train_end])
normalized = (series - mean) / std
samples = make_samples(normalized[:
    train_end], lookback)
# ... antrenare pe epoci ...
mae, rmse, nrmse = evaluate(series,
    fitted, train_end, population)
```

Listing 6.2: Împrirea cronologic, normalizarea pe antrenare i evaluarea pe test.

În final, reeaua produce o prognoz autoregresiv pe orizontul cerut, pornind de la ultima fereastr observat: fiecare predicie este reintrodus în fereastr pentru pasul urmtor. Algoritmul 6.1 sintetizeaz întregul flux, de la seria brut la metrici i prognoz.

Aplicarea concret a procedurii pe seria Romania OWID (4 ani, 22 ianuarie 2020 – 31 decembrie 2023) este ilustrat în Figura 6.1. Prediciile o-zi-înainte urmresc rezonabil seria observat atât pe fereastra de antrenare cât i pe cea de test, dar prognoza autoregresiv dincolo de ultima zi observat se aplatizeaz rapid ctre zero — o limitare cunoscut a LSTM-urilor compacte univariate, care nu au mecanisme explicite pentru tendinele i sezonalitile pe termen lung. Acest comportament confirm poziionarea LSTM-ului în aceast lucrare ca instrument de prognoz pe termen scurt i ca posibil corector de reziduuri, nu ca model autonom de extrapolare epidemiologic pe orizonturi mari.

6.7 Integrarea în arhitectura aplicaiei

Componenta LSTM este integrat printr-un runner separat, `runner-lstm`, nu prin extinderea runner-ului Python folosit pentru modelele compartimentale. Separarea este justificat: predicia LSTM are alt profil de calcul, ali parametri, alt logic de citire a datelor i alt semantic a rezultatelor. Modelele compartimentale rezolv un sistem de ecuaii difereniale, în timp ce LSTM antreneaz o reea; amestecarea celor dou în acelai executor ar fi îngreunat i codul, i interpretarea rezultatelor.

Fluxul rmâne compatibil cu arhitectura descris în capitolul 5:

- API-ul primete o cerere de predicie cu `model_type = lstm` i ruteaz jobul ctre coada Redis dedicat `epimodel:jobs:lstm`;
- `runner-lstm` consum jobul, citete seria `infected` din setul de date i marcheaz rularea ca `running`;

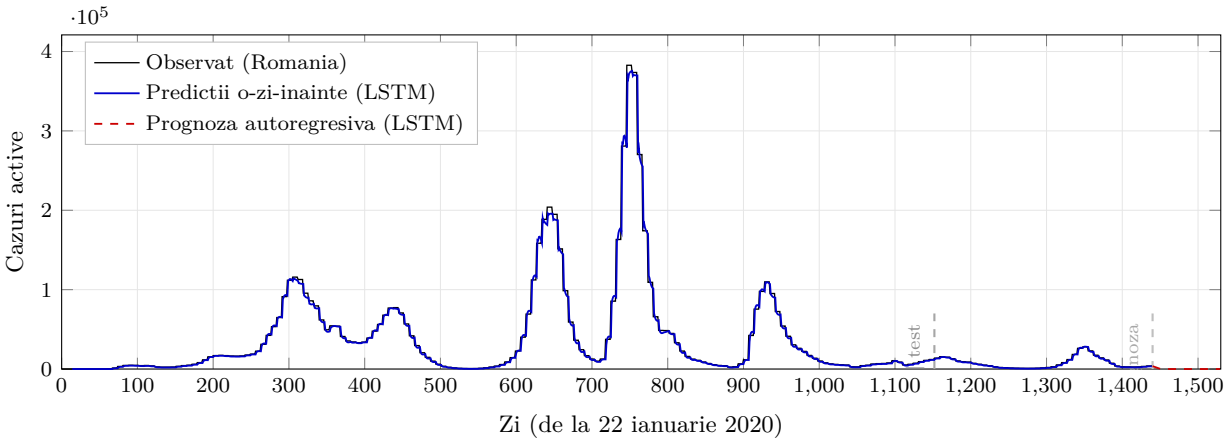


Figura 6.1: Iesirile runner-ului LSTM dedicat pe seria de cazuri active din Romania (4 ani, 22 ian. 2020 – 31 dec. 2023), cu antrenare pe primele 80% din puncte si evaluare pe ultimele 20%. Predictiile o-zi-inainte ($\hat{y}_t = f_\theta(y_{t-14:t-1})$) urmaresc seria observata atat pe fereastra de antrenare cat si pe cea de test, dar prognoza autoregresiva care reintroduce iesirile in fereastra (dincolo de ziua 1440) tinde rapid catre zero — o limitare cunoscuta a LSTM-urilor compacte fara mecanisme explicite pentru sezonalitate, asupra careia un strat ulterior cu informatie exogena (intervalul de raportare, calendarul scolar) ar putea ameliora.

- reeaua este antrenat conform algoritmului 6.1, pe poriunea de antrenare;
- prediciile i prognoza sunt persistate în `simulation_results` ca obiecte JSONB, folosind cheile `observed`, `fitted` i `predicted`, iar metricile de calcul (timp, CPU, memorie) sunt scrise în `benchmarks`;
- interfaa web afieaz cele trei serii suprapuse, distingând vizual poriunea observat de prognoz.

Parametrii reelei — `lookback`, `hidden_size`, `epochs` i `learning_rate` — sunt transmii prin acelai câmp `parameters` ca pentru modelele compartimentale, ceea ce pstreaz contractul comun de simulare. Codul complet al runner-ului este inclus în Anexa D.

Aceast separare pstreaz clar distincia dintre simularea epidemiologic i predicia statistic. Modelul compartimental rmâne responsabil pentru interpretarea mecanismului de transmitere, iar LSTM este folosit pentru prognoz pe termen scurt i ca reper empiric.

6.8 Linii de baz i limitri

O reea LSTM trebuie comparat cu repere simple, altfel o eroare mic poate fi interpretat greit drept performan. Reperele relevante pentru o serie epidemiologic sunt predicia prin persisten, $\hat{y}_t = y_{t-1}$, media mobil pe ultimele k zile i extrapolarea prin ultima diferen (drift). O component LSTM este justificat doar dac reduce eroarea pe fereastra de test fa de aceste repere, evaluat cu protocolul comparativ din capitolul 7.

Aceast comparaie a fost realizat direct, cu eroare o-zi-înainte pe fereastra de test din seria României (ultimele 20% din puncte), prin scriptul `scripts/lstm_baselines.py`. Toate metodele folosesc strict valori observate trecute. Rezultatele, în Tabelul 6.2, sunt clarificatoare: LSTM-ul compact *nu* bate persistena — dimpotriv, are o eroare MAE de aproximativ trei ori mai mare (755 fa de 251 de cazuri), iar toate cele trei repere triviale au erori sub cea a reelei pe MAE.

Tabela 6.2: Validarea LSTM fata de predictori naivi, eroare o-zi-inainte pe fereastra de test (ultimele 20% din seria Romania, 288 zile). Toate metodele folosesc doar valori observate trecute. *Skill* este reducerea relativa a MAE fata de persistenta (valori negative = mai slab decat persistenta). Pe aceasta serie, 85% din zilele de test sunt identice cu ziua precedenta (raportare saptamanala), ceea ce face persistenta extrem de greu de batut.

Metod	MAE	RMSE	Skill
LSTM (o-zi-înainte)	755	1114	-200%
Persisten	251	963	—
Drift (ultima diferen)	501	1362	-99%
Medie mobil (7 zile)	1002	1628	-298%

Explicaia nu este un defect de antrenare, ci o proprietate a datelor: 85% din zilele ferestrei de test sunt identice cu ziua precedent, deoarece seria OWID are o caden de raportare sptamânal (actualizri la interval mediu de 7,0 zile, cu valori constante între ele). Pe o astfel de serie, persistena are eroare *exact zero* în 85% din zile, iar un model care prezice o traiectorie zilnic neted este penalizat tocmai pe zilele plate. Acest rezultat este consistent cu literatura de prognoz a seriilor temporale, în care persistena este un reper notoriu de greu de btut pentru serii netede sau cu raportare agregat. Interpretarea corect nu este c LSTM-ul "nu funcioneaz" — prediciile o-zi-înainte urmresc vizual seria, conform Figurii 6.1 — ci c pe acest tip de date, la acest orizont, valoarea adugat fa de un reper trivial este negativ. Aceasta justific

Algoritm 6.1 Antrenarea i evaluarea componentei LSTM

```
Input: seria infectatilor Y, lungime fereastra l,
       dimensiune stare ascunsa H, rata invatare eta,
       numar epoci E, orizont prognoza h, populatie N
Output: predictii fitted, prognoza forecast,
        metrici MAE, RMSE, nRMSE

imparte Y cronologic: train = primele puncte,
       test = ultima fractiune
calculeaza media si abaterea standard pe train
normalizeaza intreaga serie cu acesti parametri
construieste exemple din ferestre de lungime l pe train
initializeaza reteaua LSTM
for fiecare epoca din 1..E do
    for fiecare exemplu, in ordine permutata do
        ruleaza pasul inainte si calculeaza pierderea
        propaga gradientul prin timp (BPTT)
        limiteaza gradientii si actualizeaza parametrii
    end for
end for
fitted <- predictii o-zi-inainte pe toata seria
MAE, RMSE, nRMSE <- eroare pe fereastra de test
forecast <- prognoza autoregresiva pe h pasi
return fitted, forecast, MAE, RMSE, nRMSE
```

tratarea LSTM-ului ca extensie complementar (corecie de reziduuri, intrare multivariat), nu ca înlocuitor, i motiveaz direciile din seciunea 6.9.

Limitrile sunt intrinsece abordrii. LSTM necesit mai multe date decât modelele compartimentale simple, iar pentru regiuni cu serii scurte sau raportri instabile reeaua poate învа artefacte administrative în locul dinamicii epidemiologice. Riscul de supraînvare este ridicat tocmai pentru c seriile sunt scurte i zgomotoase; de aceea evaluarea se face strict pe fereastra de test, niciodat pe cea de antrenare. În plus, rezultatul este puin interpretabil: o predicie bun numeric nu explic prin ce tranziii epidemiologice se produce evoluia. Din aceste motive, LSTM este prezentat ca extensie complementar, nu ca înlocuitor al analizei modelelor SIR/SIS/SEIR/SIRS/SIDARTHE i nici al calibrrii parametrilor epidemiologici.

6.9 Direcii de extindere

Implementarea curent este deliberat minimal: univariat i independent de modelele compartimentale. Dou extensii sunt naturale, dar *nu* sunt implementate în sistemul descris de aceast lucrare.

Prima este intrarea *multivariat*. În locul unei singure serii, reeaua ar putea primi mai multe canale: cazuri noi zilnice, decese, populaia regiunii sau ziua sptmânii, pentru a capta efectele de raportare. Acest lucru ar cere extinderea dimensiunii intrrii x_k de la un scalar la un vector i ajustarea corespunztoare a limii matricelor de poart.

A doua este modelul *hibrid compartimental–LSTM*. Modelul compartimental produce o predicie interpretabil $\hat{y}_t^{(c)}$, iar LSTM ar putea învа reziduul $r_t = y_t - \hat{y}_t^{(c)}$ în loc de seria brut. Predicia final ar fi

$$\hat{y}_t^{(\text{hibrid})} = \hat{y}_t^{(c)} + f_\theta(r_{t-\ell}, \dots, r_{t-1}) \,. \tag{6.22}$$

Aceast variant ar pstra rolul central al modelului epidemiologic, lsând reeaua s corecteze doar deviaiile statistice rmase dup simularea compartimental. Dac reziduurile au un tipar stabil, modelul hibrid ar putea îmbunti predicia pe termen scurt fr a pierde interpretabilitatea.

Capitolul prezint rezultate operaionale pe Romania (Figura 6.1), cost de execuie concret în Tabelul 8.8 din capitolul 8 i o validare predictiv fa de linii de baz în Tabelul 6.2. Aceast validare, realizat pe seria României, arat c reperul de persisten rmâne superior la orizontul de o zi. Extinderea natural const în repetarea comparaiei pe mai multe regiuni, cu modelul compartimental calibrat ca reper suplimentar i cu varierea hiperparametrilor (ℓ, H, η, E), precum i în evaluarea variantei hibride din ecuaia (6.22), în care reeaua corecteaz reziduuri în loc s prezic seria brut — singurul regim în care, plauzibil, ar putea depi un reper trivial.

Capitolul 7

Comparaia Modelelor, Analiz Numeric i Calibrare

7.1 Rolul comparaiei între modele

Modelele compartimentale discutate în lucrare nu trebuie interpretate ca variante interschimbabile ale aceleiai ecuaii. Fiecare model introduce o anumit ipotez epidemiologic i elimin alte detalii pentru a pstra sistemul analizabil. Din acest motiv, comparaia trebuie fcut pe dou niveluri: compararea structural a ipotezelor i compararea numeric a traiectoriilor obinute pe acelai set de date.

Comparaia structural rspunde la întrebarea ce fenomen poate reprezenta modelul?. Comparaia numeric rspunde la întrebarea cât de bine reproduce modelul datele observate în condiii experimentale identice?. Ambele sunt necesare. Un model simplu poate avea o eroare mic pe o fereastr scurt, dar poate ascunde mecanisme importante; un model complex poate reprezenta mai fidel fenomenul, dar poate fi dificil de calibrat din date puine.

7.2 Comparaie structural

Tabelul 7.1 sintetizeaz diferenele principale dintre modelele suportate de aplicaie. Criteriile alese sunt cele relevante pentru implementarea curent: compartimentele, parametrii necesari, ipoteza central i tipul de date observabile cu care modelul poate fi comparat direct.

SIS este potrivit pentru boli unde reinfectarea rapid sau lipsa imunitii este o ipotez acceptabil. SIR este util ca reper matematic, deoarece pstreaz dinamica minimal a unei epidemii cu imunitate. SEIR introduce latena i devine mai potrivit atunci când exist o perioad de incubaie important. SIRS relaxeaz ipoteza imunitii permanente. SIDARTHE mut accentul de la agregarea biologic la observabilitatea clinic: infeciile nedepistate, cazurile diagnosticate, formele simptomatice, formele severe, vindecrile i decesele sunt tratate separat.

7.3 Operatorul de observaie

Datele reale nu conin de obicei toate compartimentele modelului. Pentru a compara un model cu o serie observat este necesar un operator de observaie $g_m(\mathbf{x}(t))$, dependent de modelul m, care transform starea intern a modelului într-o mrime comparabil cu datele.

Pentru o serie de cazuri active, operatorul poate fi definit astfel:

$$g_{\mathrm{SIR}}(\mathbf{x}) = I, \qquad g_{\mathrm{SEIR}}(\mathbf{x}) = I, \qquad g_{\mathrm{SIRS}}(\mathbf{x}) = I. \tag{7.1}$$

Pentru SIDARTHE, dac seria observat reprezint cazuri diagnosticate active, o alegere mai realist este

$$g_{\mathrm{SIDARTHE}}(\mathbf{x}) = D + R + T, \tag{7.2}$$

deoarece compartimentul I reprezint infectai nedepistai, iar A reprezint persoane simptomatice nedepistate. Dac setul de date conine doar o aproximare general a cazurilor active, aplicaia poate folosi o proiecie mai simpl, dar interpretarea trebuie notat explicit.

Pentru decese cumulative, operatorul SIDARTHE este direct:

$$g_{\mathrm{decese}}(\mathbf{x}) = E. \tag{7.3}$$

Aceast separare este un avantaj al modelului SIDARTHE fa de modelele SIR, SEIR sau SIRS, în care decesele sunt incluse implicit în compartimentul *removed*.

7.4 Metrici de eroare numeric

Fie y_t seria observat la ziua t, iar $\hat{y}_{m,t}$ seria produs de modelul m dup aplicarea operatorului de observaie. Pentru o fereastr de evaluare cu T puncte, eroarea medie absolut este

$$\mathrm{MAE}_m = \frac{1}{T} \sum_{t=1}^{T} \left| y_t - \hat{y}_{m,t} \right|. \tag{7.4}$$

Tabela 7.1: Comparaie structural între modelele utilizate în lucrare.

Model	Compartimente	Parametri	Ipotez principal	Observabil comparabil
SIS	S, I	β, γ	infecia nu produce imunitate durabil	cazuri active aproximabile prin I
SIR	S, I, R	β, γ	recuperarea produce imunitate permanent	active prin I, cumulate prin $I + R$
SEIR	S, E, I, R	β, σ, γ	exist o faz latent neinfecioas sau slab observabil	active prin I, latena rmâne neobservat
SIRS	S, I, R	β, γ, ξ	imunitatea se pierde în timp	active prin I, revenire din R în S
SIDARTHE	S, I, D, A, R, T, H, E	16 rate	diagnosticul, simptomatologia i severitatea clinic modific dinamica observat	active diagnosticate prin $D + R + T$, decese prin E

Eroarea medie ptratic radical este

$$\mathrm{RMSE}_m = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_{m,t})^2}. \tag{7.5}$$

Pentru a compara regiuni cu populaii diferite, este util normalizarea la populaia total N:

$$\mathrm{nRMSE}_m = \frac{\mathrm{RMSE}_m}{N}. \tag{7.6}$$

În date epidemiologice, MAPE poate fi instabil atunci când y_t este aproape de zero. Din acest motiv, lucrarea prefer MAE, RMSE i nRMSE. Dac se compar mai multe serii observabile, de exemplu cazuri active i decese, se folosete o funcie de pierdere ponderat:

$$\mathcal{L}_m(\theta) = \sum_{k \in \mathcal{K}} w_k \frac{1}{T}\sum_{t=1}^{T}\left(y_{k,t} - g_{m,k}(\mathbf{x}_\theta(t))\right)^2, \tag{7.7}$$

unde k indexeaz tipul de observaie, iar w_k controleaz importana fiecrei serii. Ponderile sunt necesare deoarece seriile pot avea scri diferite.

Aplicarea concret a acestor metrici pe rezultate de calibrare pe Romania i pe alte trei ri europene este prezentat în subseciunile 7.6.1 i 7.7.2. Tabelul 7.4 raporteaz MAE pe ferestrele de antrenare i test pentru cele patru regiuni, iar extinderea aceleiai proceduri la o matrice complet model–limbaj–fereastr este o continuare direct a campaniei de benchmark din capitolul 8.

7.5 Protocol de analiz comparativ

Pentru ca analiza comparativ s fie reproductibil, toate modelele trebuie rulate în condiii identice. Protocolul propus are urmtorii pai:

1. se alege acelai set de date, aceeai regiune i aceeai fereastr temporal pentru toate modelele;
2. se aplic aceeai regul de iniializare a compartimentelor, conform Tabelului 5.1;
3. se calibreaz fiecare model pe aceeai fereastr de antrenare;
4. se evalueaz eroarea pe o fereastr separat de test;
5. se raporteaz MAE, RMSE, nRMSE i timpul de execuie;
6. se pstreaz separat interpretarea epidemiologic i costul software.

Separarea între antrenare i test este important. Un model cu muli parametri, cum este SIDARTHE, poate potrivi mai bine fereastra de antrenare, dar aceasta nu garanteaz o predicie mai bun pe zile viitoare. Prin urmare, analiza comparativ trebuie s raporteze atât eroarea de calibrare, cât i eroarea de generalizare.

7.6 Calibrarea parametrilor

Calibrarea parametrilor transform modelul dintr-un instrument de simulare exploratorie într-un instrument ajustat la date observate. Pentru un model m cu vector de parametri θ, calibrarea poate fi formulat ca problem de optimizare:

$$\hat{\theta}_m = \arg\min_{\theta \in \Omega_m} \mathcal{L}_m(\theta), \tag{7.8}$$

unde Ω_m este domeniul admisibil al parametrilor. Constrângerile sunt eseniale: ratele de tranziie trebuie s fie nenegative, iar intervalele lor trebuie alese în funcie de scar temporal i de cunotine epidemiologice.

Pentru modelele SIR/SIS/SEIR/SIRS, calibrarea poate fi fcut cu metode clasice de optimizare neliniar, de exemplu Nelder–Mead [33], L-BFGS-B sau cutare pe gril pentru intervale mici. Pentru SIDARTHE, spaiul de parametri este mai mare, iar optimizarea devine mai fragil. În acest caz sunt utile:

- fixarea unor parametri din literatur sau din ipoteze clinice;

Tabela 7.2: Locurile din analiza comparativ care necesit testare pentru obinerea rezultatelor.

Seciune	Test necesar	Rezultat de raportat
Operatorul de observaie	alegerea proieciei potrivite pentru fiecare model i set de date	seria comparabil cu datele reale: active, diagnosticate sau decese
Metrici de eroare	rularea modelelor pe aceeai fereastr temporal	MAE, RMSE, nRMSE pe fiecare model i limbaj
Protocol comparativ	împrirea cronologic train/test i rulri identice pentru toate modelele	tabel agregat cu eroare pe train i test
Calibrare parametri	optimizarea parametrilor în limite admisibile	parametrii calibrai, eroarea final i numrul de evaluri ale solverului
Identificabilitate	repetarea calibrrii din puncte iniiale diferite	stabilitatea parametrilor i variaia erorii între rulri

- calibrarea pe grupuri de parametri, nu a tuturor ratelor simultan;
- regularizarea fa de valori de referin;
- validarea pe o fereastr temporal nefolosit la optimizare.

7.6.1 Aplicare pe Romania, valul 1 din 2020

Procedura este aplicat concret modelului SIR pe seria de cazuri active din Romania (sursa OWID, vezi capitolul 5), pe valul epidemic de primvar-var 2020. Implementarea este în `scripts/calibrate.py` i folosete `scipy.optimize.minimize` cu metoda Nelder–Mead, restartat din opt puncte iniiale aleatoare pentru a reduce sensibilitatea la condiia iniial. Fereastra este împrit cronologic: 15 martie – 15 august 2020 (154 zile) pentru antrenare, 16 august – 15 septembrie 2020 (31 zile) pentru test. Funcia de pierdere este eroarea ptratic medie pe seria de cazuri active, iar operatorul de observaie este $g_{\mathrm{SIR}}(\mathbf{x}) = I$.

Calibrarea este executat în dou variante:

- *liber*: optimizare simultan a (β, γ) în $\Omega = (0, 5] \times (0, 5]$;
- *constrans*: γ fixat la $1/14$ zi^{-1}, valoarea de referin din literatura clinic pentru COVID-19 (durata medie a perioadei infecioase ~ 14 zile), iar optimizarea se face doar pe β.

Comparaia este prezentat în Tabelul 7.3 i Figura 7.1.

Rezultatele evideniaz dou aspecte caracteristice acestei clase de probleme. În primul rând, eroarea pe antrenare este aproape identic între cele dou scenarii (MAE $\approx$ 1 600 cazuri pentru ambele), dei parametrii estimai difer substanial: $\hat{\gamma}_{\mathrm{liber}} = 0{,}335$ corespunde unei durate infecioase medii de 3 zile, valoare care nu se susine clinic pentru COVID-19. Constrângerea $\gamma = 1/14$ produce un $\hat{\beta}$ de $\sim 3{,}5$ ori mai mic i un $\hat{\mathcal{R}}_0$ de 1,45, mai apropiat de intervalul de literatur 2–3 pentru tulpina original, dar tot sub aceast limit din cauza datelor agregate ale unei singure regiuni intr-o singur fereastr.

Algoritm 7.1 Calibrarea unui model compartimental pe date observate

```
Input: model m, dataset Y, parametri initiali theta0,
       limite Omega, fereastra train, operator g_m
Output: parametri calibrati theta_hat, metrici de eroare

theta <- theta0
while criteriul de oprire nu este indeplinit do
    ruleaza solverul ODE cu parametrii theta
    proiecteaza starea modelului prin g_m
    calculeaza functia de pierdere L_m(theta)
    actualizeaza theta prin algoritmul de optimizare
    proiecteaza theta inapoi in Omega daca este necesar
end while
calculeaza MAE, RMSE si nRMSE pe train si test
return theta, metrici
```

În al doilea rând, eroarea pe fereastra de test creste de aproximativ un ordin de mrime fata de antrenare (13 031 cazuri pentru fitul liber, 16 727 pentru cel constrans). Modelul SIR cu coeficienti constanti nu poate extrapola al doilea val epidemic, care apare în septembrie-octombrie 2020 cu β efectiv mai mare decât în var. Aceasta este o limitare structural a SIR clasic, nu o problem de calibrare.

7.7 Identificabilitate i limitri ale calibrrii

O eroare mic nu implic automat parametri epidemiologic coreci. În modelele neliniare, mai multe combinaii de parametri pot produce traiectorii asemntoare pe aceeai fereastr de timp. Aceast problem se numete neidenti-

Tabela 7.3: Calibrare SIR pe Romania, valul 1 (15 martie – 15 august 2020 antrenare, 16 august – 15 septembrie 2020 test). Doua scenarii: *liber* (optimizeaza ambele rate, (β, γ)), *constrans* (foloseste $\gamma = 1/14$ din durata clinica medie a infectiei COVID-19, optimizeaza doar β). Comparatia ilustreaza tensiunea dintre minimizarea pierderii si interpretabilitatea epidemiologica a parametrilor.

Marime	**Liber**	**Constrans**	**Observatii**
$\hat{\beta}$ (zi^{-1})	0.3677	0.1039	rata de transmitere
$\hat{\gamma}$ (zi^{-1})	0.3346	0.0714 (*fix*)	$1/\hat{\gamma}$ liber = 3.0 zile (non-fizic), constrans = 14 zile
$\hat{\mathcal{R}}_0$	1.10	1.45	valori de literatura pentru tulpina originala $\sim$ 2–3
MAE antrenare	1630	1612	cazuri
RMSE antrenare	2199	2203	cazuri
MAE test	13031	16727	extrapolare 1 luna
RMSE test	14889	19371	cazuri

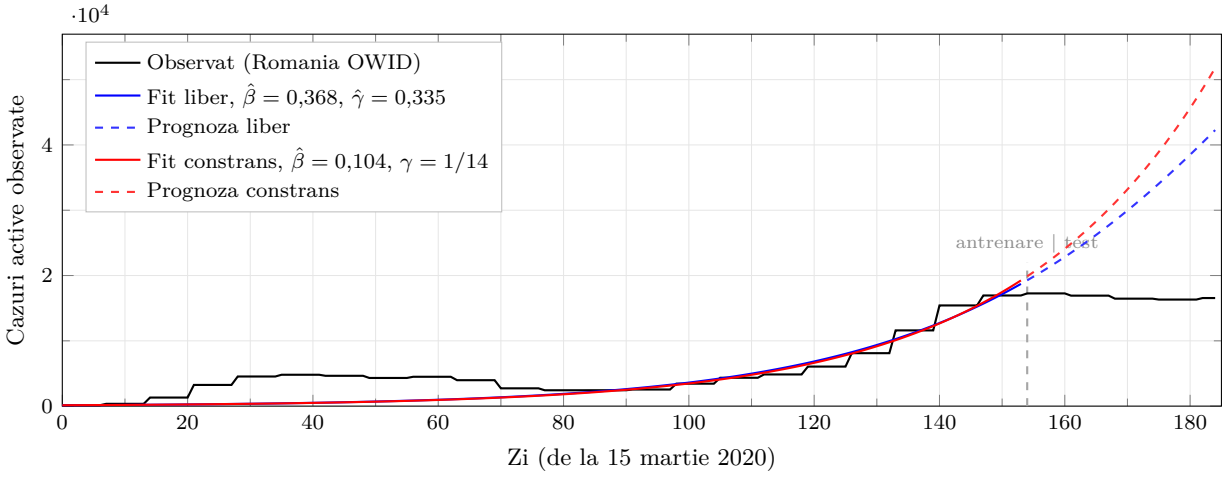


Figura 7.1: Calibrare SIR pe Romania, valul 1 din 2020. Linia neagra arata seria observata; cele doua scenarii produc fituri vizual aproape identice pe fereastra de antrenare (MAE $\approx$ 1 600 cazuri pentru ambele), dar prognoza pe luna urmatoare diverge (MAE 13 031 pentru fitul liber vs 16 727 pentru cel constrans). Parametrii estimati difera substantial, ilustrand identifiabilitatea practica slaba: aceleasi date sunt explicate aproape la fel de bine de combinatii (β, γ) foarte diferite.

ficabilitate practic i este accentuat atunci când datele observate sunt agregate.

Pentru SIDARTHE, neidentificabilitatea este mai important decât pentru SIR. Dac setul de date nu conine date despre testare, spitalizare, terapie intensiv i vindecri, parametri precum ϵ, ζ, μ, ν, σ sau τ nu pot fi estimai robust doar din cazuri active. Într-o lucrare aplicat, acest lucru trebuie declarat explicit. Rolul calibrrii nu este s ascund incertitudinea, ci s arate ce ipoteze sunt necesare pentru a obine o potrivire numeric.

7.7.1 Vizualizarea identifiabilitatii practice pentru SIR

Pentru a face explicit acest fenomen, peisajul de pierdere MSE(β, γ) a fost calculat pe o gril 56×36 în domeniul $\beta \in [0{,}05, 0{,}60]$, $\gamma \in [0{,}05, 0{,}40]$, pentru fereastra de antrenare de mai sus (Figura 7.2). Reprezentarea în logaritm relev o vale alungit, aproape liniar în planul (β, γ): combinatii cu raport β/γ aproximativ constant produc traiectorii similare pe fereastra observat. În consecin, calibrarea liber nu distinge intre $(0{,}368, 0{,}335)$ i $(0{,}104, 0{,}0714)$ pe baza datelor din valul 1 informaia lipsete în date, nu în algoritm.

Aceast observaie justific restrictionarea γ din literatura clinic ca o practic recomandabil, nu ca o limitare arbitrar. Mai mult, rezolvarea unei astfel de identifiabilitati slabe pentru SIDARTHE necesit informatii suplimentare în date testare, spitalizare, terapie intensiv, decese care permit fixarea sau constrangerea unor parametri prin operatori de observaie multipli, nu doar serie singular de cazuri active.

7.7.2 Generalizare pe mai multe regiuni

Pentru a verifica daca framework-ul de calibrare functioneaza cross-regiune si pentru a contextualiza valorile estimate pentru Romania, aceeasi procedura constransa (cu $\gamma = 1/14$) a fost aplicata pe patru tari europene cu scari de populatie si timing-uri ale valului 1 distincte: Romania, Italia, Germania si Spania. Rezultatele sunt prezentate in Tabelul 7.4 si Figura 7.3. Implementarea este in `scripts/calibrate_multiregion.py`.

Tabela 7.4: Calibrare SIR cross-regiune pe valul 1 al pandemiei COVID-19 (15 martie – 15 august 2020 antrenare, 16 august – 15 septembrie 2020 test). In toate cele patru cazuri γ este fixat la valoarea de literatura 1/14 zile, iar β este optimizat prin Nelder-Mead. Valorile $\hat{\beta}$ si $\hat{\mathcal{R}}_0$ sunt comparabile direct intre tari, dar erorile absolute reflecta scarile diferite de populatie (column N) si de I_0.

Tara	N (mil.)	I_0	$\hat{\beta}$	$\hat{\mathcal{R}}_0$	Observatii	MAE train	MAE test
Romania	19.7	135	0.1039	1.45	infectie inca incipienta la inceputul ferestrei	1612	16727
Italia	59.0	20029	0.0673	0.94	deja in plin val 1 ($I_0 \sim 20\,000$)	16243	5318
Germania	83.4	7346	0.0750	1.05	raspuns puternic la masuri	14118	3616
Spania	47.6	13026	0.0759	1.06	val 2 incepe sa creasca la finalul ferestrei	22214	77036

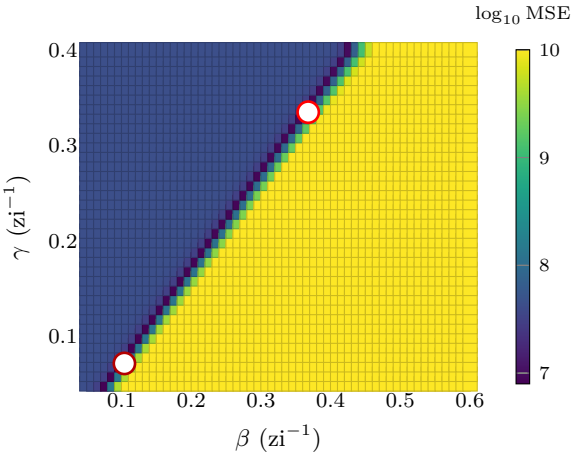


Figura 7.2: Peisajul de pierdere $\log_{10}\text{MSE}(\beta, \gamma)$ pentru modelul SIR pe fereastra de antrenare. Punctele rosii marcheaza solutia libera $(0{,}368, 0{,}335)$ si solutia constransa $(0{,}104, 0{,}0714)$; linia rosie punctata indica restrictia clinica $\gamma = 1/14$. Valea de pierdere mica este alungita si aproape liniara, ceea ce inseamna ca multe combinatii (β, γ) produc o potrivire similara cu datele observate din valul 1 — semnatura tipica a identifiabilitatii practice slabe pentru SIR cu o singura serie observata.

Trei observatii merita subliniate. (i) $\hat{\mathcal{R}}_0$ estimat variaza intre 0,94 (Italia) si 1,45 (Romania), reflectand atat timing-ul diferit al inceperii valului in fiecare tara Italia este deja in faza de declin la inceputul ferestrei, Romania abia incepe cat si intensitatea masurilor nonfarmaceutice deja in vigoare. Toate cele patru valori sunt sub intervalul de literatura 2–3 pentru tulpina originala, ceea ce este consistent cu calibrarea pe o fereastra in care interventiile au actionat deja asupra ratei de transmitere efective. (ii) Procedura functioneaza uniform: aceeasi optimizare Nelder-Mead converge pentru toate cele patru tari, fara ajustari per caz. (iii) Spania exhiba cea mai mare eroare pe test ($\sim$77 000 cazuri MAE), direct atribuibila inceperii valului doi la finalul ferestrei de test limitare structurala a SIR cu coeficienti constanti, nu deficienta de calibrare. Modelele cu rate variabile in timp (sau modelele compartimentale care includ vaccinare sau imunitate care se pierde) ar fi cerute pentru a extrapola peste un schimb de regim epidemic.

7.8 Integrarea în sistemul software

Arhitectura curent este pregtit pentru calibrare deoarece separ trei elemente: datele observate, parametrii modelului i rezultatele simulrii. Extinderea natural const într-un tip suplimentar de job, de exemplu `calibration`, care ruleaz mai multe simulri candidate i salveaz parametrii rezultai într-o tabel dedicat. Simularea final ar putea fi apoi creat automat folosind $\hat{\theta}$.

Aceast extensie trebuie totui tratat diferit de o simulare obinuit. Calibrarea poate necesita zeci sau sute de integrri numerice, deci timpul de execuie, costul CPU i consumul de memorie devin parte a rezultatului. Din acest motiv, evaluarea numeric i evaluarea software trebuie raportate împreun: modelul cu eroare mai mic poate fi mai costisitor, iar modelul mai rapid poate fi insuficient pentru datele disponibile.

7.8.1 Costul software al calibrrii

Pentru a cuantifica acest cost s-a msurat direct calibrarea SIR pe România (scenariile liber i constrâns îm-

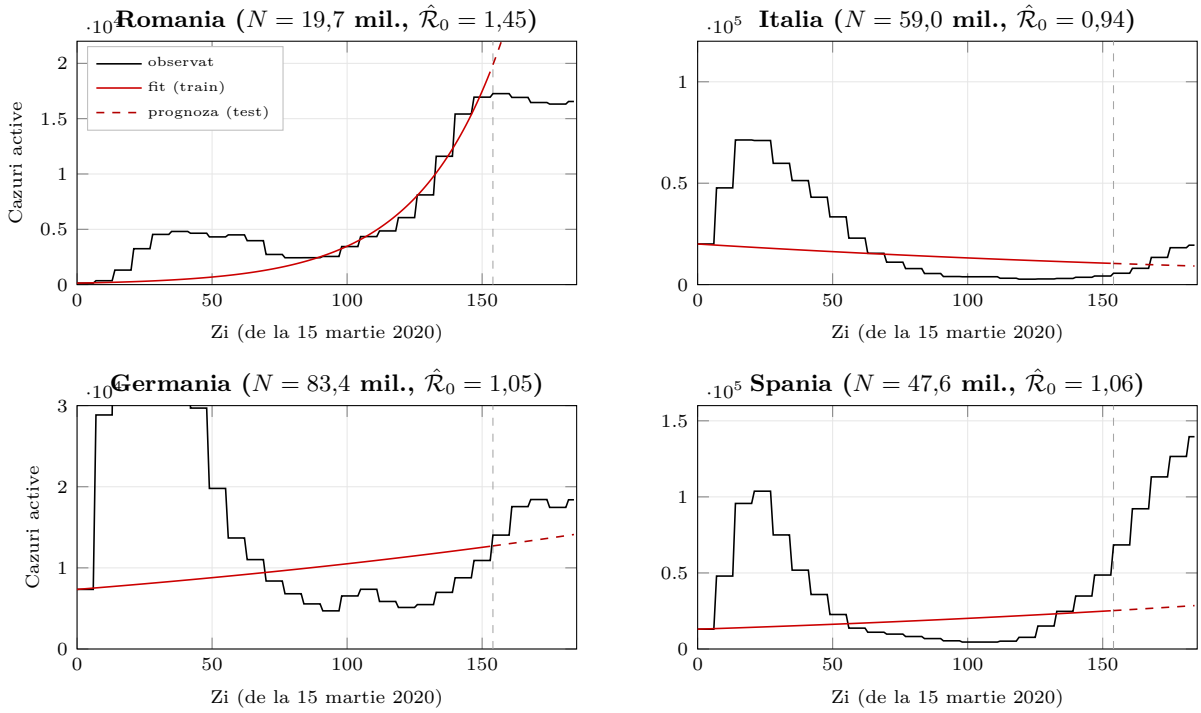


Figura 7.3: Calibrare SIR cu $\gamma = 1/14$ pe valul 1 al pandemiei COVID-19, in patru tari europene cu scari de populatie diferite (atenie la scarile y distincte). Linia neagra este seria observata, linia rosie continua este fitul pe fereastra de antrenare (15 mar. – 15 aug. 2020), iar linia rosie intrerupta este prognoza pe luna urmatoare. Linia verticala marcheaza granita antrenare/test. Modelul reproduce calitativ valul 1 in toate cele patru cazuri; divergenta pe test este vizibila in Spania, unde valul 2 incepe sa creasca rapid in septembrie, evidentiind limitarea structurala a SIR cu coeficienti constanti.

preun, cu restarturi multiple Nelder–Mead), prin scriptul `scripts/bench_calibration_cost.py`. Calibrarea a necesitat 1 377 de simulri candidate — adic 1 377 de evaluri ale funciei obiectiv, fiecare o integrare numeric complet pe fereastra de 154 de zile — realizate *in-process* în 17,2 secunde, cu un consum de memorie de vârf sub 1 MB. Acesta este numrul real de integrri, nu o estimare.

Întrebarea de proiectare este cum se traduce acest cost într-un job al aplicaiei. Tabelul 7.5 compar dou arhitecturi. În varianta *naiv*, fiecare simulare candidat ar fi dispecerizat ca job separat prin aplicaie (Redis → runner → scriere în YugabyteDB). Folosind latena end-to-end msurat a unui job SIR real la acest orizont (∼1 047 ms/job, median pe 8 rulri), cele 1 377 de candidate ar costa ∼1 442 de secunde (aproape 24 de minute) i ar scrie peste 210 000 de rânduri în baza de date — de circa 84 de ori mai scump decât calibrarea in-process, fr niciun câtig de acuratee.

În varianta *recomandat*, calibrarea ruleaz in-process într-un singur job de tip `calibration`, iar în baza de date se scrie doar simularea final obinut cu $\hat{\theta}$ (un singur set de rezultate). Costul de calcul rmâne cel msurat (17,2 s), dar overhead-ul de coad i de persistare — care în varianta naiv ar fi pltit de 1 377 de ori — este amortizat o singur dat. Concluzia de proiectare este clar: calibrarea nu trebuie modelat ca N simulri independente, ci ca un singur job care încapsuleaz bucla de optimizare i persist numai rezultatul final. Aceast observaie justific tratarea calibrrii ca tip de job distinct, cu propriile metrici de cost raportate alturi de eroarea numeric, exact cum recomand seciunea anterioar.

Tabela 7.5: Costul software al calibrarii SIR (Romania, valul 1; fereastra de 154 zile, restart-uri multiple Nelder–Mead pentru scenariile liber si constrans). Coloana *in-process* masoara direct costul procedurii din `calibrate.py`; coloana *naiv (ca job-uri)* proiecteaza costul daca fiecare simulare candidata ar fi un job separat prin aplicatie, folosind latenta end-to-end masurata a unui job SIR real (1047 ms/job); coloana *recomandat* corespunde calibrarii intr-un singur job, cu o singura simulare finala scrisa in baza de date. Numarul de simulari candidate este numarul real de evaluari ale functiei obiectiv.

Marime	**In-process**	**Naiv (ca job-uri)**	**Recomandat**
Simulari candidate	1377	1377	1377
Job-uri prin aplicatie	1	1377	1
Randuri scrise in DB	0	213435	155
Timp total wall (s)	17.24	1441.8*	17.24
Timp CPU (s)	17.23	—	17.23
Memorie de varf (MB)	0.7	—	0.7

* Proiectat: 1377×1047 ms $= 1442$ s; de $\sim$84$\times$ mai scump decat calibrarea in-process, fara castig de acuratete — overhead-ul de coada si persistare ar fi platit pentru fiecare integrare candidata.

Capitolul 8

Evaluarea Software, Performanei i Scalabilitii

8.1 Scopul evalurii

Evaluarea software completeaz analiza numeric din Capitolul 7. Dac analiza numeric msoar cât de bine se potrivete un model cu datele, evaluarea software msoar cât cost obinerea acelui rezultat într-un sistem real. În aplicaia implementat, simularea nu este un apel local ctre o funcie, ci un flux distribuit: API-ul creeaz o rulare, Redis transport job-ul, runner-ul execut solver-ul, YugabyteDB persist rezultatele, iar frontend-ul afieaz statusul i graficul.

Prin urmare, evaluarea trebuie s separe patru componente:

- costul numeric al solverului;
- costul de serializare i persistare a rezultatelor;
- costul operaional introdus de coad i de baza de date;
- costul de utilizare perceput în interfaa web.

Aceast separare este important deoarece optimizarea unei singure componente nu garanteaz îmbuntirea întregului sistem. Un solver rapid poate fi mascat de scrieri lente în baza de date, iar un model cu puine compartimente poate deveni costisitor dac este rulat de multe ori în cadrul calibrrii.

8.2 Metrici colectate

Primele trei metrici (`wall_time_ms`, `cpu_time_ms`, `peak_memory_mb`) sunt scrise de fiecare runner în tabela `benchmarks` i sunt disponibile pentru toate rulrile din campania descris în seciunea 8.7.1. Metricile derivate (`queue_wait_ms`, latene API, dimensiunea rspunsului JSON) ar necesita o instrumentare suplimentar i nu fac parte din rezultatele actuale; ele apar în Tabelul 8.1 pentru completitudine a cadrului de evaluare.

8.3 Matricea experimental

O evaluare complet trebuie s acopere toate combinaiile relevante dintre model, limbaj i orizont temporal. Matricea minim recomandat este prezentat în Tabelul 8.2.

Aceast matrice permite dou tipuri de întrebri. Prima este numeric: cât crete durata când modelul trece de la dou compartimente la opt? A doua este operaional: cum se schimb timpul total când mai multe simulri sunt lansate simultan?

Dou precizri sunt importante pentru interpretarea matricei. (i) Pentru runnerele compartimentale (R, Julia, Python), durata unei rulri nu depinde de mrimea setului de date — runnerul citete doar prima înregistrare pentru condiiile iniiale i integreaz $t_{\max}$ pai. Levierul real pentru rulri lungi este, prin urmare, $t_{\max}$, complexitatea modelului i toleranele solverului. (ii) Pentru runnerul LSTM, dimpotriv, costul de antrenare scaleaz cu lungimea seriei (numrul de eantioane glisante), cu numrul de epoci i cu dimensiunea strii ascunse; în aceast privin, seturile sintetice de zeci de mii de zile sunt eseniale pentru evaluarea regimului saturat al reelei.

Execuia campaniei este automatizat prin scriptul `scripts/run_benchmark.py`, care primete credenialele super-admin, identificatorii unui studiu i ai unuia sau mai multor seturi de date, lista modelelor, lista limbajelor, valorile $t_{\max}$ de baleiat i numrul de repetri. Driverul lanseaz combinaiile prin API (`POST /api/studies/{id}/simulations`), ateapt finalizarea fiecrei rulri prin polling i msoar latena end-to-end pe partea de client (de la crearea simulrii pân la statusul `completed`), care include coada Redis, solverul i persistarea — exact descompunerea propus mai sus. Rezultatele sunt scrise într-un CSV per campanie, iar metricile interne ale solverului (`wall_time_ms`, `cpu_time_ms`, `peak_memory_mb`) rmân disponibile în tabela `benchmarks`, agregabile prin `GET /api/studies/{id}/benchmarks`.

8.4 Protocol de rulare

Pentru rezultate reproductibile, campania de benchmark trebuie s respecte un protocol fix:

1. se pornete aceeai versiune de cod i aceeai configuraie Docker;
2. se ruleaz migraiile bazei de date i se cur datele ex-

Tabela 8.1: Metricile colectate sau derivabile pentru evaluarea software.

Metric	**Nivel**	**Interpretare**
`wall_time_ms`	runner	durata observat între începutul execuiei i finalizarea persistrii rezultatelor
`cpu_time_ms`	runner	timpul de procesor consumat de procesul numeric
`peak_memory_mb`	runner	vârful de memorie rezident raportat de runtime
`queue_wait_ms`	sistem	timpul dintre crearea rulrii i preluarea job-ului de ctre runner
`result_rows`	baz de date	numrul de puncte temporale persistate pentru rulare
`compartment_count`	model	dimensiunea vectorului de stare salvat la fiecare pas
`api_latency_ms`	API	timpul de rspuns pentru crearea unei simulri i citirea rezultatului

Tabela 8.2: Matrice minim pentru campania de benchmark.

Dimensiune	**Valori**	**Scop**	**Observaii**
Model	SIS, SIR, SEIR, SIRS, SIDARTHE	cost în funcie de complexitate	dou pân la opt compartimente
Limbaj	R, Julia, Python	comparaie între ecosisteme	prima rulare Julia trebuie separat de rulrile calde
Orizont $t_{\max}$	365, 2 000, 10 000, 50 000 zile	scalare cu numrul de pai de integrare i de puncte salvate	singurul lever real pentru rulrile ODE; lungimea setului de date nu afecteaz durata solverului
Repetri	minim 10	robustee statistic	raportare prin median i IQR
Concuren	1, 3, 5 job-uri	comportament operaional	msoar coad, DB i scheduling

Tabela 8.3: Teste necesare pentru obinerea rezultatelor de evaluare software.

Zon evaluat	**Test necesar**	**Rezultat de raportat**
Runner numeric	rulri repetate pentru fiecare model, limbaj i orizont temporal	median, IQR, minim i maxim pentru timp i memorie
Cold-start/Warm-start	separarea primei rulri de rulrile ulterioare	costul de iniializare al runtime-ului, în special pentru Julia
Concuren	lansarea simultan a 1, 3 i 5 job-uri	timp total, timp de ateptare în coad i degradarea latenei
Persisten rezultate	rulri cu 30, 90, 180 i 365 puncte temporale	timp de scriere, numr de rânduri i dimensiunea JSONB
API i frontend	citirea rezultatelor i exportul CSV pentru fiecare model	laten endpoint, dimensiune rspuns i timp de export

perimentale vechi;

3. se execut rulri de înclzire pentru fiecare limbaj i model;
4. se ruleaz combinaiile din matricea experimental;
5. se export tabela `benchmarks` i metadatele din `simulation_runs`;
6. se agreg rezultatele prin median, IQR, minim i maxim;
7. se pstreaz separat rezultatele cold-start i warm-start.

Separarea cold-start/warm-start este necesar în special pentru Julia, unde prima rulare poate include cost de compilare JIT. Pentru Python i R, costurile mari apar de obicei la pornirea containerului i la importul bibliotecilor, nu la fiecare job.

Campania de baz a fost executat pe clusterul `kube-4kdhb` (3 worker-i × 4 vCPU / 8 GiB pe OpenStack), folosind imaginile `ghcr.io/masterprogramercpu/emedicine-*:latest` construite din branch-ul principal. Rezultatele sunt prezentate în seciunea 8.7.1; reproducerea pe alte configuraii hardware (de exemplu o main local sau un nod cu GPU) este planificat ca script separat, în afara aplicaiei, i permite plasarea acelorai combinaii $\textit{model} \times \textit{limbaj} \times t_{\max}$ pe scri de hardware diferite.

8.5 Scalabilitate i complexitatea modelului

În modelele compartimentale agregate, mrimea populaiei N nu crete direct dimensiunea sistemului diferenial. O simulare SIDARTHE pentru $N = 10^3$ i una pentru $N = 10^7$ au acelai numr de ecuaii; se schimb doar scara valorilor. Costul crete în schimb cu numrul de compartimente, cu orizontul temporal $t_{\max}$, cu numrul de puncte salvate i cu numrul de fluxuri calculate în funcia RHS.

SIDARTHE este mai costisitor decât SIR/SIS/SEIR/-SIRS nu pentru c populaia este mai mare, ci pentru c fiecare evaluare RHS calculeaz mai multe tranziii clinice i fiecare punct salvat conine mai multe valori. În plus, persistarea în JSONB introduce un cost de serializare care nu exista într-o schem rigid dedicat unui singur model, dar acest cost cumpr flexibilitate: aceeai tabel poate stoca modele cu structuri diferite de stare.

8.6 Compararea implementrilor R, Julia i Python

Cele trei implementri nu folosesc exact acelai algoritm numeric. Runner-ul R folosete RK4 cu pas fix $h = 1$, Python folosete `solve_ivp` cu RK45, iar Julia folosete `Tsit5`. Comparaia trebuie interpretat ca o comparaie între implementri practice echivalente funcional, nu ca un experiment izolat pe acelai solver. Pentru o comparaie strict ar fi necesar implementarea manual a aceleiai metode în toate cele trei limbaje.

Avantajul abordrii curente este c reflect utilizarea natural a fiecrui ecosistem: R prin `deSolve`, Python prin SciPy, Julia prin SciML. Rezultatele benchmark-ului trebuie deci citite ca o comparaie între ecosisteme de calcul tiinific integrate într-un sistem software, nu doar între limbaje de programare.

8.7 Indicatori agregai

Pentru fiecare combinaie model–limbaj se raporteaz mediana timpului wall-clock:

$$\widetilde{T}_{m,l} = \operatorname{median}\left(T_{m,l}^{(1)}, T_{m,l}^{(2)}, \ldots, T_{m,l}^{(r)}\right), \tag{8.1}$$

unde r este numrul de repetri. Variabilitatea este exprimat prin intervalul intercuartilic:

$$\mathrm{IQR}_{m,l} = Q_{3,m,l} - Q_{1,m,l}. \tag{8.2}$$

Pentru comparaii între limbaje se poate raporta raportul de vitez fa de Python:

$$\mathrm{speedup}_l = \frac{\widetilde{T}_{m,\mathrm{Python}}}{\widetilde{T}_{m,l}}. \tag{8.3}$$

Aceast valoare trebuie interpretat per model i per orizont temporal. Un limbaj poate fi mai rapid pentru SIDARTHE pe 365 de zile i mai lent pentru SIR pe 30 de zile dac overheadul runtime-ului domin calculul numeric.

8.7.1 Rezultate msurate

A fost executat o campanie complet prin scriptul `run_benchmark.py`, conform protocolului din seciunea 8.4: cinci modele compartimentale, trei limbaje, patru orizonturi temporale ($t_{\max} \in \{90, 365, 2\,000, 10\,000\}$), 3 repetri per combinaie, pe setul de date Romania OWID. Toate cele 180 de rulri s-au finalizat fr erori. Datele sunt extrase direct din tabela `benchmarks` a aplicaiei deployate, valorile reflectând timpul msurat de ctre runner între începutul integrrii ODE i finalul scrierii rezultatelor.

Vizual, scalarea wall-time-ului în funcie de $t_{\max}$ (pentru modelul cel mai complex, SIDARTHE) este prezentat în Figura 8.1; cele trei limbaje produc trei regimuri calitativ diferite, identificabile dintr-o privire pe o scar log-log.

Trei observaii dominante reies din date.

(i) Julia este cel mai rapid solver, dar are un cost de pornire JIT semnificativ. La $t_{\max} = 10\,000$, mediana wall-time-ului Julia este de 5–6 ms pentru orice model, inclusiv SIDARTHE cu opt compartimente. Python rmâne în intervalul 5–9 ms, iar R urc de la 665 la

Tabela 8.4: Wall-time median al solverului (ms), pe model si limbaj, pentru patru orizonturi $t_{\max}$. Rezultate din campania de benchmark prin aplicatie (cluster `kube-4kdhb`, 3 worker x 4 vCPU / 8 GiB), 3 repetari per combinatie, dataset Romania OWID, prima inregistrare ca conditie initiala.

Model	**Limbaj**	**Wall-time median (ms)**			
		$t_{\max} = 90$	$t_{\max} = 365$	$t_{\max} = 2000$	$t_{\max} = 10000$
`sis`	`r`	12.0	41.0	149.0	738.0
`sis`	`julia`	0.0	0.0	1.0	5.0
`sis`	`python`	4.0	5.0	4.0	6.0
`sir`	`r`	12.0	35.0	146.0	807.0
`sir`	`julia`	0.0	1.0	1.0	5.0
`sir`	`python`	4.0	4.0	5.0	5.0
`seir`	`r`	19.0	35.0	161.0	673.0
`seir`	`julia`	1.0	0.0	1.0	5.0
`seir`	`python`	4.0	3.0	5.0	9.0
`sirs`	`r`	10.0	40.0	154.0	665.0
`sirs`	`julia`	0.0	1.0	1.0	5.0
`sirs`	`python`	5.0	4.0	5.0	5.0
`sidarthe`	`r`	16.0	62.0	274.0	1327.0
`sidarthe`	`julia`	2.0	1.0	2.0	6.0
`sidarthe`	`python`	5.0	5.0	5.0	8.0

Tabela 8.5: Raport de viteza al solverului fata de Python (mediana wall-time Python / mediana limbaj). Valori > 1 inseamna mai rapid decat Python; valori < 1 inseamna mai lent. Pragul ‘$> x$’ apare cand mediana sub-milisecundara face raportul exact instabil.

Model	**Limbaj**	**Speedup vs Python**			
		$t_{\max} = 90$	$t_{\max} = 365$	$t_{\max} = 2000$	$t_{\max} = 10000$
`sis`	`r`	0.33	0.12	0.03	0.01
`sis`	`julia`	> 8	> 10	4.00	1.20
`sir`	`r`	0.33	0.11	0.03	0.01
`sir`	`julia`	> 8	4.00	5.00	1.00
`seir`	`r`	0.21	0.09	0.03	0.01
`seir`	`julia`	4.00	> 6	5.00	1.80
`sirs`	`r`	0.50	0.10	0.03	0.01
`sirs`	`julia`	> 10	4.00	5.00	1.00
`sidarthe`	`r`	0.31	0.08	0.02	0.01
`sidarthe`	`julia`	2.50	5.00	2.50	1.33

Tabela 8.6: Memorie reziduala peak (MB), mediana pe toate combinatiile model x $t_{\max}$.

Limbaj	Caracteristica runtime	Memorie median (MB)
`r`	interpretat, pachete deSolve+RPostgres incarcate	129
`julia`	JIT, runtime DifferentialEquations.jl	940
`python`	interpretat, NumPy+SciPy	95

1 327 ms, de dou pân la trei ordine de mrime peste celelalte. Pentru orizonturi mici ($t_{\max} = 90$), prima rulare Julia pltete îns costul compilrii *just-in-time*, vizibil în diferena dintre repetarea 1 i repetrile 2–3 (Tabelul 8.7): ∼900–1 500 ms doar pentru încrcarea i compilarea funciilor RHS i a solverului `Tsit5`.

Tabela 8.7: Costul de pornire JIT pentru runner-ul Julia: prima rulare include compilarea, repetarile ulterioare ruleaza din cod cache-uit. Valori la $t_{\max} = 90$.

Model	Rep 1 (ms)	Mediana rep 2-3 (ms)	Cost JIT (ms)
`sis`	1435	0	1435
`sir`	891	0	891
`seir`	887	0	886
`sirs`	948	0	948
`sidarthe`	2	1	1

(ii) Python (SciPy `solve_ivp`) este cel mai stabil ca performan. Wall-time-ul rmâne aproape constant în jurul a 4–9 ms indiferent de model sau orizont, fr cold-start vizibil; aceasta îl face limbajul implicit recomandabil pentru utilizare interactiv, unde predictibilitatea conteaz la fel de mult ca viteza brut.

(iii) Costul mare al runner-ului R la orizonturi mari este dominat de inserrile individuale în baza de date, nu de `deSolve`. Cele 10 001 inserii rând-cu-rând prin `dbExecute()` reprezint componenta principal a celor ∼1 secund msurat la `sidarthe`/`r`/$t_{\max} = 10\,000$; integrarea numeric propriu-zis cu `deSolve::ode` este sub-secundar. Aceast observaie este un punct concret de îmbuntire ulterioar (de exemplu prin `COPY FROM` sau prin tranzacii batch), nu o limitare intrinsec a ecosistemului R.

În ce privete memoria rezidual peak (Tabelul 8.6), Julia pltete un overhead constant de ∼940 MB pentru runtime-ul SciML, în timp ce Python rmâne sub 100 MB iar R sub 135 MB. Acest cost este independent de complexitatea modelului i de orizont, deci nu se amortizeaz la rulri scurte; pentru job-uri foarte multe i mici, Python

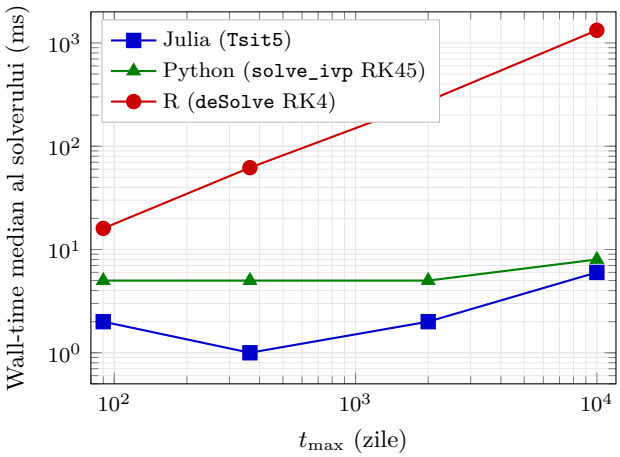


Figura 8.1: Scalarea wall-time-ului median al solverului SIDARTHE in functie de orizontul $t_{\max}$, pentru cele trei limbaje. R prezinta o crestere aproape liniara cu $t_{\max}$ (∼130× intre 90 si 10 000 de pasi), dominata de inserarile rand-cu-rand in baza de date prin `dbExecute()`; integrarea propriu-zisa cu `deSolve::ode` este sub-secundara. Python ramane stabil (4–8 ms), iar Julia este mai rapid in regimul amortizat dupa compilarea JIT, atingand 6 ms la 10 000 de pasi.

i R sunt opiunile mai economicoase.

8.7.2 Costul componentei LSTM

Runner-ul LSTM are un profil de cost diferit calitativ de runnerele compartimentale. Costul ODE depinde de orizontul de integrare $t_{\max}$ (numr de pai i de puncte salvate), în timp ce costul LSTM este dominat de *antrenare* — numrul de epoci × numrul de eantioane glisante — iar $t_{\max}$ controleaz doar orizontul prognozei autoregresive (neglijabil în comparaie).

Tabelul 8.8 arat rezultatele pe setul Romania OWID (1 440 zile, 80% antrenare / 20% test cronologic) cu parametrii impliciti din API. Mediana wall-time-ului este ∼290 s, aproape identic între $t_{\max} = 30$ i $t_{\max} = 90$, ceea ce confirm empiric c forecast-ul nu contribuie semnificativ. Utilizarea CPU este peste 98% din wall-time (LSTM hand-rolled în NumPy fr paralelism), iar memoria peak este de doar ∼56 MB, mai mic decât a runnerelor ODE în Python — nu se construiesc matrice de stare comparabile ca dimensiune.

Pentru contextul tezei, aceast valoare absolut este mai puin important decât observaia de scalare: pe seriile sintetice de 20 000 de zile, antrenarea ar crete la ∼70 minute cu aceiai parametri, ceea ce plaseaz LSTM ca operaie asincron lung — exact rolul pentru care `runner-lstm` este izolat într-o coad Redis dedicat.

8.7.3 Variabilitatea cross-hardware

Toate rezultatele de mai sus provin de pe un singur profil hardware (clusterul `kube-4kdhb`). Pentru a separa contribuia hardware-ului de cea a stack-ului software,

Tabela 8.8: Costul rularii pentru runner-ul LSTM dedicat (Python, hand-rolled NumPy), parametri impliciti (`lookback=14`, `hidden_size=12`, `epochs=150`, `learning_rate=0.01`), dataset Romania OWID (1 440 zile, 80% antrenare / 20% test cronologic). Forecast-ul autoregresiv pe orizontul $t_{\max}$ are cost neglijabil fata de antrenare.

$t_{\max}$	n	Wall median (s)	CPU median (s)	Memorie (MB)
30	2	296.8	295.3	55.6
90	2	286.2	285.0	56.2

aceeai logic de solver a fost rulat ca script standalone (`scripts/bench_local_solver.py`) pe o main local complet diferit — Apple M1 Pro arm64, nativ, fr container i fr persistare în baza de date — folosind exact aceleai tolerante numerice (`RK45`, rtol = 10^{-8}, atol = 10^{-10}), aceiai parametri i aceleai condiii iniiale ca rulrile prin aplicaie. Aa cum este indicat explicit în seciunea 8.4, aceast masurare se face în afara aplicaiei pentru a evita amestecarea variaiilor de hardware cu deciziile de ortografie ale containerului i ale bazei de date distribuite.

Tabelul 8.9 compar median wall-time pentru SIR i SIDARTHE la trei orizonturi temporale, pe workload-ul sintetic non-trivial (setul `synthetic_multiwave`, $I_0 = 100$ pentru a fora solverul s parcurg efectiv pai de integrare). Datele de cluster sunt re-rulate în mod identic prin API (între 1 i 3 repetri per combinaie, în funcie de campania anterioar), iar datele M1 provin din scriptul local (3 repetri per combinaie, cu o rulare de înclzire ignorat).

Dou observaii se desprind. (i) Pentru acelai cod Python i aceeai încrcare numeric, M1 Pro nativ este de ∼4×–6× mai rapid decât worker-ul amd64 al clusterului. Acest raport include diferena de microarhitectur (Apple Silicon vs. CPU x86 virtualizat în KVM), diferena de mediu (nativ vs. container) i costul redus al stratului de persistare (M1 nu scrie rezultatele într-o baz de date distribuit cu factor de replicare 3). Raportul este relativ stabil între modele i orizonturi, ceea ce sugereaz c marea parte a diferenei vine din execuia solver-ului propriu-zis, nu din overhead-uri fixe.

(ii) Pentru workload-ul trivial (setul Romania OWID cu $I_0 = 0$, ca în campania prin aplicaie), M1 nativ ruleaz în 0,4–0,8 ms per solver, iar cluster-ul în 3–9 ms (din Tabelul 8.4); diferena este mai mare (10× sau mai mult) tocmai pentru c, atunci când solver-ul nu are dinamic real, *toat* cheltuiala wall-time pe cluster vine din scrierea celor $t_{\max}+1$ rânduri în `simulation_results`, cost care nu exist în scriptul local.

Concluzia metodologic este c un raport *absolut* între cele dou hardware-uri depinde semnificativ de natura workload-ului (solver-light vs. solver-heavy) i de stack-ul software inclus în msurare. Recomandarea de limbaj din seciunea 8.10 rmâne valabil pe clusterul testat; pentru o decizie cantitativ pe un alt profil hardware — în special cu accelerare GPU pentru LSTM sau cu solvere implementate în Fortran/C++ legate static — aceeai campanie trebuie re-rulat local, iar scriptul `bench_local_solver.py` este punctul de pornire pentru aceast replicare.

8.8 Acuratee numeric i verificri de regresie

Corectitudinea rezultatelor este verificat prin conservarea populaiei. Pentru SIDARTHE, suma $S+I+D+A+R+T+H+E$ trebuie s rmân egal cu N la fiecare pas salvat; pentru SEIR, $S+E+I+R=N$; iar pentru modelele SIR/SIS/SIRS se verific suma compartimentelor definite de fiecare model.

Un al doilea criteriu este apropierea traiectoriilor obinute în cele trei limbaje pentru aceeai configuraie. Deoarece solverele difer, egalitatea exact nu este realist. Se verific în schimb abaterea maxim normalizat:

$$\Delta_{l_1,l_2} = \max_{t,c} \frac{|x_{l_1,c}(t) - x_{l_2,c}(t)|}{N}. \tag{8.4}$$

Dac aceast abatere rmâne sub un prag ales, implementrile sunt considerate compatibile funcional pentru scopul aplicaiei.

Verificarea a fost rulat concret pe setul de date sintetic `multiwave` (populaie $N = 50 \cdot 10^6$), pentru care condiiile iniiale produc dinamic ne-trivial (spre deosebire de Romania OWID, unde prima zi din serie are $I = 0$). Rezultatele sunt sintetizate în Tabelul 8.10.

Dou observaii se desprind. (i) Implementrile Julia i Python produc traiectorii practic identice ($\sim 10^{-8} \cdot N$, la limita preciziei flotante), pentru c ambele folosesc solvere Runge–Kutta adaptive cu tolerane similare (`Tsit5` i respectiv `RK45`). (ii) Implementarea R, care folosete RK4 cu pas fix $h = 1$ zi, deviaz de la celelalte dou cu $\sim 10^{-4} \cdot N$ (∼2 200 cazuri pentru SIR, ∼6 600 pentru SIDARTHE). Pentru o populaie de 50 de milioane, aceast abatere reprezint 0,013% din populaie i este sub-vizual pe o reprezentare grafic tipic — compatibil funcional cu scopul aplicaiei, dar deliberat mai puin precis decât metodele adaptive.

8.9 Evaluarea API-ului i a bazei de date

Performana perceput de utilizator depinde i de API i baza de date. Pentru crearea unei simulri, latena API include validarea inputului, citirea primului rând epidemiologic, inserarea în `simulation_runs` i publicarea în Redis. Pentru vizualizarea rezultatului, latena include citirea rândurilor din `simulation_results`, deserializarea JSONB i construirea rspunsului.

Din acest motiv, benchmark-ul numeric a fost completat cu o campanie de teste de API, realizat printr-un

Tabela 8.9: Comparatia timpilor de executie pentru solverul Python (`scipy.solve_ivp` RK45) pe doua profile hardware distincte: cluster `kube-4kdhb` (worker amd64, OpenStack VM, 4 vCPU / 8 GiB, runner in container, wall-time include si scrierea rezultatelor in YugabyteDB) vs M1 Pro arm64 nativ (script standalone, fara container, fara DB). Workload non-trivial: set sintetic `multiwave` cu $I_0 = 100$, $N = 5 \cdot 10^7$. Coloana *speedup M1* este raportul medianelor; valori > 1 inseamna M1 mai rapid.

Model	$t_{\max}$	n_{cluster}	Cluster wall (ms)	n_{M1}	M1 wall (ms)	Speedup M1
`sidarthe`	365	1	86.0	3	16.44	5.2×
	2000	3	127.0	3	31.46	4.0×
	10000	3	345.0	3	90.48	3.8×
`sir`	365	1	73.0	3	12.25	6.0×
	2000	3	96.0	3	16.07	6.0×
	10000	3	100.0	3	22.98	4.4×

Tabela 8.10: Verificare numerica de regresie cross-limbaj: abaterea maxima absoluta intre traiectoriile compartimentale produse in R, Julia si Python pentru aceiasi parametri si aceleasi conditii initiale, pe setul sintetic `multiwave` (populatia $N = 50 \cdot 10^6$, $t_{\max} = 365$ pasi zilnici). Modelele folosesc solvere diferite (R cu RK4 pas fix $h = 1$, Python cu RK45 adaptiv, Julia cu Tsit5), deci abaterile reflecta tolerantele numerice cumulate, nu erori de implementare.

Model	Pereche	Max $\lvert\Delta\rvert$	Normalizat ($/N$)
`sir`	`julia` vs `python`	0.5	$1{,}01 \cdot 10^{-8}$
	`julia` vs `r`	2185.8	$4{,}37 \cdot 10^{-5}$
	`python` vs `r`	2186.2	$4{,}37 \cdot 10^{-5}$
`sidarthe`	`julia` vs `python`	0.2	$3{,}72 \cdot 10^{-9}$
	`julia` vs `r`	6570.9	$1{,}31 \cdot 10^{-4}$
	`python` vs `r`	6571.1	$1{,}31 \cdot 10^{-4}$

script reproductibil (`scripts/bench_api.py`). Pentru fiecare model s-au msurat cele patru operaii expuse de API i percepute direct de utilizator: *create* (crearea unei simulri), *status* (interogarea strii), *results* (citirea traiectoriei complete în format JSON) i *export* (exportul CSV al aceleiai traiectorii). Latenele *create* au fost colectate din 12 repetri, iar cele de citire din 30 de repetri per combinaie; msurtorile au fost fcute prin `port-forward` ctre serviciul `api` din clusterul `kube-4kdhb`, deci includ tot lanul HTTP → API → YugabyteDB → HTTP, plus variabilitatea de reea introdus de tunel.

Tabelul 8.11 prezint rezultatele la orizontul de un an (365 puncte). Se desprind câteva observaii. (i) Latenele mediane sunt omogene între modele i se situeaz în intervalul 24–71 ms pentru toate endpointurile; modelul nu influeneaz semnificativ latena, deoarece costul este dominat de drumul dus-întors prin baza de date, nu de structura compartimental. (ii) Endpointul *create* rspunde asincron — el doar valideaz inputul, citete primul rând epidemiologic, insereaz în `simulation_runs` i public în coada Redis, înainte ca solverul s ruleze — deci latena sa nu reflect costul de integrare numeric, ci pe cel de iniiere a unui job. (iii) Dimensiunea rspunsului JSON depinde de model: SIDARTHE, cu opt compartimente, produce ∼49 KB fa de ∼36 KB pentru SIS cu dou compartimente, la acelai orizont. (iv) Percentila 95 este de 2,5–3 ori mai mare decât mediana (124–156 ms), reflectând cozile de laten tipice unei msurtori prin tunel; pe un acces direct în cluster acestea ar fi mai mici, dar raportarea lor este corect pentru profilul de testare.

Tabelul 8.12 izoleaz efectul orizontului pentru modelul cel mai greu (SIDARTHE), variind numrul de puncte de la 30 la 365. Dimensiunea rspunsului JSON crete aproape liniar cu numrul de puncte (de la ∼4 KB la ∼49 KB), în timp ce latena median *results* crete mult mai lent (de la ∼30 la ∼47 ms): la aceste dimensiuni, costul fix al interogrii domin costul de serializare. Exportul CSV produce constant rspunsuri de 5–10 ori mai mici decât JSON-ul echivalent (text plat, fr chei repetate pe rând), confirmând c formatul de transport conteaz pentru volumul transferat, chiar dac latena rmâne comparabil.

Concluzia practic este c, pentru orizonturile uzuale (≤ 365 puncte) i modelele studiate, limita sistemului nu este nici solverul (sub 10 ms pe cluster, conform Tabelului 8.4), nici transportul JSON, ci costul fix al drumului prin baza de date, de ordinul a câtorva zeci de milisecunde per cerere. Aceasta este o laten confortabil pentru o interfa interactiv i indic faptul c optimizrile viitoare ar trebui s vizeze reducerea numrului de cereri (de exemplu prin agregarea rezultatelor mai multor rulri într-un singur rspuns), nu accelerarea unei singure cereri.

8.10 Interpretarea rezultatelor

Evaluarea corect trebuie s combine trei axe: eroarea numeric, timpul de execuie i complexitatea modelului. Un model nu este automat superior doar pentru c are eroare mai mic. Dac obine aceast eroare cu un numr mare de parametri i cu timp de calibrare ridicat, poate fi mai puin potrivit pentru analize rapide. În sens invers, un model simplu i rapid poate fi foarte util pentru explorare, dar insuficient pentru întrebri clinice.

Combinând rezultatele msurate, recomandarea practic derivat din aceast campanie este urmtoarea. Pentru rulri numeroase i scurte (de exemplu explorarea sensibili-

Tabela 8.11: Latenta API pe model, la orizontul de 365 puncte (1 an, pas zilnic). Pentru fiecare endpoint se raporteaza mediana (p_{50}); pentru *results*, endpointul cel mai greu, se raporteaza si percentila 95 (p_{95}) si dimensiunea raspunsului JSON. *create* raspunde asincron (inainte de rularea solver-ului); *results* si *export* includ deserializarea din YugabyteDB. Masurat prin port-forward la clusterul `kube-4kdhb`.

Model	**Create** (ms)	**Status** (ms)	**Results** p_{50} (ms)	**Results** p_{95} (ms)	**JSON** (KB)	**Export** (ms)
`sir`	46.4	23.9	48.5	139.8	38.1	48.3
`sis`	70.6	31.5	45.5	123.9	36.0	39.6
`seir`	49.9	27.0	53.8	155.6	40.3	36.6
`sirs`	50.0	28.3	46.9	124.0	38.1	37.3
`sidarthe`	44.2	29.7	47.1	126.3	48.9	39.3

Tabela 8.12: Scalarea latentei de citire cu orizontul, pentru modelul `sidarthe` (8 compartimente). Pe masura ce numarul de puncte creste, latenta *results* si dimensiunea JSON cresc aproximativ liniar, in timp ce *export* CSV ramane mai ieftin (text plat, fara structura JSON).

Puncte	**Results** p_{50} (ms)	**Results** p_{95} (ms)	**JSON** (KB)	**Export** (ms)
30	30.5	135.6	4.1	31.1
90	34.3	129.7	12.1	35.5
180	40.0	113.4	24.1	34.7
365	47.1	126.3	48.9	39.3

tii parametrilor), **Python** este alegerea implicit cea mai bun — timpii sunt sub 10 ms i predictibili pentru toate modelele i orizonturile testate, fr cost de pornire. Pentru rulri puine i foarte lungi (de exemplu integrare pe orizonturi $t_{\max} \gg 10\,000$ sau pentru calibrare cu mii de iteraii în care Julia se compileaz o singur dat), **Julia** ofer un solver cu un ordin de mrime mai rapid, cu condiia ca cele $\sim$1 secund de pornire s fie amortizate. **R** rmâne potrivit pentru explorare interactiv în pipeline-uri statistice familiare cercettorilor din epidemiologie, dar runner-ul curent are o limitare operaional (inseriile rând-cu-rând în baza de date) care domin wall-time-ul la orizonturi mari i care poate fi corectat independent de schimbarea limbajului.

Pentru predicie cu LSTM, costul este dominat de antrenare i nu se compar direct cu solverele ODE. Este important ca interfaa utilizatorului s trateze LSTM ca operaie asincron explicit (lansare, polling, finalizare dup minute), aa cum este deja proiectat în aplicaie.

Toate aceste afirmaii sunt valide pentru profilul hardware al clusterului `kube-4kdhb`. Reproducerea pe alte configuraii (maini locale, noduri mai puternice, eventual GPU pentru LSTM) este lsat ca extensie a campaniei, prin scripturi rulate în afara aplicaiei.

Capitolul 9

Aplicaia Web E-Medicine

9.1 Rolul aplicaiei

Aplicaia web este interfaa prin care modelul matematic i infrastructura de execuie devin utilizabile. Ea nu este un sistem clinic complet, ci un mediu de lucru pentru organizarea datelor pe studii epidemiologice, încrcarea dataseturilor, configurarea simulrilor SIDARTHE, SIR, SIS, SEIR sau SIRS i compararea rezultatelor obinute în R, Julia i Python. Din aceast perspectiv, aplicaia are rol de instrument experimental i colaborativ.

Fluxul principal începe cu autentificarea utilizatorului. Dup autentificare, acesta alege un studiu sau creeaz unul nou. Toate operaiile ulterioare se raporteaz la studiul curent, iar interfaa afieaz aciunile disponibile în funcie de rolul utilizatorului: `admin`, `researcher`, `contributor` sau `viewer`. Utilizatorii cu rol suficient pot încrca un fiier CSV cu date epidemiologice, pot alege modelul, pot introduce parametrii i pot urmri statusul execuiei. Când runner-ul finalizeaz job-ul, interfaa afieaz traiectoriile compartimentale i permite exportul rezultatelor în format CSV. O pagin separat permite compararea mai multor rulri finalizate, iar pagina de benchmark prezint agregrile calculate la nivel de model i limbaj.

Pe lâng aplicaia principal exist o interfa `frontend-admin`, servit de pe host-ul `admin.sir-workbench.ro`. Aceasta este disponibil doar conturilor cu `is_super_admin=true` i ofer statistici globale, gestionarea utilizatorilor i vizualizarea tuturor studiilor.

9.2 Backend Go

Backend-ul este implementat în Go i expune un API REST sub prefixul `/api`. Serverul folosete routerul `chi`, middleware pentru logging, recuperare dup panic, timeout de request i CORS, iar accesul la rutele protejate este controlat prin JWT. Alegerea Go este potrivit pentru aceast component deoarece API-ul este stateless, are nevoie de latene predictibile i poate fi distribuit uor sub form de binar static în container.

Structura codului separ intrarea HTTP de accesul la date. Pachetul `handlers` parseaz cererile, valideaz input-ul i construiete rspunsurile JSON. Pachetul `repository` conine interogrile SQL i operaiile tranzacionale peste `pgx`. Pachetul `runner` definete schema mesajului trimis în Redis i publicarea job-urilor. Contractul de simulare include tipul modelului, parametrii, condiiile iniiale i conexiunea la baza de date.

Rutele aplicaiei principale sunt organizate sub `/api/studies/{studyId}`. Middleware-ul RBAC verific membership-ul i rolul înainte ca handler-ul s ajung la date. Super-adminii au acces global, dar consola administrativ folosete rute dedicate sub `/api/admin`.

9.3 Gestionarea dataseturilor

Dataseturile sunt încrcate printr-un formular multipart, în contextul studiului curent. API-ul accept câmpurile `name`, `disease_code` i `file`; fiierul trebuie s conin cel puin coloanele `date`, `region`, `infected` i `population`. Coloanele `recovered` i `susceptible` sunt opionale, dar atunci când sunt prezente sunt folosite pentru condiiile iniiale. Pentru SIDARTHE, cazurile active sunt plasate iniial în compartimentul I, persoanele ieite din infecie în H, iar compartimentul E al deceselor pornete de la zero. Pentru SEIR, compartimentul expuilor pornete de asemenea de la zero, deoarece nu este observat direct în CSV.

Persistarea datasetului se face într-o tranzacie. Mai întâi este creat rândul din `datasets`, apoi înregistrrile epidemiologice sunt inserate prin `CopyFrom`. Aceast alegere reduce costul încrcrii CSV-urilor fa de inserarea rând-cu-rând i pstreaz atomicitatea: dac una dintre înregistrri nu poate fi scris, setul de date nu rmâne într-o stare parial. La tergere, `admin`-ul poate elimina orice set de date din studiu, în timp ce `contributor`-ul poate elimina doar seturile de date încrcate de el.

9.4 Execuia simulrilor

Crearea unei simulri este un proces scurt în API i un proces asincron în ansamblul sistemului. Endpointul `POST /api/studies/{studyId}/simulations` verific existena datasetului în studiul curent, modelul, limbajul selectat, parametrii i $t_{\max}$, apoi extrage prima înregistrare epidemiologic pentru a determina valorile iniiale. Dup ce rândul din `simulation_runs` este creat cu

status `pending`, API-ul public mesajul JSON în coada Redis corespunztoare limbajului.

Runner-ele sunt responsabile pentru tranziiile ulterioare. Ele marcheaz rularea ca `running`, execut solverul pentru `model_type`, insereaz rezultatele i benchmarkul, apoi marcheaz rularea ca `completed`. Dac apare o eroare, statusul devine `failed`, iar mesajul este salvat în baza de date. Interfaa web nu trebuie s in o conexiune deschis pe durata calculului; pagina de rezultat interogheaz periodic endpointul de status pân la finalizare.

9.5 Editor de cod pentru cercettori

Pe lâng cele ase modele built-in i runner-ele plug-in înregistrate de super-admin (seciunea 4.7), aplicaia include un *editor de cod* accesibil oricrui membru al studiului cu drept de scriere de simulri. Aceast component rspunde unei cerine distincte: cercettorul vrea s încerce rapid un algoritm propriu (o regresie, un filtru, o euristic de prognoz) peste dataset-ul disponibil, fr a parcurge procesul de a construi o imagine container i a o înregistra ca runner persistent.

9.5.1 Flux de lucru

Pagina `/script-editor` prezint un layout în dou coloane: în stânga, o list vertical cu draft-urile salvate ale cercettorului în studiul curent; în dreapta, un editor Monaco (acelai engine ca VS Code) preîncrcat cu un script demonstrativ de netezire exponenial i un set iniial de parametri. Sub editor se gsesc trei controale: dropdown-ul de dataset, un textarea cu parametrii în format JSON i butonul “Ruleaz”. La apsarea butonului, clientul trimite o cerere `POST /simulations` cu `model_type = 'script'` i coninutul curent al editorului; aceeai pagin polleaz apoi endpointul de status i, la finalizare, afieaz rezultatele cu acelai component grafic folosit pentru toate celelalte tipuri de rulri.

Salvarea draft-urilor este o operaie separat, decuplat de execuie. Cercettorul introduce un nume în antetul workspace-ului i apas “Salveaz”; clientul face `POST /script-drafts` (sau `PATCH /script-drafts/{id}` dac draftul are deja un id), iar în sidebar apare o intrare nou marcat cu data ultimei actualizri. Click pe un draft din sidebar îl reîncarc în editor împreun cu parametrii salvai, iar butonul “Œ” îl terge dup confirmare. Cu acest flux, codul scris în pagin nu se pierde la reîncrcarea browser-ului i este accesibil din orice device pe care cercettorul se autentific.

9.5.2 Contractul scriptului

Scriptul are aceeai form cu cea declarat pentru runner-ele plug-in: o singur funcie `main` cu o semntur fix.

```
def main(records, parameters, context):
    """
    records:    randurile dataset-ului, fiecare cu
                cheile: date, region, susceptible,
                infected, recovered, population
                + (opt) deaths, new_cases, new_deaths,
                hospitalized, icu, vaccinated, tested,
                stringency_index, reproduction_rate
    parameters: dict cu parametri (validati JSON
                in UI)
    context:    {simulation_id, population_n,
                t_max}
    return:     [{"t": int, "compartments":
                  {nume: float, ...}}, ...]
    """
```

Listing 9.1: Contractul scriptului din editor.

Returnarea este aceeai list de pai temporali cu obiecte `compartments` ca pentru runner-ele built-in, ceea ce permite graficului existent s afieze rezultatele fr tratare special. Cheile epidemiologice extinse din `records` sunt populate atunci când sursa CSV de la upload le-a furnizat (*e.g.*, dataseturile OWID COVID au de regul `deaths`, `stringency_index` i `reproduction_rate` pentru mare parte din 2020–2023); altfel valoarea cheii este `None`, iar scriptul trebuie s trateze explicit absena. Stack-ul Python disponibil în executor este fix (`numpy`, `scipy`, `pandas`, `scikit-learn`); cercettorul care are nevoie de dependene netriviale poate trece pe traseul de runner cu imagine extern.

9.5.3 Execuie i separare de runner-e persistente

Scriptul este executat de un Deployment permanent numit `runner-script`, care ascult pe coada Redis `epimodel:jobs:script`. Pentru fiecare job, executorul scrie coninutul scriptului într-un fiier temporar, îl compileaz ca modul Python, apeleaz `main(records, parameters, context)` cu msurarea timpilor wall i CPU i a memoriei de vârf, valideaz structura rezultatului i scrie traiectoriile în `simulation_results` alturi de o intrare în `benchmarks` — exact ca runner-ele built-in. Modulul este reîncrcat la fiecare job pentru a nu lsa stare global între execuii.

Aceast arhitectur separ explicit dou responsabiliti. Editorul de cod este pentru *explorare ad-hoc*: nu exist resurse Kubernetes create per script, executorul partajat absoarbe toate cererile, iar permisiunea este deschis oricrui cercettor cu drept de scriere de simulri. Runner-

ele cu imagine container (capitolul 4, seciunea 4.7) sunt pentru *algoritmi operationali*: imagine construit extern, Deployment dedicat în cluster, acces de înregistrare restrâns la super-admin.

9.5.4 Izolare între utilizatori i securitate

Draft-urile sunt strict private cercettorului care le-a creat. Tabela `script_drafts` are coloana `user_id` NOT NULL, iar toate interogrile din `ScriptDraftRepository` filtreaz cumulativ pe `(user_id, study_id, id)`: chiar dac un utilizator încearc s cear direct id-ul unui draft creat de un alt cercettor (atac de tip IDOR), API-ul rspunde `404 not found`. Listarea expune doar metadatele (`script_content` este omis); coninutul este servit doar la deschiderea explicit a unui draft, ceea ce pstreaz compact rspunsul de listare i clarific intenia fiecrei cereri.

Dimensiunea scriptului trimis la execuie i a draft-ului salvat este plafonat la 1 MB, suficient pentru orice script de explorare obinuit i strict mai mic decât payload-urile pe care le-ar tolera baza de date. Validarea formatului parametrilor (JSON) i a structurii rezultatului (`list[{t, compartments}]`) se face server-side, astfel încât un script ru formatat este respins cu un mesaj clar înainte de a polua tabelele de rezultate.

Frontend-ul principal este o aplicaie Vue 3 construit cu Vite i TypeScript. Rutarea este gestionat cu Vue Router, iar starea aplicaiei cu Pinia. Clientul HTTP este o instan Axios configurat cu prefixul `/api`; un interceptor adaug automat headerul `Authorization` atunci când exist token în `localStorage`. Store-ul `studies.store.ts` pstreaz studiul curent i rolul curent, iar store-urile pentru seturi de date, simulri i benchmark-uri construiesc URL-urile în funcie de acest `studyId`.

Paginile sunt organizate dup fluxurile principale ale aplicaiei: `StudiesPage` selecteaz contextul de lucru, `CreateStudyPage` creeaz un studiu nou, `StudyManagementPage` gestioneaz membrii i rolurile, `DashboardPage` sumarizeaz seturile de date i simulrile recente, `DatasetsPage` gestioneaz încrcarea CSV-urilor, `NewSimulationPage` colecteaz modelul i parametrii, `SimulationResultPage` afieaz statusul i rezultatele, `ComparePage` suprapune seriile mai multor rulri, iar `BenchmarksPage` afieaz agregrile pe model i limbaj. Componenta `CompartmentChart` folosete Chart.js prin `vue-chartjs` i construiete dinamic seriile în funcie de cheile disponibile în `compartments`. Astfel, acelai grafic poate afia modele cu dou, trei, patru sau opt compartimente fr modificri suplimentare.

Decizia de a folosi polling la dou secunde pentru actualizarea statusului unei simulri este simpl, dar adecvat pentru durata ateptat a job-urilor. O soluie bazat pe WebSocket sau Server-Sent Events ar reduce numrul de cereri, îns ar introduce stare suplimentar în backend. Pentru un API stateless i pentru job-uri care dureaz de la câteva secunde la câteva minute, polling-ul pstreaz implementarea previzibil.

9.6 Consola super-admin

Consola administrativ este un build Vue separat, cu propriul router i propriul token în `localStorage`. La autentificare, aplicaia verific rspunsul `/api/auth/login`; dac utilizatorul nu este super-admin, tokenul nu este pstrat i login-ul este refuzat. Aceast separare reduce riscul ca aciunile globale de administrare s fie amestecate cu fluxurile obinuite de lucru ale unui studiu. Toate paginile sunt deliberat *de citire*: mutaiile asupra studiilor i seturilor de date rmân în aplicaia principal, controlate prin RBAC pe studiu.

Dashboard. Pagina iniial afieaz cele opt contoare globale (`/api/admin/stats`) i ase distribuii agregate între studii, obinute din `/api/admin/breakdown`: simulri pe status, model i limbaj; seturi de date i studii pe boal; top zece studii dup volumul cumulat de înregistrri. Distribuiile sunt randate ca bare orizontale proporionale, iar top-ul de studii are link-uri direct ctre pagina de detaliu.

Utilizatori i studii. Pagina de utilizatori (`/api/admin/users`) listeaz toate conturile cu un buton care comut flag-ul `is_super_admin` prin `PATCH /api/admin/users/{userId}/super-admin`. Pagina de studii (`/api/admin/studies`) listeaz toate studiile cu un câmp de filtrare text i permite navigarea ctre pagina de detaliu prin click pe rând.

Detaliu studiu. `StudyDetailPage` reutilizeaz endpointurile `/api/studies/{studyId}/*` (membri, seturi de date, simulri, benchmark-uri agregate), de care superadminul beneficiaz datorit bypass-ului din middleware-ul RBAC. Pagina prezint, în ordine, un antet cu metadatele studiului i patru contoare (membri, seturi de date cu volum cumulat, simulri, combinaii model × limbaj cu rulri complete), urmate de tabelele detaliate. Simulrile au badge de status colorat, iar tabelul de benchmark-uri arat wall-time median, percentila 95, CPU median i memorie median pentru fiecare combinaie model–limbaj.

Activitate. Pagina `/activity` consum `/api/admin/recent-simulations` i `/api/admin/recent-datasets` pentru a afia cele mai recente N intrri (configurabil între 20 i 200), indiferent de studiu, cu link-uri ctre studiul corespunztor. Acest feed este util pentru audit operaional, pentru identificarea rapid a simulrilor blocate în starea `running` i pentru verificarea încrcrilor de seturi de date proaspete.

9.7 Consideraii de securitate

Fiind un sistem care gestioneaz date de cercetare grupate pe studii, cu acces difereniat pe roluri, securitatea este o cerin funcional, nu un adaos ulterior. Aceast seciune documenteaz mecanismele implementate efectiv i

limitrile cunoscute, pe baza unei treceri în revist a codului backend.

9.7.1 Mecanisme implementate

Autentificare. Identitatea este stabilit prin token JWT semnat cu HMAC-SHA256. Middleware-ul de validare verific explicit c metoda de semnare a token-ului este HMAC înainte de a accepta cheia, respingând astfel atacul de confuzie de algoritm (un token forjat cu `alg=none` sau cu o cheie public RSA reinterpretat ca secret HMAC). Expirarea (`exp`) este verificat la fiecare cerere, iar secretul JWT este obligatoriu la pornirea serviciului: în absena lui aplicaia refuz s porneasc, ceea ce elimin riscul unui secret implicit slab.

Parole. Parolele nu sunt niciodat stocate în clar, ci ca hash bcrypt cu factor de cost 12. La autentificare se folosete o comparaie în timp constant, iar mesajul de eroare este identic pentru email inexistent i parol greit (*invalid credentials*), pentru a nu dezvlui ce conturi exist.

Autorizare (RBAC contextual). Accesul este organizat pe studii, cu patru roluri: `admin`, `researcher`, `contributor`, `viewer`, plus flag-ul global `is_super_admin`. Regulile sunt aplicate ca lan de middleware-uri pe rute: citirea este permis oricrui membru, încrcarea de seturi de date cere rol `admin` sau `contributor`, crearea de simulri cere `admin` sau `researcher`, iar administrarea studiului (membri, tergere) este rezervat rolului `admin`. Endpointurile de platform (`/api/admin/*`) sunt protejate separat prin verificarea super-adminului.

Izolarea resurselor între studii. Resursele imbricate sunt interogate întotdeauna în contextul studiului din URL: o cerere de tipul `/studies/{A}/simulations/{id}` verific nu doar c utilizatorul este membru al studiului `A`, ci i c simularea `id` aparine studiului `A`. Astfel, un membru al unui studiu nu poate citi simulri sau seturi de date dintr-un alt studiu prin substituirea identificatorului (atac de tip IDOR).

Integritatea interogrilor i a intrrilor. Toate interogrile ctre YugabyteDB folosesc parametri legai (`$1, $2, ...`), nu concatenare de iruri, eliminând injecia SQL. Parametrii simulrilor sunt validai pe server: tipul modelului i limbajul sunt verificate fa de liste albe, ratele negative sunt respinse, iar hiperparametrii LSTM sunt restrâni la intervale întregi valide. Încrcrile de fiiere sunt plafonate la 32 MB prin `MaxBytesReader`, prevenind epuizarea memoriei la parsarea CSV.

9.7.2 Limitri cunoscute i remedieri

Trecerea în revist a identificat i dou probleme punctuale, remediate în cursul acestei lucrri. Prima: endpointul de creare a simulrilor accepta orice $t_{\max} > 0$, fr plafon superior, ceea ce permitea unei cereri legitime s declaneze o integrare arbitrar de lung i scrieri masive în baza de date — o cale de epuizare a resurselor. S-a introdus un plafon explicit ($t_{\max} \leq 100\,000$), care acoper cu marj orizonturile folosite în campania de benchmark. A doua: la autentificare, un email inexistent returna rspunsul înainte de verificarea bcrypt, astfel încât diferena de laten putea fi folosit pentru a enumera conturile valide (timing oracle). S-a adugat o comparaie bcrypt cu un hash fals pe ramura "email inexistent", egalizând timpul de rspuns.

Rmân limitri tratate ca direcii ulterioare. Nu exist înc limitare a ratei cererilor pe endpointul de autentificare, ceea ce las deschis posibilitatea atacurilor de tip brute-force sau credential stuffing; soluia natural este un middleware de rate limiting per IP. Token-urile JWT sunt fr stare, deci nu pot fi revocate înainte de expirare (24 h); pentru un control mai fin ar fi necesar o list de revocare sau token-uri de reîmprosptare cu durat scurt. În fine, politica CORS este permisiv (`AllowedOrigins: *`), ceea ce este acceptabil în configuraia curent — autentificarea se face prin antet `Authorization` cu token Bearer, nu prin cookie, iar `AllowCredentials` este dezactivat, deci nu exist suprafa de atac CSRF — dar ar trebui restrâns la originile cunoscute într-un deployment public.

9.8 Validare funcional

Validarea funcional a aplicaiei poate fi fcut prin fluxuri end-to-end care urmeaz operaiile reale ale utilizatorului: înregistrare, autentificare, creare sau selectare studiu, adugare membru, încrcare set de date, pornire simulare SIDARTHE, ateptare finalizare, afiare grafic i export CSV. Aceeai suit ar trebui s verifice i rulrile SIR/SIS/SEIR/SIRS, deoarece acestea asigur compatibilitatea cu familia de modele discutat în capitolul teoretic i cu comparaiile simple. Un flux separat valideaz consola admin: login cu un cont super-admin, listarea utilizatorilor, modificarea flag-ului `is_super_admin` i listarea studiilor globale.

Capitolul 10

Concluzii i Direcii de Dezvoltare Viitoare

10.1 Sinteza lucrrii

Lucrarea a fost reorientat în jurul modelului SIDARTHE, deoarece acesta descrie mai bine situaiile în care diagnosticul, simptomatologia, severitatea clinic i mortalitatea influeneaz evoluia observat a unei epidemii. Modelele SIR/SIS/SEIR/SIRS rmân prezente, dar rolul lor este secundar: ofer repere matematice i numerice simple, utile pentru comparaii i verificri de regresie.

Pe aceast baz a fost construit un sistem software în care datele, simulrile i metricile de performan sunt tratate unitar. API-ul Go ofer autentificare, organizare pe studii, control RBAC, încrcare de seturi de date i pornire de simulri; Redis decupleaz cererile utilizatorului de execuia numeric; runner-ele R, Julia i Python rezolv modelul cerut prin `model_type`; cele dou interfee Vue.js separ fluxurile de lucru ale cercettorilor de operaiile globale ale super-adminilor. Contribuia practic a sistemului const în aceast legtur între model, execuie, izolare organizaional i observabilitatea rezultatelor.

Lucrarea a fost extins cu un cadru de comparaie numeric i calibrare. Aceast extensie nu este doar metodologic: a fost executat concret o campanie de benchmark prin aplicaie (180 rulri ODE plus 4 rulri LSTM, fr eecuri), iar calibrarea SIR cu $\gamma = 1/14$ a fost aplicat pe Romania i pe trei ri europene suplimentare (Italia, Germania, Spania), rezultatele fiind raportate ca tabele i figuri în capitolele 7 i 8. În plus, a fost introdus o direcie predictiv bazat pe LSTM, tratat ca extensie statistic pentru prognoz pe termen scurt i pentru analiza reziduurilor rmase dup simularea compartimental, cu rezultate operaionale pe seria Romania OWID.

10.1.1 Rspuns la obiectivele propuse

Cele patru obiective formulate în seciunea 1.3 au fost atinse concret, dup cum urmeaz.

Obiectivul principal — proiectarea unui sistem care leag modelarea matematic de implementarea software, cu SIDARTHE ca model de lucru. A fost construit întregul lan, de la formalismul cu opt compartimente i aisprezece rate (capitolul 2) pân la implementarea în trei limbaje (capitolul 5), expus printr-un API Go cu organizare pe studii i control RBAC (capitolul 4), rulat pe un cluster Kubernetes real (`kube-4kdhb`). SIDARTHE este tratat ca model clinic principal pe tot parcursul lucrrii, iar familia SIR/SIS/SEIR/SIRS ofer reperele de baz.

Al doilea obiectiv — acelai contract de simulare în R, Julia i Python. Cele trei runnere au fost implementate cu bibliotecile uzuale ale fiecrui ecosistem (`deSolve`, `DifferentialEquations.jl`, `scipy.integrate`) i folosesc acelai contract de mesaj prin Redis i aceeai schem de rezultate în YugabyteDB. Verificarea de regresie cross-limbaj (Tabelul 8.10) arat c abaterile între traiectoriile produse sunt sub $10^{-4} \cdot N$ pentru toate modelele, iar Tabelul 8.4 compar direct costurile de execuie.

Al treilea obiectiv — metodologie explicit de comparaie i calibrare. Comparaia folosete operatori de observaie definii per model (seciunea 7.3), eroare MAE/RMSE pe ferestre separate de antrenare i test, iar calibrarea a fost rulat efectiv: pe Romania cu analiz de identifiabilitate (Figura 7.2) i pe patru ri simultan (Tabelul 7.4). Costul software al calibrrii a fost msurat i raportat alturi de eroarea numeric (Tabelul 7.5), conform principiului enunat în obiectiv.

Obiectivul suplimentar — extensie predictiv bazat pe LSTM. Componenta este implementat ca runner Python dedicat (`runner-lstm`), antrenat handrolled în NumPy, cu rezultate operaionale pe seria Romania (Figura 6.1). Validarea fa de predictori naivi (Tabelul 6.2) este onest: LSTM-ul univariat nu bate persistena la orizontul de o zi pe aceast serie, ceea ce delimiteaz clar regimul în care componenta are sens (corecie de reziduuri sau intrri multivariate), aa cum era anticipat în obiectiv.

10.2 Concluzii privind modelele

SIDARTHE introduce o structur mai apropiat de realitatea raportrii epidemiologice decât SIR. Separarea cazurilor nedepistate de cele diagnosticate, a formelor uoare de cele severe i a deceselor permite formularea unor întrebri care nu pot fi abordate natural într-un model cu trei compartimente. În acelai timp, aceast granularitate crete numrul de parametri i face calibrarea mai dificil.

Modelele SIR, SIS, SEIR i SIRS rmân utile tocmai prin simplitate. Ele permit verificarea conservrii populaiei, a pozitivitii soluiei i a comportamentului solverelor

într-un cadru uor de controlat. În sistemul implementat, aceste modele sunt tratate ca repere secundare, nu ca explicaii complete ale datelor reale.

Comparaia numeric trebuie interpretat împreun cu structura modelului. Un RMSE mai mic nu este suficient pentru a declara un model superior dac acesta necesit mai muli parametri neobservabili sau dac supraînva fereastra de calibrare. Pentru SIDARTHE, valoarea real este dat de capacitatea de a formula întrebri clinice mai bogate, nu doar de reducerea unei erori agregate.

10.3 Concluzii privind implementarea

Implementarea arat c acelai contract de simulare poate fi rulat în ecosisteme diferite fr a schimba API-ul sau interfaa web. R ofer o formulare compact prin `deSolve`; Python ofer integrare direct cu infrastructura i bibliotecile tiinifice uzuale; Julia este potrivit pentru experimente numerice mai ambiioase, mai ales atunci când costul de compilare este amortizat.

Schimbarea schemei rezultatelor din coloane fixe `s/i/r` într-un obiect JSONB `compartments` este una dintre modificrile importante ale pivotrii. Ea face sistemul mai flexibil i permite adugarea unor modele noi fr migraii ale tabelei de rezultate. Costul acestei flexibiliti este o serializare mai general i o nevoie mai mare de validare la nivel de aplicaie.

Pe latura operaional, sistemul a fost deployat efectiv pe un cluster Kubernetes v1.31 gzduit pe o platform cloud privat, cu trei noduri control plane i trei workeri, YugabyteDB la factor de replicare 3 i Helm pentru ciclul de via al bazei de date. Aceast experien confirm portabilitatea arhitecturii i pune în eviden câteva ajustri specifice mediilor distribuite, precum aplicarea migraiilor prin `psql` în locul `golang-migrate` sau adugarea extensiei `pgcrypto` la momentul instalrii, datorit bazei PostgreSQL 11 a YugabyteDB. Aceste ajustri sunt locale i nu afecteaz nici codul aplicativ, nici interfaa REST.

10.4 Concluzii experimentale

Cele patru zone empirice ale lucrrii — calibrarea pe date reale, campania de benchmark prin aplicaie, verificarea de regresie cross-limbaj i msurtorile cross-hardware — produc rezultate care nu trebuie citite în izolare, ci ca un set coerent de constrângeri pentru luarea deciziilor de modelare i de implementare. Aceast seciune sintetizeaz firul empiric înainte de discuia limitrilor i a direciilor viitoare.

Calibrarea sustine alegerea constrangerilor din literatura. Comparaia dintre SIR liber i SIR constrans (cu $\gamma = 1/14$) pe valul 1 COVID-19 în Romania (subseciunea 7.6.1) arat c erorile pe fereastra de antrenare sunt aproape identice ($\sim$1 600 cazuri MAE pentru ambele scenarii), dar parametrii estimai difer substanial ($\hat{\gamma}$ liber = 0,335, adic 3 zile de infeciozitate, contra valorii clinice de 14 zile). Peisajul de pierdere (subseciunea 7.7.1) confirm vizual cauza: valea de pierdere mic este alungit în (β, γ), deci datele nu disting între aceste combinaii. Extinderea la patru ri (subseciunea 7.7.2) arat c procedura constransa este robust cross-regiune — $\hat{\mathcal{R}}_0$ rmâne între 0,94 (Italia) i 1,45 (Romania), valori sub intervalul de literatur 2–3 pentru tulpina original, ceea ce este consistent cu interveniile non-farmaceutice deja active în fereastr. În toate cazurile, divergena pe fereastra de test este atribuibil unei schimbri de regim epidemic (val 2), nu unei deficiene a calibrrii: este o limitare structural a SIR cu coeficieni constani.

Benchmark-ul reveleaza trei regimuri de cost calitativ diferite. Pe clusterul `kube-4kdhb` (capitolul 8), Julia este solver-ul cel mai rapid în regim amortizat (5–6 ms la $t_{\max} = 10\,000$, indiferent de model), dar pltete un cost de pornire JIT de $\sim$900–1 500 ms pe prima rulare. Python (SciPy) este cel mai stabil ca performan (4–9 ms peste tot, fr cold-start), iar R (`deSolve`) este de dou pân la trei ordine de mrime mai lent la orizonturi mari — nu din cauza solver-ului în sine, ci a buclei de inserri rând-cu-rând în baza de date. Aceast distincie este important pentru dou decizii practice: pentru calibrare, care face zeci sau sute de evaluri succesive ale solver-ului, Julia este alegerea natural deoarece costul JIT se amortizeaz; pentru interfaa interactiv a aplicaiei, unde predictibilitatea conteaz mai mult decât viteza brut, Python rmâne default-ul rezonabil.

Implementarile sunt compatibile numeric, dar nu identice. Verificarea cross-limbaj (seciunea 8.8) arat c Julia i Python produc traiectorii identice la nivelul preciziei flotante ($\sim 10^{-8} \cdot N$), deoarece ambele folosesc solvere Runge–Kutta adaptive cu tolerane similare (`Tsit5` i respectiv `RK45`). Implementarea R, care folosete RK4 cu pas fix $h = 1$ zi, deviaz de la celelalte dou cu $\sim 10^{-4} \cdot N$ (0,013% din populaie) — diferen deliberat din alegerea solver-ului, nu eroare de implementare. Acest rezultat d o limit cantitativ a interschimbabilitii runnerelor: pentru analize cu cerine de precizie sub $10^{-3} \cdot N$, R este acceptabil; pentru reproducerea exact a unei traiectorii Julia, Python este interschimbabilul direct.

Hardware-ul conteaza, dar nu inlocuieste analiza per-stack. Comparaia cross-hardware (subseciunea 8.7.3) arat c acelai cod Python ruleaz de 4 pân la 6 ori mai rapid pe Apple M1 Pro arm64 nativ decât pe un worker amd64 din cluster, pe workload-uri solver-heavy. Pentru workload-uri solver-light (initial trivial), raportul crete la 10× sau mai mult, dar acolo diferena nu mai reflect hardware-ul brut, ci costul fix al persistrii în baza de date distribuit. Concluzia operaional este c etapele de explorare rapid (calibrri experimentale, sensitivity sweeps, prototipare LSTM cu hiperparametri variai) sunt sensibil mai eficiente local, în timp ce clusterul aduce concurena, izolarea pe studii i persistena necesare pentru o utilizare colaborativ real.

LSTM este o componenta calitativ diferita de

solverele ODE. Cele patru rulri LSTM (subseciunea 8.7.2) au costat ∼290 secunde fiecare pe seria Romania OWID (1 440 zile, 80%/20% split), de aproximativ *cinci ordine de mrime* peste solverele ODE pe acelai cluster. Acest decalaj confirm decizia arhitectural de a izola `runner-lstm` într-o coad Redis dedicat, separat de runnerele compartimentale; figura observat/fitted/predicted pentru Romania (Figura 6.1) confirm c prediciile o-zi-înainte urmresc rezonabil seria, dar prognoza autoregresiv se aplatizeaz rapid — LSTM-ul actual este un instrument de prognoz pe termen scurt i de detectare a reziduurilor, nu un model autonom de extrapolare epidemiologic pe orizonturi mari. Validarea fa de repere triviale (Tabelul 6.2) întrete aceast concluzie cu cifre: pe fereastra de test din seria României, persistena rmâne superioar LSTM-ului la orizontul de o zi, deoarece 85% din zile sunt plate (raportare sptamânal). Acest rezultat onest delimiteaz clar rolul componentei: valoare adugat plauzibil doar în regimul hibrid de corecie a reziduurilor sau cu intrri multivariate, nu ca predictor univariat de sine stttor.

10.5 Limitri

Prima limitare este calibrarea parametrilor SIDARTHE. Datele OWID folosite în lucrare nu ofer direct distribuia pe compartimentele D, A, R, T i E. Iniializarea folosit de sistem este reproductibil, dar nu înlocuiete o estimare epidemiologic dedicat. Calibrarea SIR a fost realizat concret pe patru ri europene cu rezultate interpretabile; pentru SIDARTHE, îns, este necesar integrarea unor serii suplimentare (testare, spitalizare, terapie intensiv, vindecri) i a unor operatori de observaie multipli, care s constrânga subgrupuri de parametri separat.

Calibrarea este realizat în lucrare ca script offline, în afara aplicaiei (`scripts/calibrate.py` i `scripts/calibrate_multiregion.py`). Promovarea acesteia la un tip de job în Redis, alturi de `simulation_runs`, ar permite expunerea în interfaa web i stocarea istoricului parametrilor estimai — o extensie de sistem, nu o re-derivare metodologic.

A doua limitare privete acoperirea evalurii empirice. Campania de benchmark prin aplicaie a fost executat pe un singur profil hardware (clusterul `kube-4kdhb`, 3 worker-i × 4 vCPU / 8 GiB pe OpenStack); reproducerea aceleiai matrici pe alte configuraii (maini locale, noduri mai puternice, eventual cu accelerare GPU pentru LSTM) este planificat ca script separat, în afara aplicaiei, dup cum este menionat în seciunea 8.4. De asemenea, evaluarea concurenei (mai multe job-uri lansate simultan pe coad) i msurarea latenelor specifice ale API-ului sunt înc tratate metodologic, fr rezultate empirice.

Componenta LSTM este implementat operaional i are rezultate cantitative (cost de antrenare msurat i serie observed/fitted/predicted pe Romania). Validarea predictiv complet — comparaia sistematic cu liniile de baz (persisten, medie mobil) i cu modelul compartimental calibrat, pe mai multe regiuni — este urmtorul pas natural; LSTM-ul actual este o component univariat hand-rolled, iar versiunile hibride (LSTM peste reziduurile unui model SIR calibrat) sunt formulate în capitolul 6 doar ca direcie.

O alt limitare este nivelul de maturitate al aplicaiei web. Fluxurile principale sunt acoperite, inclusiv studiile, rolurile i consola super-admin, îns lipsesc înc teste automate end-to-end, mecanisme de retry mai sofisticate pentru job-uri, politici de rate limiting i audit al operaiilor utilizatorilor. Aceste elemente ar fi necesare înainte de folosirea sistemului într-un context operaional.

10.6 Direcii viitoare

Direcia imediat este calibrarea modelului SIDARTHE pe date mai bogate decât seria de cazuri active, în special date despre testare, spitalizare, terapie intensiv, vindecri i decese. Aceast extindere preia infrastructura demonstrat deja pentru SIR (calibrare cross-regiune cu γ fixat din literatur) i o extinde la operatori de observaie multipli, necesari pentru distingerea grupurilor de parametri ale SIDARTHE.

O direcie tehnic imediat este promovarea calibrrii din script offline într-un tip de job asincron în mecanismul Redis al aplicaiei, alturi de `simulation_runs`. Rezultatul ar trebui s includ parametrii estimai, erorile MAE/RMSE pe train i test, numrul de evaluri ale solverului i timpul total de optimizare — toate stocate într-o tabel dedicat i expuse prin API.

Pentru componenta ML, comparaia cu liniile de baz a fost deja realizat pe seria României (Tabelul 6.2) i a artat c persistena domin la orizontul de o zi. Urmtorul pas realist este repetarea acestei comparaii pe toate cele 38 de ri europene din setul de date OWID, cu modelul compartimental calibrat ca reper suplimentar, agregând MAE, RMSE, nRMSE i timpul de antrenare per regiune. Având în vedere rezultatul de pe România, prioritatea ar trebui s fie varianta hibrid LSTM-peste-reziduuri — singurul regim în care reeaua ar putea depi un reper trivial.

La nivel de evaluare, urmtorul pas este extinderea campaniei de benchmark pe configuraii hardware diferite (maini locale, noduri mai puternice, eventual cu GPU pentru LSTM). Tot la nivel de sistem, lipsete o suit automat de teste: teste unitare i de integrare în backend cu o baz de date izolat i teste end-to-end pentru frontend care s verifice fluxurile reale din browser.

În zona de infrastructur, ar fi util introducerea unor mecanisme explicite de retry i dead-letter queue pentru job-urile care eueaz repetat, precum i activarea autoscalrii orizontale (HPA) a runner-elor pe metrici personalizate (de exemplu lungimea cozii Redis) i a politicilor de reea (NetworkPolicy) pentru izolare zero-trust intracluster. Terminarea TLS automat prin `cert-manager` i un stack de observabilitate (Prometheus + Grafana) ar

completa profilul operaional al deployment-ului Kubernetes.

Pe termen mai lung, sistemul poate fi extins cu vizualizri geografice, surse de date actualizate periodic i module de alertare. Aceste extensii trebuie construite pe aceeai disciplin metodologic care a ghidat lucrarea de fa: ipotezele modelului, calitatea datelor i metricile software trebuie s rmân vizibile, iar afirmaiile cantitative trebuie susinute prin rulri concrete i reproductibile, aa cum sunt cele din capitolele 7 i 8.

Bibliografie

[1] Roy M. Anderson and Robert M. May. *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press, Oxford, 1991.

[2] Herbert W. Hethcote. The mathematics of infectious diseases. *SIAM Review*, 42(4):599–653, 2000.

[3] W. O. Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927.

[4] Karline Soetaert, Thomas Petzoldt, and R. Woodrow Setzer. Solving differential equations in R: Package deSolve. *Journal of Statistical Software*, 33(9):1–25, 2010.

[5] Christopher Rackauckas and Qing Nie. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017.

[6] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[7] Samuel M. Jenness, Steven M. Goodreau, and Martina Morris. EpiModel: An R package for mathematical modeling of infectious disease over networks. *Journal of Statistical Software*, 84(8):1–47, 2018.

[8] Cliff C. Kerr, Robyn M. Stuart, Dina Mistry, Romesh G. Abeysuriya, Katherine Rosenfeld, Gregory R. Hart, Rafael C. Núñez, Jamie A. Cohen, Prashanth Selvaraj, Brittany Hagedorn, and Daniel J. Klein. Covasim: An agent-based model of COVID-19 dynamics and interventions. *PLOS Computational Biology*, 17(7):e1009149, 2021.

[9] Neil M. Ferguson, Daniel Laydon, Gemma Nedjati-Gilani, et al. Report 9: Impact of non-pharmaceutical interventions (NPIs) to reduce COVID-19 mortality and healthcare demand. Technical report, Imperial College London, 2020.

[10] Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017.

[11] Perry de Valpine, Daniel Turek, Christopher J. Paciorek, Clifford Anderson-Bergman, Duncan Temple Lang, and Rastislav Bodik. Programming with models: Writing statistical algorithms for general model structures with NIMBLE. *Journal of Computational and Graphical Statistics*, 26(2):403–413, 2017.

[12] Estee Y. Cramer, Evan L. Ray, Velma K. Lopez, et al. Evaluation of individual and ensemble probabilistic forecasts of COVID-19 mortality in the United States. *Proceedings of the National Academy of Sciences*, 119(15):e2113561119, 2022.

[13] Odo Diekmann, Johan A. P. Heesterbeek, and Johan A. J. Metz. On the definition and the computation of the basic reproduction ratio $\mathcal{R}_0$ in models for infectious diseases in heterogeneous populations. *Journal of Mathematical Biology*, 28(4):365–382, 1990.

[14] Giulia Giordano, Franco Blanchini, Raffaele Bruno, Patrizio Colaneri, Alessandro Di Filippo, Angela Di Matteo, and Marta Colaneri. Modelling the COVID-19 epidemic and implementation of population-wide interventions in italy. *Nature Medicine*, 26:855–860, 2020.

[15] Pauline van den Driessche and James Watmough. Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission. *Mathematical Biosciences*, 180(1–2):29–48, 2002.

[16] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2 edition, 1993.

[17] John R. Dormand and Peter J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[18] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024.

[19] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[20] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.

[21] Yugabyte, Inc. YugabyteDB architecture documentation. Documentaie tehnic, 2024.

[22] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016.

[23] The Kubernetes Authors. Kubernetes documentation. Documentaie tehnic, 2024.

[24] The Go Authors. The Go programming language specification. Specificaie oficial, 2024.

[25] Evan You et al. Vue.js documentation. Documentaie oficial, 2024.

[26] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC'14)*, pages 305–319, Philadelphia, PA, 2014. USENIX Association.

[27] Rakyll and contributors. Standard Go project layout. Depozit GitHub, 2023.

[28] Michael Jones, John Bradley, and Nat Sakimura. RFC 7519: JSON web token (JWT). Request for Comments, Internet Engineering Task Force, 2015.

[29] Evan You and contributors. Pinia — the intuitive store for Vue.js. Documentaie oficial, 2024.

[30] Edouard Mathieu, Hannah Ritchie, Lucas Rodés-Guirao, Cameron Appel, Charlie Giattino, Joe Hasell, Bobbie Macdonald, Saloni Dattani, Diana Beltekian, Esteban Ortiz-Ospina, and Max Roser. Coronavirus pandemic (COVID-19). Our World in Data, 2020.

[31] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[32] Paul J. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[33] J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308–313, 1965.

Anexa A

Cod Surs Implementarea R

Aceast anex include runner-ul R folosit de sistem pentru consumul job-urilor Redis, rezolvarea modelelor compartimentale suportate i persistarea rezultatelor în YugabyteDB.

```r
## Epidemiological model runner -- R.
##
## Consumes simulation jobs from Redis list ``epimodel:jobs:r`` (BRPOP via redux),
## solves SIS/SIR/SEIR/SIRS/SIDARTHE ODE systems with
## ``deSolve::ode`` (RK4, fixed step h = 1),
## and inserts compartment trajectories into YugabyteDB.

suppressPackageStartupMessages({
  library(deSolve)
  library(DBI)
  library(RPostgres)
  library(redux)
  library(jsonlite)
})

QUEUE <- "epimodel:jobs:r"
BRPOP_TIMEOUT_SEC <- 5L
ERROR_BACKOFF_SEC <- 2L

# Flag de oprire; R are signal handling limitat, ne bazam pe SIGTERM de la docker
    stop.
.running <- TRUE

sir_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

sis_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    infections <- beta * S * I / N
    recoveries <- gamma * I
    dS <- -infections + recoveries
    dI <-  infections - recoveries
    list(c(dS, dI))
  })
}

```

```r
seir_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    infections <- beta * S * I / N
    dS <- -infections
    dE <-  infections - sigma * E
    dI <-  sigma * E - gamma * I
    dR <-  gamma * I
    list(c(dS, dE, dI, dR))
  })
}

sirs_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    infections <- beta * S * I / N
    recoveries <- gamma * I
    waning <- xi * R
    dS <- -infections + waning
    dI <-  infections - recoveries
    dR <-  recoveries - waning
    list(c(dS, dI, dR))
  })
}

sidarthe_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    force <- (alpha * S_I + beta * D + gamma * A + delta * R) / N
    new_infections <- S * force
    dS <- -new_infections
    dI <- new_infections - (epsilon + zeta + lambda) * S_I
    dD <- epsilon * S_I - (eta + rho) * D
    dA <- zeta * S_I - (theta + mu + kappa) * A
    dR <- eta * D + theta * A - (nu + xi) * R
    dT <- mu * A + nu * R - (sigma + tau) * T
    dH <- lambda * S_I + rho * D + kappa * A + xi * R + sigma * T
    dE <- tau * T
    list(c(dS, dI, dD, dA, dR, dT, dH, dE))
  })
}

model_layout <- function(model_type) {
  if (identical(model_type, "sis")) {
    return(list(
      names = c("S", "I"),
      json_names = c("s", "i"),
      rhs = sis_rhs
    ))
  }
  if (identical(model_type, "sir")) {
    return(list(
      names = c("S", "I", "R"),
      json_names = c("s", "i", "r"),
      rhs = sir_rhs
    ))
  }
  if (identical(model_type, "seir")) {
    return(list(
      names = c("S", "E", "I", "R"),
      json_names = c("s", "e", "i", "r"),
      rhs = seir_rhs
```

```
    ))
  }
  if (identical(model_type, "sirs")) {
    return(list(
      names = c("S", "I", "R"),
      json_names = c("s", "i", "r"),
      rhs = sirs_rhs
    ))
  }
  if (identical(model_type, "sidarthe")) {
    return(list(
      names = c("S", "S_I", "D", "A", "R", "T", "H", "E"),
      json_names = c("s", "i", "d", "a", "r", "t", "h", "e"),
      rhs = sidarthe_rhs
    ))
  }
  stop(sprintf("unsupported model_type: %s", model_type))
}

peak_rss_mb <- function() {
  # /proc/self/status VmHWM is Linux-only; on macOS fall back to gc()
  status <- tryCatch(readLines("/proc/self/status", warn = FALSE), error = function(e
    ) NULL)
  if (!is.null(status)) {
    line <- grep("^VmHWM:", status, value = TRUE)
    if (length(line) == 1) {
      kb <- as.numeric(sub("[^0-9]+([0-9]+).*", "\\1", line))
      return(kb / 1024)
    }
  }
  used_mb <- sum(gc()[, 2])
  used_mb
}

parse_dsn <- function(dsn) {
  # postgres://user:pass@host:port/db?sslmode=disable
  m <- regmatches(dsn, regexec(
    "postgres(?:ql)?://([^:]+):([^@]*)@([^:/]+):([0-9]+)/([^?]+)", dsn
  ))[[1]]
  if (length(m) < 6) stop(sprintf("invalid DSN: %s", dsn))
  list(user = m[2], password = m[3], host = m[4],
       port = as.integer(m[5]), dbname = m[6])
}

db_connect <- function(dsn) {
  p <- parse_dsn(dsn)
  dbConnect(RPostgres::Postgres(),
            host = p$host, port = p$port, dbname = p$dbname,
            user = p$user, password = p$password)
}

run_simulation <- function(conn, job) {
  sim_id <- job$simulation_id
  model_type <- if (!is.null(job$model_type)) job$model_type else "sidarthe"
  layout <- model_layout(model_type)
  parameters <- as.list(job$parameters)
  parameters <- lapply(parameters, as.numeric)
  N <- as.numeric(job$population_n)
  t_max <- as.integer(job$t_max)
  initials <- as.list(job$initial)
  y0 <- setNames(
    vapply(layout$json_names, function(name) {
```

```
      value <- initials[[name]]
      if (is.null(value)) 0 else as.numeric(value)
    }, numeric(1)),
    layout$names
  )

  dbExecute(conn, "UPDATE simulation_runs SET status = 'running' WHERE id = $1",
            params = list(sim_id))

  rss_before <- peak_rss_mb()
  wall_start <- proc.time()["elapsed"]
  cpu_start  <- proc.time()["user.self"] + proc.time()["sys.self"]

  times <- seq(0, t_max, by = 1)
  parms <- c(unlist(parameters), N = N)
  out <- ode(y = y0, times = times, func = layout$rhs, parms = parms, method = "rk4")

  wall_ms <- as.integer((proc.time()["elapsed"] - wall_start) * 1000)
  cpu_ms  <- as.integer(((proc.time()["user.self"] + proc.time()["sys.self"]) - cpu_
    start) * 1000)
  peak_mb <- max(peak_rss_mb(), rss_before)

  times_out <- as.integer(out[, "time"])

  dbWithTransaction(conn, {
    for (idx in seq_along(times_out)) {
      values <- as.list(pmax(0, as.numeric(out[idx, layout$names])))
      names(values) <- layout$json_names
      dbExecute(conn,
        "INSERT INTO simulation_results (simulation_id, t, compartments) VALUES ($1,
    $2, $3::jsonb)",
        params = list(sim_id, times_out[idx], toJSON(values, auto_unbox = TRUE)))
    }
    dbExecute(conn,
      "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms, peak_memory_
    mb) VALUES ($1, $2, $3, $4)",
      params = list(sim_id, wall_ms, cpu_ms, peak_mb))
    dbExecute(conn,
      "UPDATE simulation_runs SET status = 'completed', completed_at = NOW() WHERE id
     = $1",
      params = list(sim_id))
  })

  message(sprintf("sim %s done: wall=%dms cpu=%dms mem=%.1fMB rows=%d",
                  sim_id, wall_ms, cpu_ms, peak_mb, length(times_out)))
}

mark_failed <- function(conn, sim_id, msg) {
  tryCatch(
    dbExecute(conn,
      "UPDATE simulation_runs SET status = 'failed', error_message = $1, completed_at
     = NOW() WHERE id = $2",
      params = list(substr(msg, 1, 2000), sim_id)),
    error = function(e) message(sprintf("could not mark failed: %s", e$message))
  )
}

main <- function() {
  redis_url <- Sys.getenv("REDIS_URL", "redis://redis:6379/0")
  db_dsn_env <- Sys.getenv("DATABASE_URL", "")

  # redux uses host/port/db number; parse URL
```

```
  m <- regmatches(redis_url, regexec(
    "redis://(?:[^@]*@)?([^:/]+):([0-9]+)(?:/([0-9]+))?", redis_url
  ))[[1]]
  if (length(m) < 4) stop(sprintf("invalid REDIS_URL: %s", redis_url))
  rdb <- redux::hiredis(host = m[2], port = as.integer(m[3]),
                        db = if (m[4] == "") 0L else as.integer(m[4]))
  rdb$PING()
  message(sprintf("runner-r connected to redis, consuming %s", QUEUE))

  while (.running) {
    item <- tryCatch(rdb$BRPOP(QUEUE, BRPOP_TIMEOUT_SEC),
                     error = function(e) {
                       message(sprintf("redis error: %s", e$message))
                       Sys.sleep(ERROR_BACKOFF_SEC)
                       NULL
                     })
    if (is.null(item)) next

    raw <- item[[2]]
    job <- tryCatch(fromJSON(raw), error = function(e) {
      message(sprintf("invalid job json: %s", e$message))
      NULL
    })
    if (is.null(job)) next

    sim_id <- if (!is.null(job$simulation_id)) job$simulation_id else "?"
    dsn <- if (!is.null(job$db_dsn) && nzchar(job$db_dsn)) job$db_dsn else db_dsn_env
    if (!nzchar(dsn)) {
      message(sprintf("sim %s: no db_dsn", sim_id))
      next
    }

    conn <- tryCatch(db_connect(dsn), error = function(e) {
      message(sprintf("sim %s db connect failed: %s", sim_id, e$message))
      NULL
    })
    if (is.null(conn)) next

    tryCatch({
      run_simulation(conn, job)
    }, error = function(e) {
      message(sprintf("sim %s failed: %s", sim_id, e$message))
      mark_failed(conn, sim_id, e$message)
    }, finally = {
      dbDisconnect(conn)
    })
  }
  message("runner-r stopped")
}

main()
```

Listing A.1: Runner-ul R pentru modelele SIDARTHE/SIR/SIS/SEIR/SIRS.

Anexa B

Cod Surs Implementarea Julia

Aceast anex include runner-ul Julia, bazat pe `DifferentialEquations.jl`, `Redis.jl` i `LibPQ.jl`.

```
#=
Epidemiological model runner -- Julia.

Consumes simulation jobs from Redis list ‘epimodel:jobs:julia‘ (BRPOP),
solves SIS/SIR/SEIR/SIRS/SIDARTHE ODE systems with
DifferentialEquations.jl (Tsit5, saveat=1),
and inserts compartment trajectories into YugabyteDB via LibPQ.
=#

using DifferentialEquations
using JSON3
using LibPQ
using Redis

const QUEUE = "epimodel:jobs:julia"
const BRPOP_TIMEOUT_SEC = 5
const ERROR_BACKOFF_SEC = 2

mutable struct RunCtx
    running::Bool
end
const CTX = RunCtx(true)

function sir_rhs!(du, u, p, t)
    _ = t
    S, I, R = u
    beta = p["beta"]
    gamma = p["gamma"]
    N = p["N"]
    du[1] = -beta * S * I / N
    du[2] =  beta * S * I / N - gamma * I
    du[3] =  gamma * I
end

function sis_rhs!(du, u, p, t)
    _ = t
    S, I = u
    beta = p["beta"]
    gamma = p["gamma"]
    N = p["N"]
    infections = beta * S * I / N
    recoveries = gamma * I
    du[1] = -infections + recoveries
    du[2] =  infections - recoveries
end
```

```
46
47  function seir_rhs!(du, u, p, t)
48      _ = t
49      S, E, I, R = u
50      beta = p["beta"]
51      sigma = p["sigma"]
52      gamma = p["gamma"]
53      N = p["N"]
54      infections = beta * S * I / N
55      du[1] = -infections
56      du[2] =  infections - sigma * E
57      du[3] =  sigma * E - gamma * I
58      du[4] =  gamma * I
59  end
60
61  function sirs_rhs!(du, u, p, t)
62      _ = t
63      S, I, R = u
64      beta = p["beta"]
65      gamma = p["gamma"]
66      xi = p["xi"]
67      N = p["N"]
68      infections = beta * S * I / N
69      recoveries = gamma * I
70      waning = xi * R
71      du[1] = -infections + waning
72      du[2] =  infections - recoveries
73      du[3] =  recoveries - waning
74  end
75
76  function sidarthe_rhs!(du, u, p, t)
77      _ = t
78      S, I, D, A, R, T, H, E = u
79      _ = E
80      N = p["N"]
81      force = (p["alpha"] * I + p["beta"] * D + p["gamma"] * A + p["delta"] * R) / N
82      new_infections = S * force
83
84      du[1] = -new_infections
85      du[2] = new_infections - (p["epsilon"] + p["zeta"] + p["lambda"]) * I
86      du[3] = p["epsilon"] * I - (p["eta"] + p["rho"]) * D
87      du[4] = p["zeta"] * I - (p["theta"] + p["mu"] + p["kappa"]) * A
88      du[5] = p["eta"] * D + p["theta"] * A - (p["nu"] + p["xi"]) * R
89      du[6] = p["mu"] * A + p["nu"] * R - (p["sigma"] + p["tau"]) * T
90      du[7] = p["lambda"] * I + p["rho"] * D + p["kappa"] * A + p["xi"] * R + p["sigma"
        ] * T
91      du[8] = p["tau"] * T
92      nothing
93  end
94
95  function model_layout(model_type::AbstractString)
96      if model_type == "sis"
97          return (state_names = ["s", "i"], rhs = sis_rhs!)
98      elseif model_type == "sir"
99          return (state_names = ["s", "i", "r"], rhs = sir_rhs!)
100     elseif model_type == "seir"
101         return (state_names = ["s", "e", "i", "r"], rhs = seir_rhs!)
102     elseif model_type == "sirs"
103         return (state_names = ["s", "i", "r"], rhs = sirs_rhs!)
104     elseif model_type == "sidarthe"
105         return (state_names = ["s", "i", "d", "a", "r", "t", "h", "e"], rhs =
        sidarthe_rhs!)
```

```
    end
    error("unsupported model_type: $model_type")
end

peak_rss_mb() = Sys.maxrss() / (1024 * 1024)

function parse_redis_url(url::AbstractString)
    m = match(r"^redis://(?:[^@]*@)?(?<host>[^:/]+):(?<port>\d+)(?:/(?<db>\d+))?$", url)
    m === nothing && error("invalid REDIS_URL: $url")
    (host = m[:host],
     port = parse(Int, m[:port]),
     db   = m[:db] === nothing ? 0 : parse(Int, m[:db]))
end

function run_simulation(conn::LibPQ.Connection, job::Dict{Symbol, Any})
    sim_id = String(job[:simulation_id])
    model_type = String(get(job, :model_type, "sidarthe"))
    layout = model_layout(model_type)
    params = Dict{String, Float64}(String(k) => Float64(v) for (k, v) in pairs(job[:parameters]))
    N = Float64(job[:population_n])
    t_max = Int(job[:t_max])
    params["N"] = N
    initial = Dict{String, Float64}(String(k) => Float64(v) for (k, v) in pairs(job[:initial]))
    y0 = Float64[get(initial, name, 0.0) for name in layout.state_names]

    LibPQ.execute(conn,
        "UPDATE simulation_runs SET status = 'running' WHERE id = \$1",
        [sim_id])

    rss_before = peak_rss_mb()
    wall_start = time_ns()
    cpu_start  = time()  # process CPU not directly exposed; use elapsed as fallback

    prob = ODEProblem(layout.rhs, y0, (0.0, Float64(t_max)), params)
    sol = solve(prob, Tsit5(); saveat = 0:1.0:t_max, reltol = 1e-8, abstol = 1e-10)

    wall_ms = round(Int, (time_ns() - wall_start) / 1_000_000)
    cpu_ms  = round(Int, (time() - cpu_start) * 1000)
    peak_mb = max(peak_rss_mb(), rss_before)

    LibPQ.execute(conn, "BEGIN")
    try
        stmt = LibPQ.prepare(conn,
            "INSERT INTO simulation_results (simulation_id, t, compartments) VALUES (\$1, \$2, \$3::jsonb)")
        for (idx, t) in enumerate(sol.t)
            u = sol.u[idx]
            values = Dict(name => max(0.0, Float64(u[pos])) for (pos, name) in enumerate(layout.state_names))
            LibPQ.execute(stmt, (sim_id, Int(t), JSON3.write(values)))
        end
        LibPQ.execute(conn,
            "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES (\$1, \$2, \$3, \$4)",
            [sim_id, wall_ms, cpu_ms, peak_mb])
        LibPQ.execute(conn,
            "UPDATE simulation_runs SET status = 'completed', completed_at = NOW() WHERE id = \$1",
            [sim_id])
```

```
        LibPQ.execute(conn, "COMMIT")
    catch e
        LibPQ.execute(conn, "ROLLBACK")
        rethrow(e)
    end

    println("sim $sim_id done: wall=$(wall_ms)ms cpu=$(cpu_ms)ms mem=$(round(peak_mb, digits=1))MB rows=$(length(sol.t))")
end

function mark_failed(conn::LibPQ.Connection, sim_id::AbstractString, msg::AbstractString)
    try
        LibPQ.execute(conn,
            "UPDATE simulation_runs SET status = 'failed', error_message = \$1, completed_at = NOW() WHERE id = \$2",
            [first(msg, 2000), sim_id])
    catch e
        @warn "could not mark failed" exception=e
    end
end

function main()
    redis_url = get(ENV, "REDIS_URL", "redis://redis:6379/0")
    db_dsn_env = get(ENV, "DATABASE_URL", "")
    rinfo = parse_redis_url(redis_url)
    rcli = RedisConnection(host = rinfo.host, port = rinfo.port, db = rinfo.db)
    Redis.ping(rcli)
    println("runner-julia connected to redis, consuming $QUEUE")

    while CTX.running
        item = try
            Redis.brpop(rcli, [QUEUE], BRPOP_TIMEOUT_SEC)
        catch e
            @warn "redis error" exception=e
            sleep(ERROR_BACKOFF_SEC)
            nothing
        end
        item === nothing && continue
        # Redis.brpop returns Vector{String} [queue_name, payload] or nil
        length(item) < 2 && continue
        raw = item[2]

        job = try
            JSON3.read(raw, Dict{Symbol, Any})
        catch e
            @warn "invalid job json" exception=e
            continue
        end

        sim_id = get(job, :simulation_id, "?")
        dsn = get(job, :db_dsn, "")
        if isempty(dsn)
            dsn = db_dsn_env
        end
        if isempty(dsn)
            @warn "no db_dsn for sim" sim_id
            continue
        end

        conn = try
            LibPQ.Connection(dsn)
```

```
        catch e
            @warn "db connect failed" sim_id exception=e
            continue
        end
        try
            run_simulation(conn, job)
        catch e
            @warn "sim failed" sim_id exception=e
            mark_failed(conn, String(sim_id), sprint(showerror, e))
        finally
            close(conn)
        end
    end
    println("runner-julia stopped")
end

main()
```

Listing B.1: Runner-ul Julia pentru modelele SIDARTHE/SIR/SIS/SEIR/SIRS.

Anexa C

Cod Surs Implementarea Python

Aceast anex include runner-ul Python, care folosete `scipy.integrate.solve_ivp` pentru integrarea numeric i `psycopg` pentru scrierea rezultatelor.

```
"""Epidemiological model runner  Python.

Consumes simulation jobs from Redis list ``epimodel:jobs:python`` (BRPOP),
solves SIS/SIR/SEIR/SIRS/SIDARTHE ODE systems with
``scipy.integrate.solve_ivp`` (RK45),
inserts compartment trajectories into YugabyteDB and records the benchmark
(wall time, CPU time, peak RSS) into ``benchmarks``.
"""

from __future__ import annotations

import json
import os
import resource
import signal
import sys
import time
from typing import Any

import numpy as np
import psycopg
import redis
from psycopg.rows import dict_row
from psycopg.types.json import Json
from scipy.integrate import solve_ivp

QUEUE = "epimodel:jobs:python"
BRPOP_TIMEOUT_SEC = 5
ERROR_BACKOFF_SEC = 2

_running = True


def _handle_signal(signum, _frame):
    global _running
    print(f"signal {signum} received, shutting down", file=sys.stderr, flush=True)
    _running = False


def sir_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[
    float]:
    del t
    s, i, r = y
    beta = params["beta"]
```

```
    gamma = params["gamma"]
    return [
        -beta * s * i / n,
        beta * s * i / n - gamma * i,
        gamma * i,
    ]


def sis_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, i = y
    beta = params["beta"]
    gamma = params["gamma"]
    infections = beta * s * i / n
    recoveries = gamma * i
    return [-infections + recoveries, infections - recoveries]


def seir_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, e, i, r = y
    beta = params["beta"]
    sigma = params["sigma"]
    gamma = params["gamma"]
    infections = beta * s * i / n
    return [
        -infections,
        infections - sigma * e,
        sigma * e - gamma * i,
        gamma * i,
    ]


def sirs_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, i, r = y
    beta = params["beta"]
    gamma = params["gamma"]
    xi = params["xi"]
    infections = beta * s * i / n
    recoveries = gamma * i
    waning = xi * r
    return [-infections + waning, infections - recoveries, recoveries - waning]


def sidarthe_rhs(t: float, y: np.ndarray, params: dict[str, float], n: float) -> list[float]:
    del t
    s, i, d, a, r, threatened, h, extinct = y
    del extinct
    force = (params["alpha"] * i + params["beta"] * d + params["gamma"] * a + params["delta"] * r) / n
    new_infections = s * force

    return [
        -new_infections,
        new_infections - (params["epsilon"] + params["zeta"] + params["lambda"]) * i,
        params["epsilon"] * i - (params["eta"] + params["rho"]) * d,
        params["zeta"] * i - (params["theta"] + params["mu"] + params["kappa"]) * a,
```

```
        params["eta"] * d + params["theta"] * a - (params["nu"] + params["xi"]) * r,
        params["mu"] * a + params["nu"] * r - (params["sigma"] + params["tau"]) *
    threatened,
        params["lambda"] * i + params["rho"] * d + params["kappa"] * a + params["xi"]
     * r + params["sigma"] * threatened,
        params["tau"] * threatened,
    ]


def model_layout(model_type: str) -> tuple[list[str], Any]:
    if model_type == "sis":
        return ["s", "i"], sis_rhs
    if model_type == "sir":
        return ["s", "i", "r"], sir_rhs
    if model_type == "seir":
        return ["s", "e", "i", "r"], seir_rhs
    if model_type == "sirs":
        return ["s", "i", "r"], sirs_rhs
    if model_type == "sidarthe":
        return ["s", "i", "d", "a", "r", "t", "h", "e"], sidarthe_rhs
    raise ValueError(f"unsupported model_type: {model_type}")


def _peak_rss_mb() -> float:
    """Return peak RSS in MB across platforms.

    Linux returns ``ru_maxrss`` in KiB, macOS in bytes.
    """
    rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    if sys.platform == "darwin":
        return rss / (1024 * 1024)
    return rss / 1024


def run_simulation(conn: psycopg.Connection, job: dict[str, Any]) -> None:
    sim_id = job["simulation_id"]
    model_type = job.get("model_type", "sidarthe")
    params = {str(k): float(v) for k, v in job["parameters"].items()}
    initial = {str(k): float(v) for k, v in job["initial"].items()}
    n = float(job["population_n"])
    t_max = int(job["t_max"])
    compartments, rhs = model_layout(model_type)
    y0 = [initial.get(name, 0.0) for name in compartments]

    with conn.transaction():
        conn.execute(
            "UPDATE simulation_runs SET status = 'running' WHERE id = %s",
            (sim_id,),
        )

    rss_before = _peak_rss_mb()
    wall_start = time.perf_counter()
    cpu_start = time.process_time()

    sol = solve_ivp(
        fun=rhs,
        t_span=(0.0, float(t_max)),
        y0=y0,
        method="RK45",
        t_eval=np.arange(0, t_max + 1, dtype=float),
        args=(params, n),
        rtol=1e-8,
```

```
        atol=1e-10,
    )
    if not sol.success:
        raise RuntimeError(f"solve_ivp failed: {sol.message}")

    wall_ms = int((time.perf_counter() - wall_start) * 1000)
    cpu_ms = int((time.process_time() - cpu_start) * 1000)
    peak_mb = max(_peak_rss_mb(), rss_before)

    rows = []
    for idx, t in enumerate(sol.t):
        values = {
            name: max(0.0, float(sol.y[pos][idx]))
            for pos, name in enumerate(compartments)
        }
        rows.append((sim_id, int(t), Json(values)))

    with conn.transaction():
        with conn.cursor() as cur:
            cur.executemany(
                "INSERT INTO simulation_results (simulation_id, t, compartments)
    VALUES (%s, %s, %s)",
                rows,
            )
            cur.execute(
                "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms,
    peak_memory_mb) VALUES (%s, %s, %s, %s)",
                (sim_id, wall_ms, cpu_ms, peak_mb),
            )
            cur.execute(
                "UPDATE simulation_runs SET status = 'completed', completed_at = NOW
    () WHERE id = %s",
                (sim_id,),
            )
    print(
        f"sim {sim_id} done: wall={wall_ms}ms cpu={cpu_ms}ms mem={peak_mb:.1f}MB rows
    ={len(rows)}",
        flush=True,
    )


def mark_failed(conn: psycopg.Connection, sim_id: str, msg: str) -> None:
    try:
        with conn.transaction():
            conn.execute(
                "UPDATE simulation_runs SET status = 'failed', error_message = %s,
    completed_at = NOW() WHERE id = %s",
                (msg[:2000], sim_id),
            )
    except Exception as exc:  # noqa: BLE001
        print(f"could not mark failed: {exc}", file=sys.stderr, flush=True)


def main() -> int:
    signal.signal(signal.SIGINT, _handle_signal)
    signal.signal(signal.SIGTERM, _handle_signal)

    redis_url = os.environ["REDIS_URL"]
    db_dsn = os.environ.get("DATABASE_URL")

    rdb = redis.from_url(redis_url)
    rdb.ping()
```

```
    print(f"runner-python connected to redis, consuming {QUEUE}", flush=True)

    while _running:
        try:
            item = rdb.brpop(QUEUE, timeout=BRPOP_TIMEOUT_SEC)
        except redis.RedisError as exc:
            print(f"redis error: {exc}", file=sys.stderr, flush=True)
            time.sleep(ERROR_BACKOFF_SEC)
            continue
        if item is None:
            continue

        _, raw = item
        try:
            job = json.loads(raw)
        except json.JSONDecodeError as exc:
            print(f"invalid job json: {exc}", file=sys.stderr, flush=True)
            continue

        sim_id = job.get("simulation_id", "?")
        dsn = job.get("db_dsn") or db_dsn
        if not dsn:
            print(f"sim {sim_id}: no db_dsn", file=sys.stderr, flush=True)
            continue

        try:
            with psycopg.connect(dsn, row_factory=dict_row) as conn:
                run_simulation(conn, job)
        except Exception as exc:  # noqa: BLE001
            print(f"sim {sim_id} failed: {exc}", file=sys.stderr, flush=True)
            try:
                with psycopg.connect(dsn) as conn:
                    mark_failed(conn, sim_id, str(exc))
            except Exception as inner:  # noqa: BLE001
                print(f"could not connect to mark failed: {inner}", file=sys.stderr,
    flush=True)

    print("runner-python stopped", flush=True)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Listing C.1: Runner-ul Python pentru modelele SIDARTHE/SIR/SIS/SEIR/SIRS.

Anexa D

Cod Surs — Runner LSTM

Aceast anex include runner-ul LSTM separat, responsabil pentru consumul cozii `epimodel:jobs:lstm`, antrenarea modelului recurent pe seria `infected` i persistarea seriilor `observed`, `fitted` i `predicted`.

```
"""Dedicated LSTM runner.

Consumes prediction jobs from Redis list ``epimodel:jobs:lstm``, trains a
small univariate LSTM over the dataset's infected time series, writes observed,
fitted and forecast values into ``simulation_results``, and records execution
metrics in ``benchmarks``.
"""

from __future__ import annotations

import json
import os
import resource
import signal
import sys
import time
from dataclasses import dataclass
from typing import Any

import numpy as np
import psycopg
import redis
from psycopg.rows import dict_row
from psycopg.types.json import Json

QUEUE = "epimodel:jobs:lstm"
BRPOP_TIMEOUT_SEC = 5
ERROR_BACKOFF_SEC = 2
TEST_FRACTION = 0.2  # ultima parte a seriei, tinuta deoparte pentru evaluarea
    generalizarii

_running = True


def _handle_signal(signum, _frame):
    global _running
    print(f"signal {signum} received, shutting down", file=sys.stderr, flush=True)
    _running = False


def _peak_rss_mb() -> float:
    rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    if sys.platform == "darwin":
        return rss / (1024 * 1024)
```

```
    return rss / 1024


def _sigmoid(x: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-np.clip(x, -50.0, 50.0)))


@dataclass
class SampleSet:
    x: np.ndarray
    y: np.ndarray


@dataclass
class TrainOutcome:
    fitted: list[float | None]  # predictii o-zi-inainte, lungime egala cu seria
    forecast: list[float]       # prognoza autoregresiva dincolo de ultima zi observata
    train_end: int              # primul indice al ferestrei de test
    mae: float
    rmse: float
    nrmse: float


class LSTMRegressor:
    def __init__(self, hidden_size: int, learning_rate: float, seed: int = 42) -> None:
        self.hidden_size = hidden_size
        self.learning_rate = learning_rate
        rng = np.random.default_rng(seed)
        width = hidden_size + 1
        scale = 0.1

        self.wf = rng.normal(0.0, scale, (hidden_size, width))
        self.wi = rng.normal(0.0, scale, (hidden_size, width))
        self.wo = rng.normal(0.0, scale, (hidden_size, width))
        self.wg = rng.normal(0.0, scale, (hidden_size, width))
        self.bf = np.full((hidden_size, 1), 0.5)
        self.bi = np.zeros((hidden_size, 1))
        self.bo = np.zeros((hidden_size, 1))
        self.bg = np.zeros((hidden_size, 1))
        self.wy = rng.normal(0.0, scale, (1, hidden_size))
        self.by = np.zeros((1, 1))

    def forward(self, seq: np.ndarray) -> tuple[float, list[dict[str, np.ndarray]]]:
        h = np.zeros((self.hidden_size, 1))
        c = np.zeros((self.hidden_size, 1))
        cache: list[dict[str, np.ndarray]] = []

        for value in seq:
            x = np.array([[float(value)]])
            h_prev = h
            c_prev = c
            z = np.vstack([h_prev, x])

            f = _sigmoid(self.wf @ z + self.bf)
            i = _sigmoid(self.wi @ z + self.bi)
            o = _sigmoid(self.wo @ z + self.bo)
            g = np.tanh(self.wg @ z + self.bg)

            c = f * c_prev + i * g
            h = o * np.tanh(c)
```

```
            cache.append({
                "z": z,
                "f": f,
                "i": i,
                "o": o,
                "g": g,
                "c": c,
                "c_prev": c_prev,
                "h": h,
            })

        y_hat = float((self.wy @ h + self.by)[0, 0])
        return y_hat, cache

    def predict(self, seq: np.ndarray) -> float:
        y_hat, _ = self.forward(seq)
        return y_hat

    def train_one(self, seq: np.ndarray, target: float) -> float:
        y_hat, cache = self.forward(seq)
        error = y_hat - float(target)

        grads = {
            "wf": np.zeros_like(self.wf),
            "wi": np.zeros_like(self.wi),
            "wo": np.zeros_like(self.wo),
            "wg": np.zeros_like(self.wg),
            "bf": np.zeros_like(self.bf),
            "bi": np.zeros_like(self.bi),
            "bo": np.zeros_like(self.bo),
            "bg": np.zeros_like(self.bg),
            "wy": np.zeros_like(self.wy),
            "by": np.zeros_like(self.by),
        }

        dy = 2.0 * error
        grads["wy"] += dy * cache[-1]["h"].T
        grads["by"] += dy

        dh_next = self.wy.T * dy
        dc_next = np.zeros((self.hidden_size, 1))

        for step in reversed(cache):
            z = step["z"]
            f = step["f"]
            i = step["i"]
            o = step["o"]
            g = step["g"]
            c = step["c"]
            c_prev = step["c_prev"]

            tanh_c = np.tanh(c)
            do = dh_next * tanh_c
            dao = do * o * (1.0 - o)

            dc = dh_next * o * (1.0 - tanh_c * tanh_c) + dc_next
            df = dc * c_prev
            daf = df * f * (1.0 - f)
            di = dc * g
            dai = di * i * (1.0 - i)
            dg = dc * i
            dag = dg * (1.0 - g * g)
```

```
            grads["wf"] += daf @ z.T
            grads["wi"] += dai @ z.T
            grads["wo"] += dao @ z.T
            grads["wg"] += dag @ z.T
            grads["bf"] += daf
            grads["bi"] += dai
            grads["bo"] += dao
            grads["bg"] += dag

            dz = self.wf.T @ daf + self.wi.T @ dai + self.wo.T @ dao + self.wg.T @ dag
            dh_next = dz[: self.hidden_size]
            dc_next = dc * f

        for key, grad in grads.items():
            np.clip(grad, -5.0, 5.0, out=grad)
            setattr(self, key, getattr(self, key) - self.learning_rate * grad)

        return float(error * error)


def fetch_infected_series(conn: psycopg.Connection, dataset_id: str) -> np.ndarray:
    rows = conn.execute(
        """
        SELECT infected
        FROM epidemiological_records
        WHERE dataset_id = %s
        ORDER BY recorded_date ASC, id ASC
        """,
        (dataset_id,),
    ).fetchall()
    if not rows:
        raise RuntimeError("dataset has no epidemiological records")
    return np.array([float(row["infected"]) for row in rows], dtype=float)


def make_samples(values: np.ndarray, lookback: int) -> SampleSet:
    x_rows = []
    y_rows = []
    for idx in range(lookback, len(values)):
        x_rows.append(values[idx - lookback:idx])
        y_rows.append(values[idx])
    if not x_rows:
        raise RuntimeError("not enough data points for LSTM lookback")
    return SampleSet(x=np.array(x_rows, dtype=float), y=np.array(y_rows, dtype=float))


def fit_scaler(values: np.ndarray) -> tuple[float, float]:
    """Media si abaterea standard ale normalizarii, calibrate doar pe antrenare."""
    mean = float(np.mean(values))
    std = float(np.std(values))
    if std < 1e-9:
        std = 1.0
    return mean, std


def denormalize(value: float, mean: float, std: float) -> float:
    return max(0.0, float(value * std + mean))
```

```
def split_index(n: int, lookback: int) -> int:
    """Primul indice al ferestrei de test, dupa o impartire cronologica.

    Fereastra de test este ultima parte a seriei (``TEST_FRACTION``), dar
    portiunea de antrenare trebuie sa pastreze cel putin un esantion complet
    de lungime ``lookback``.
    """
    test_size = min(round(TEST_FRACTION * n), n - lookback - 1)
    if test_size < 1:
        raise RuntimeError("dataset prea scurt pentru o impartire train/test")
    return n - test_size


def evaluate(
    series: np.ndarray, fitted: list[float | None], train_end: int, population: float
) -> tuple[float, float, float]:
    """MAE, RMSE si nRMSE pe fereastra de test (definitii din capitolul 7)."""
    errors: list[float] = []
    for idx in range(train_end, len(series)):
        pred = fitted[idx]
        if pred is not None:
            errors.append(pred - float(series[idx]))
    if not errors:
        return 0.0, 0.0, 0.0
    arr = np.array(errors, dtype=float)
    mae = float(np.mean(np.abs(arr)))
    rmse = float(np.sqrt(np.mean(arr**2)))
    nrmse = rmse / population if population > 0 else 0.0
    return mae, rmse, nrmse


def train_model(series: np.ndarray, params: dict[str, float], population: float) ->
    TrainOutcome:
    lookback = int(params["lookback"])
    hidden_size = int(params["hidden_size"])
    epochs = int(params["epochs"])
    learning_rate = float(params["learning_rate"])
    horizon = int(params["forecast_horizon"])

    n = len(series)
    train_end = split_index(n, lookback)

    # Normalizarea se calibreaza exclusiv pe portiunea de antrenare, pentru a nu
    # permite scurgeri de informatie din fereastra de test.
    mean, std = fit_scaler(series[:train_end])
    normalized = (series - mean) / std

    samples = make_samples(normalized[:train_end], lookback)
    model = LSTMRegressor(hidden_size=hidden_size, learning_rate=learning_rate)
    rng = np.random.default_rng(123)
    for _ in range(epochs):
        for idx in rng.permutation(len(samples.x)):
            model.train_one(samples.x[idx], float(samples.y[idx]))

    # Predictii o-zi-inainte pe toata seria: in portiunea de antrenare sunt
    # in-sample, iar pe fereastra de test sunt out-of-sample, deoarece modelul
    # nu a vazut acele tinte.
    fitted: list[float | None] = [None] * n
    for idx in range(lookback, n):
        pred_norm = model.predict(normalized[idx - lookback:idx])
        fitted[idx] = denormalize(pred_norm, mean, std)

```

```
    mae, rmse, nrmse = evaluate(series, fitted, train_end, population)

    # Prognoza autoregresiva dincolo de ultima zi observata.
    window = list(normalized[-lookback:])
    forecast: list[float] = []
    for _ in range(horizon):
        pred_norm = model.predict(np.array(window, dtype=float))
        forecast.append(denormalize(pred_norm, mean, std))
        window = [*window[1:], pred_norm]

    return TrainOutcome(
        fitted=fitted, forecast=forecast, train_end=train_end,
        mae=mae, rmse=rmse, nrmse=nrmse,
    )


def run_lstm(conn: psycopg.Connection, job: dict[str, Any]) -> None:
    sim_id = job["simulation_id"]
    dataset_id = job["dataset_id"]
    params = {str(k): float(v) for k, v in job["parameters"].items()}
    params["forecast_horizon"] = float(int(job["t_max"]))
    population = float(job["population_n"])

    with conn.transaction():
        conn.execute(
            "UPDATE simulation_runs SET status = 'running' WHERE id = %s",
            (sim_id,),
        )

    rss_before = _peak_rss_mb()
    wall_start = time.perf_counter()
    cpu_start = time.process_time()

    series = fetch_infected_series(conn, dataset_id)
    if len(series) <= int(params["lookback"]) + 1:
        raise RuntimeError("dataset length must exceed lookback + 1 for a train/test split")
    outcome = train_model(series, params, population)

    wall_ms = int((time.perf_counter() - wall_start) * 1000)
    cpu_ms = int((time.process_time() - cpu_start) * 1000)
    peak_mb = max(_peak_rss_mb(), rss_before)

    rows = []
    for idx, observed in enumerate(series):
        values = {"observed": max(0.0, float(observed))}
        fit = outcome.fitted[idx]
        if fit is not None:
            values["fitted"] = float(fit)
        rows.append((sim_id, idx, Json(values)))

    for step, predicted in enumerate(outcome.forecast):
        rows.append((sim_id, len(series) + step, Json({"predicted": float(predicted)})))

    with conn.transaction():
        with conn.cursor() as cur:
            cur.executemany(
                "INSERT INTO simulation_results (simulation_id, t, compartments) VALUES (%s, %s, %s)",
                rows,
            )
```

```
            cur.execute(
                "INSERT INTO benchmarks (simulation_id, wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES (%s, %s, %s, %s)",
                (sim_id, wall_ms, cpu_ms, peak_mb),
            )
            cur.execute(
                "UPDATE simulation_runs SET status = 'completed', completed_at = NOW() WHERE id = %s",
                (sim_id,),
            )

    print(
        f"lstm sim {sim_id} done: wall={wall_ms}ms cpu={cpu_ms}ms mem={peak_mb:.1f}MB "
        f"rows={len(rows)} train_end={outcome.train_end}/{len(series)} "
        f"MAE={outcome.mae:.2f} RMSE={outcome.rmse:.2f} nRMSE={outcome.nrmse:.3e}",
        flush=True,
    )


def mark_failed(conn: psycopg.Connection, sim_id: str, msg: str) -> None:
    try:
        with conn.transaction():
            conn.execute(
                "UPDATE simulation_runs SET status = 'failed', error_message = %s, completed_at = NOW() WHERE id = %s",
                (msg[:2000], sim_id),
            )
    except Exception as exc:  # noqa: BLE001
        print(f"could not mark failed: {exc}", file=sys.stderr, flush=True)


def main() -> int:
    signal.signal(signal.SIGINT, _handle_signal)
    signal.signal(signal.SIGTERM, _handle_signal)

    redis_url = os.environ["REDIS_URL"]
    db_dsn = os.environ.get("DATABASE_URL")

    rdb = redis.from_url(redis_url)
    rdb.ping()
    print(f"runner-lstm connected to redis, consuming {QUEUE}", flush=True)

    while _running:
        try:
            item = rdb.brpop(QUEUE, timeout=BRPOP_TIMEOUT_SEC)
        except redis.RedisError as exc:
            print(f"redis error: {exc}", file=sys.stderr, flush=True)
            time.sleep(ERROR_BACKOFF_SEC)
            continue
        if item is None:
            continue

        _, raw = item
        try:
            job = json.loads(raw)
        except json.JSONDecodeError as exc:
            print(f"invalid job json: {exc}", file=sys.stderr, flush=True)
            continue

        sim_id = job.get("simulation_id", "?")
        dsn = job.get("db_dsn") or db_dsn
```

```
        if not dsn:
            print(f"sim {sim_id}: no db_dsn", file=sys.stderr, flush=True)
            continue

        try:
            if job.get("model_type") != "lstm":
                raise RuntimeError(f"runner-lstm received unsupported model_type: {job.get('model_type')}")
            with psycopg.connect(dsn, row_factory=dict_row) as conn:
                run_lstm(conn, job)
        except Exception as exc:  # noqa: BLE001
            print(f"sim {sim_id} failed: {exc}", file=sys.stderr, flush=True)
            try:
                with psycopg.connect(dsn) as conn:
                    mark_failed(conn, sim_id, str(exc))
            except Exception as inner:  # noqa: BLE001
                print(f"could not connect to mark failed: {inner}", file=sys.stderr, flush=True)

    print("runner-lstm stopped", flush=True)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Listing D.1: Runner-ul LSTM dedicat prediciilor epidemiologice.

Anexa E

Schema Bazei de Date

Anexa include migraiile SQL care definesc schema relaional folosit de API i runner-e. Primele migraii creeaz structura iniial; cele intermediare adaug studiile, membership-ul, flag-ul de super-admin, legtura `study_id` pentru resursele epidemiologice i extinderile pentru modelele `lstm` i `script`; ultimele introduc tabelele auxiliare (runner-e plug-in, draft-uri de script) i îmbogirea schemei de date epidemiologice cu metrici suplimentare (decese, spitalizri, ATI, vaccinri, teste, indici de rspuns sanitar).

```sql
CREATE TABLE IF NOT EXISTS users (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email         VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
```

Listing E.1: Crearea tabelului `users`.

```sql
CREATE TABLE IF NOT EXISTS datasets (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name          VARCHAR(255) NOT NULL,
    disease_code  VARCHAR(64) NOT NULL,
    uploaded_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    record_count  INTEGER NOT NULL DEFAULT 0
);

CREATE INDEX IF NOT EXISTS idx_datasets_user_id ON datasets(user_id);
CREATE INDEX IF NOT EXISTS idx_datasets_disease_code ON datasets(disease_code);
```

Listing E.2: Crearea tabelului `datasets`.

```sql
CREATE TABLE IF NOT EXISTS epidemiological_records (
    id            UUID DEFAULT gen_random_uuid(),
    dataset_id    UUID NOT NULL REFERENCES datasets(id) ON DELETE CASCADE,
    recorded_date DATE NOT NULL,
    region_code   VARCHAR(10) NOT NULL,
    susceptible   BIGINT,
    infected      BIGINT NOT NULL,
    recovered     BIGINT,
    population    BIGINT NOT NULL,
    PRIMARY KEY (dataset_id, recorded_date, id),
    CHECK (infected >= 0),
    CHECK (population > 0),
    CHECK (recovered IS NULL OR recovered >= 0),
    CHECK (susceptible IS NULL OR susceptible >= 0)
);

```

```
CREATE INDEX IF NOT EXISTS idx_epi_records_region ON epidemiological_records(
    region_code);
```

Listing E.3: Crearea tabelului epidemiological_records.

```
CREATE TABLE IF NOT EXISTS simulation_runs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    dataset_id    UUID NOT NULL REFERENCES datasets(id) ON DELETE CASCADE,
    model_type    VARCHAR(16) NOT NULL DEFAULT 'sidarthe',
    language      VARCHAR(16) NOT NULL,
    parameters    JSONB NOT NULL,
    population_n  BIGINT NOT NULL,
    t_max         INTEGER NOT NULL,
    status        VARCHAR(16) NOT NULL DEFAULT 'pending',
    error_message TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at  TIMESTAMPTZ,
    CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe')),
    CHECK (language IN ('r', 'julia', 'python')),
    CHECK (status IN ('pending', 'running', 'completed', 'failed')),
    CHECK (jsonb_typeof(parameters) = 'object'),
    CHECK (population_n > 0),
    CHECK (t_max > 0)
);

CREATE INDEX IF NOT EXISTS idx_sim_runs_user_id ON simulation_runs(user_id);
CREATE INDEX IF NOT EXISTS idx_sim_runs_dataset_id ON simulation_runs(dataset_id);
CREATE INDEX IF NOT EXISTS idx_sim_runs_status ON simulation_runs(status);
CREATE INDEX IF NOT EXISTS idx_sim_runs_language ON simulation_runs(language);
CREATE INDEX IF NOT EXISTS idx_sim_runs_model_type ON simulation_runs(model_type);
```

Listing E.4: Crearea tabelului simulation_runs.

```
CREATE TABLE IF NOT EXISTS simulation_results (
    simulation_id UUID NOT NULL REFERENCES simulation_runs(id) ON DELETE CASCADE,
    t             INTEGER NOT NULL,
    compartments  JSONB NOT NULL,
    PRIMARY KEY (simulation_id, t),
    CHECK (t >= 0),
    CHECK (jsonb_typeof(compartments) = 'object')
);
```

Listing E.5: Crearea tabelului simulation_results.

```
CREATE TABLE IF NOT EXISTS benchmarks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    simulation_id   UUID NOT NULL REFERENCES simulation_runs(id) ON DELETE CASCADE,
    wall_time_ms    BIGINT NOT NULL,
    cpu_time_ms     BIGINT NOT NULL,
    peak_memory_mb  REAL NOT NULL,
    CHECK (wall_time_ms >= 0),
    CHECK (cpu_time_ms >= 0),
    CHECK (peak_memory_mb >= 0)
);

CREATE INDEX IF NOT EXISTS idx_benchmarks_simulation_id ON benchmarks(simulation_id);
```

Listing E.6: Crearea tabelului benchmarks.

```
CREATE TABLE IF NOT EXISTS studies (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(255) NOT NULL,
```

```
    disease_code  VARCHAR(64) NOT NULL,
    description   TEXT,
    created_by    UUID NOT NULL REFERENCES users(id),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_studies_disease_code ON studies(disease_code);
CREATE INDEX IF NOT EXISTS idx_studies_created_by ON studies(created_by);
```

Listing E.7: Crearea tabelului `studies`.

```
CREATE TABLE IF NOT EXISTS study_members (
    study_id   UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE,
    user_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role       VARCHAR(32) NOT NULL,
    joined_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (study_id, user_id),
    CHECK (role IN ('admin', 'researcher', 'contributor', 'viewer'))
);

CREATE INDEX IF NOT EXISTS idx_study_members_user_id ON study_members(user_id);
CREATE INDEX IF NOT EXISTS idx_study_members_role ON study_members(role);
```

Listing E.8: Crearea tabelului `study_members`.

```
-- Adauga campul is_super_admin pe users si promoveaza primul user existent
-- (cel mai vechi created_at) la super admin daca nu exista inca niciunul.
-- Idempotent: la rulari repetate nu modifica nimic daca exista deja un super admin.
ALTER TABLE users ADD COLUMN IF NOT EXISTS is_super_admin BOOLEAN NOT NULL DEFAULT
    FALSE;

UPDATE users
SET is_super_admin = TRUE
WHERE id = (
    SELECT id FROM users ORDER BY created_at ASC LIMIT 1
)
AND NOT EXISTS (
    SELECT 1 FROM users WHERE is_super_admin = TRUE
);
```

Listing E.9: Adugarea flag-ului `is_super_admin`.

```
-- Adauga study_id la dataseturi si simulari. NOT NULL pentru consistenta
-- presupunem ca nu exista date pre-existente (cluster fresh) sau ca un studiu
-- default e creat manual inainte de migrare in mediile cu date.
ALTER TABLE datasets
    ADD COLUMN IF NOT EXISTS study_id UUID NOT NULL REFERENCES studies(id) ON DELETE
    CASCADE;

ALTER TABLE simulation_runs
    ADD COLUMN IF NOT EXISTS study_id UUID NOT NULL REFERENCES studies(id) ON DELETE
    CASCADE;

CREATE INDEX IF NOT EXISTS idx_datasets_study_id ON datasets(study_id);
CREATE INDEX IF NOT EXISTS idx_simulation_runs_study_id ON simulation_runs(study_id);
```

Listing E.10: Adugarea coloanei `study_id` pe dataseturi i simulri.

```
ALTER TABLE simulation_runs
    DROP CONSTRAINT IF EXISTS simulation_runs_model_type_check;

UPDATE simulation_runs
```

```
SET model_type = 'sidarthe'
WHERE model_type = 'sidarth';

-- Include si valorile adaugate de migratiile ulterioare (000012: 'lstm',
-- 000015: 'script')  altfel re-rularea job-ului pe un DB matur esueaza pe
-- randurile existente cu acele valori. Comportamentul final (dupa 000015)
-- ramane identic.
ALTER TABLE simulation_runs
    ADD CONSTRAINT simulation_runs_model_type_check
    CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe', 'lstm', 'script')
    );
```

Listing E.11: Extinderea constrângerii `model_type` pentru toate modelele suportate.

```
ALTER TABLE simulation_runs
    DROP CONSTRAINT IF EXISTS simulation_runs_model_type_check;

UPDATE simulation_runs
SET model_type = 'sidarthe'
WHERE model_type = 'sidarth';

-- Include si 'script' (adaugat de 000015), ca re-rularea pe DB matur sa nu
-- esueze. Comportamentul final dupa 000015 ramane identic.
ALTER TABLE simulation_runs
    ADD CONSTRAINT simulation_runs_model_type_check
    CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe', 'lstm', 'script')
    );
```

Listing E.12: Adugarea modelului `lstm` în constrângerea `model_type`.

```
-- Tabel pentru runner-ele de algoritmi definii de utilizator (plug-in-uri).
-- Fiecare runner este scope-uit la un singur studiu i poate fi creat doar de
-- un super-admin. Scriptul (entry.py) este stocat ca text in baza de date.
-- Runner-ul trece prin starile: pending -> building -> ready -> stopped/failed.

CREATE TABLE IF NOT EXISTS runners (
    id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    study_id         UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE,
    uploaded_by      UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name             VARCHAR(128) NOT NULL,
    description      TEXT NOT NULL DEFAULT '',
    script_content   TEXT NOT NULL,
    parameter_schema JSONB NOT NULL DEFAULT '{}'::jsonb,
    status           VARCHAR(32) NOT NULL DEFAULT 'pending',
    status_message   TEXT NOT NULL DEFAULT '',
    image_tag        VARCHAR(255) NOT NULL DEFAULT '',
    queue_name       VARCHAR(128) NOT NULL,
    deployment_name  VARCHAR(128) NOT NULL DEFAULT '',
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    ready_at         TIMESTAMPTZ,
    CHECK (status IN ('pending', 'building', 'ready', 'failed', 'stopped'))
);

CREATE INDEX IF NOT EXISTS idx_runners_study_id ON runners(study_id);
CREATE INDEX IF NOT EXISTS idx_runners_status ON runners(status);
CREATE UNIQUE INDEX IF NOT EXISTS idx_runners_study_name ON runners(study_id, name);
```

Listing E.13: Crearea tabelului `runners` pentru runner-ele plug-in.

```
-- Adauga modul "image" pentru runner-ele plug-in: in loc sa incarce un script
-- Python (procesat de wrapper-ul runner-custom), super-adminul poate da direct
-- o referinta catre o imagine container construita extern. Cele doua moduri
-- sunt mutual exclusive si toate runner-ele existente raman valide (script_mode).
```

```
--
-- NOTA: migratia 000015 ulterior dropeaza coloana script_content (pivotul:
-- runner-ele in mod script sunt eliminate, ramane doar mod image). Pentru ca
-- re-rularea job-ului de migrare pe un DB matur (post-15) sa nu esueze pe
-- referinte la script_content, incadram fiecare comanda intr-un bloc care o
-- executa doar daca coloana mai exista.

DO $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM information_schema.columns
        WHERE table_name='runners' AND column_name='script_content'
    ) THEN
        ALTER TABLE runners ALTER COLUMN script_content DROP NOT NULL;
    END IF;
END $$;

ALTER TABLE runners
    ADD COLUMN IF NOT EXISTS image VARCHAR(255) NOT NULL DEFAULT '';

-- XOR: exact unul dintre script_content (non-empty) sau image (non-empty)
-- trebuie sa fie prezent. Asta permite migrarea celor existente (toate au
-- script_content non-empty si image='') si validarea celor noi. Pe DB matur
-- (post-15) coloana nu mai exista; constrangerea finala e instalata de 000015.
DO $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM information_schema.columns
        WHERE table_name='runners' AND column_name='script_content'
    ) AND NOT EXISTS (
        SELECT 1 FROM pg_constraint WHERE conname='runners_source_check'
    ) THEN
        ALTER TABLE runners
            ADD CONSTRAINT runners_source_check CHECK (
                (script_content IS NOT NULL AND script_content <> '' AND image = '')
                OR
                (image <> '' AND (script_content IS NULL OR script_content = ''))
            );
    END IF;
END $$;
```

Listing E.14: Pivot ctre modul `image` pentru runner-e (DDL idempotent: condiional pe existena coloanei `script_content`, dropped ulterior).

```
-- Pivot: scripturile nu mai sunt "runner-e" persistente, ci o forma de
-- simulare ad-hoc care ruleaza pe un executor partajat. Tabela runners ramane
-- strict pentru runner-ele persistente bazate pe imagine externa.

-- 1) Curatare runners: pastram doar runner-ele in mod image. Eventualele
--    runner-e in mod script sunt sterse (Deployment-urile lor au fost
--    deja oprite de testele anterioare).
DELETE FROM runners WHERE script_content IS NOT NULL AND image = '';

ALTER TABLE runners DROP CONSTRAINT IF EXISTS runners_source_check;
ALTER TABLE runners DROP COLUMN IF EXISTS script_content;

-- Acum image este strict obligatoriu.
ALTER TABLE runners ADD CONSTRAINT runners_image_required CHECK (image <> '');

-- 2) Extinderea simulation_runs cu script_content (nullable, doar pentru
--    simularile de tip 'script'). Plus extinderea CHECK-ului pe model_type.
ALTER TABLE simulation_runs ADD COLUMN IF NOT EXISTS script_content TEXT;

```

```sql
ALTER TABLE simulation_runs DROP CONSTRAINT IF EXISTS
    simulation_runs_model_type_check;
ALTER TABLE simulation_runs ADD CONSTRAINT simulation_runs_model_type_check
    CHECK (model_type IN ('sis', 'sir', 'seir', 'sirs', 'sidarthe', 'lstm', 'script')
    );

-- Coerenta: pentru model_type='script' script_content trebuie sa fie non-vid;
-- pentru celelalte trebuie sa fie NULL (sau lipsa).
ALTER TABLE simulation_runs ADD CONSTRAINT simulation_runs_script_content_check
    CHECK (
        (model_type = 'script' AND script_content IS NOT NULL AND script_content <> '
    ')
        OR
        (model_type <> 'script' AND script_content IS NULL)
    );
```

Listing E.15: Simulri de tip `script`: `script_content` mutat în `simulation_runs`, runner-ele devin strict *image-based*.

```sql
-- Pentru simularile de tip 'script', t_max este metadata optionala (scriptul
-- isi alege orizontul singur). Pentru celelalte tipuri ramane obligatorie.
ALTER TABLE simulation_runs DROP CONSTRAINT IF EXISTS simulation_runs_t_max_check;
ALTER TABLE simulation_runs ADD CONSTRAINT simulation_runs_t_max_check
    CHECK ((model_type = 'script') OR (t_max > 0));
```

Listing E.16: Relaxarea constrângerii `t_max` pentru simulri de tip `script` (scriptul îi alege singur orizontul).

```sql
-- Draft-uri de cod pentru editorul de scripturi din UI. Fiecare cercetator are
-- propriile draft-uri per studiu si nu vede draft-urile altor utilizatori.
-- Continutul este pastrat ca text in DB (scripturi <1 MB tipic); pentru scale-up
-- la sute de MB per draft, datele pot fi mutate la stocare obiect (S3/MinIO),
-- mentinand metadatele in tabela.

CREATE TABLE IF NOT EXISTS script_drafts (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    study_id        UUID NOT NULL REFERENCES studies(id) ON DELETE CASCADE,
    name            VARCHAR(128) NOT NULL,
    description     TEXT NOT NULL DEFAULT '',
    script_content  TEXT NOT NULL,
    parameters_json JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_script_drafts_user_study ON script_drafts(user_id,
    study_id);
CREATE UNIQUE INDEX IF NOT EXISTS idx_script_drafts_user_study_name ON script_drafts(
    user_id, study_id, name);
```

Listing E.17: Crearea tabelului `script_drafts` pentru persistena draft-urilor din editor.

```sql
-- Extindere schema epidemiological_records cu coloane suplimentare folosite
-- frecvent in literatura si in seturile OWID/WHO/ECDC, dincolo de cele 4
-- coloane SIR-centrice initiale (susceptible/infected/recovered/population).
-- Toate sunt nullable pentru ca nu orice dataset le include simultan.
-- Runner-ii built-in (SIR/SIDARTHE/LSTM) le ignora silentios; scripturile
-- definite de utilizator au acces la toate prin records[i] in main().

ALTER TABLE epidemiological_records
    ADD COLUMN IF NOT EXISTS deaths              BIGINT,     -- decese cumulate
    ADD COLUMN IF NOT EXISTS new_cases           BIGINT,     -- cazuri noi (interval =
    granularitatea dataset-ului)
```

```
11      ADD COLUMN IF NOT EXISTS new_deaths          BIGINT,     -- decese noi (interval ca
        mai sus)
12      ADD COLUMN IF NOT EXISTS hospitalized        BIGINT,     -- pacienti spitalizati la
        moment
13      ADD COLUMN IF NOT EXISTS icu                 BIGINT,     -- pacienti in terapie
        intensiva la moment
14      ADD COLUMN IF NOT EXISTS vaccinated          BIGINT,     -- persoane vaccinate (cel
        putin o doza) cumulat
15      ADD COLUMN IF NOT EXISTS tested              BIGINT,     -- teste cumulate
16      ADD COLUMN IF NOT EXISTS stringency_index    REAL,       -- Oxford Stringency Index
        , 0..100
17      ADD COLUMN IF NOT EXISTS reproduction_rate   REAL;       -- Rt observat (cand este
        disponibil)
```

Listing E.18: Extinderea schemei `epidemiological_records` cu metrici suplimentare (decese, spitalizri, ATI, vaccinri, teste, indicele Oxford de stringen, R_t observat).

Anexa F

Configuraii Kubernetes

Aceast anex include manifestele declarative pentru rularea sistemului pe un cluster Kubernetes. Toate resursele sunt aplicate în namespace-ul `epimodel` i urmeaz structura arhitectural descris în capitolul 4: API stateless, dou frontend-uri servite prin Nginx, runnere scalabile independent, Redis pentru coada de mesaje i YugabyteDB pentru persisten (3 master + 3 tserver, RF=3).

Manifestele se afl în directorul `code/k8s/` al repository-ului i sunt organizate cu prefixe numerice pentru aplicare secvenial. Table F.1 sintetizeaz coninutul directorului.

Tabela F.1: Manifestele Kubernetes incluse în repository.

Fiier	**Resurse principale**	**Rol**
`00-namespace.yaml`	`Namespace`	Izoleaz aplicaia într-un namespace dedicat
`01-configmap.yaml`	`ConfigMap`	Variabile non-sensibile (HTTP, Redis, JWT)
`02-secret.example.yaml`	`Secret` (template)	ablon pentru `JWT_SECRET` i `DATABASE_URL`
`10-yugabyte-values.yaml`	Helm values	Configurarea chart-ului oficial `yugabytedb/yugabyte` (RF=3, sizing custom)
`12-yugabyte-tserver-service.yaml`	`Service`	`ClusterIP` `yb-tserver-service` expus pe portul 5433 (YSQL)
`20-redis.yaml`	`Deployment` + `Service`	Coad Redis (`emptyDir`, fr persisten)
`30-migrate-job.yaml`	`Job`	Aplic migraiile SQL prin `psql`
`40-api.yaml`	`Deployment` + `Service`	Backend Go (REST API + RBAC pe studii)
`50-runner-python.yaml`	`Deployment`	Runner Python (`scipy.solve_ivp`)
`51-runner-r.yaml`	`Deployment`	Runner R (`deSolve::ode`)
`52-runner-julia.yaml`	`Deployment`	Runner Julia (`DifferentialEquations.jl`)
`53-runner-lstm.yaml`	`Deployment`	Runner LSTM dedicat pentru predicii pe seria `infected`
`60-frontend.yaml`	`Deployment` + `Service`	Frontend user-facing servit prin Nginx
`61-frontend-admin.yaml`	`Deployment` + `Service`	Consol super-admin servit prin Nginx
`70-ingress.yaml`	`Ingress`	Rutare HTTP pentru `sir-workbench.ro` i `admin.sir-workbench.ro`

Listrile care urmeaz reproduc fiierele cele mai relevante pentru explicitarea arhitecturii: namespace-ul de izolare, fiierul de `values` pentru YugabyteDB, Service-ul de acces YSQL, coada Redis, Job-ul de migraii, API-ul, un exemplu de runner (Julia, cel mai mare consumator de memorie), runner-ul LSTM dedicat, frontend-ul admin i ingress-ul de rutare. Celelalte manifeste — ConfigMap-ul, template-ul Secret, frontend-ul principal, runner-ele R i Python — urmeaz aceleai convenii i pot fi consultate direct în repository.

Namespace dedicat

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: epimodel
  labels:
    app.kubernetes.io/part-of: epimodel
    app.kubernetes.io/managed-by: kubectl
```

Listing F.1: `00-namespace.yaml` — Namespace-ul dedicat aplicaiei.

YugabyteDB — valori Helm

YugabyteDB ruleaz ca dou `StatefulSet`-uri (`yb-master` i `yb-tserver`) cu câte 3 replici, generate de Helm chart-ul oficial. Fiierul de `values` suprascrie sizing-ul implicit pentru a se potrivi resurselor cluster-ului (workeri 4 vCPU / 8 GiB) i dezactiveaz componentele neutilizate (`yugabytedUi`, `enableLoadBalancer`).

```yaml
# Helm values pentru YugabyteDB pe clusterul kube-4kdhb
# Profil "PoC pentru disertatie": RF=3, dar requests mici pentru workeri 2vCPU/4GiB.
#
# Instalare:
#   helm install yugabyte yugabytedb/yugabyte \
#     --namespace epimodel \
#     --create-namespace=false \
#     -f code/k8s/10-yugabyte-values.yaml \
#     --wait --timeout 10m
#
# Verifica:
#   kubectl -n epimodel get pods,svc,pvc
#   kubectl -n epimodel exec -it yb-tserver-0 -- ysqlsh -h yb-tserver-service -U
    yugabyte -c "\l"
#
# Deinstalare (atenie, terge PVC-urile):
#   helm uninstall yugabyte -n epimodel
#   kubectl -n epimodel delete pvc -l app.kubernetes.io/name=yb-master
#   kubectl -n epimodel delete pvc -l app.kubernetes.io/name=yb-tserver

Image:
  tag: "2024.1.3.0-b105"   # versiune stabil LTS (2025.2.x e foarte recent)
  pullPolicy: IfNotPresent

storage:
  ephemeral: false
  master:
    count: 1               # 1 PV per master (default e 2)
    size: 5Gi
    storageClass: "block-ceph"
  tserver:
    count: 1               # 1 PV per tserver
    size: 10Gi
    storageClass: "block-ceph"

resource:
  # Workeri 4 vCPU / 8 GiB: tserver primeste 2 GiB request / 3 GiB limit pentru
  # cache decent. Master ramane mai mic (1 GiB)  tine metadata, nu date.
  master:
    requests:
      cpu: "500m"
      memory: 1Gi
    limits:
```

```
      cpu: "1"
      memory: 1.5Gi
  tserver:
    requests:
      cpu: "1"
      memory: 2Gi
    limits:
      cpu: "2"
      memory: 3Gi

replicas:
  master: 3
  tserver: 3

# Octavia nu functioneaza pe acest cluster, asa ca folosim ClusterIP peste tot.
enableLoadBalancer: false

# Disable UI yugabyted (consuma resurse, deschide ports pe care nu le expunem).
yugabytedUi:
  enabled: false

# Personalizam gflags pentru cluster mic.
gflags:
  master:
    default_memory_limit_to_ram_ratio: "0.6"
  tserver:
    default_memory_limit_to_ram_ratio: "0.6"
    ysql_num_shards_per_tserver: "2"

# Endpoints expuse  toate ClusterIP, fara LoadBalancer.
serviceEndpoints:
  - name: "yb-master-ui"
    type: ClusterIP
    app: "yb-master"
    ports:
      http-ui: "7000"

  - name: "yb-tserver-service"
    type: ClusterIP
    app: "yb-tserver"
    ports:
      tcp-yql-port: "9042"
      tcp-ysql-port: "5433"

# Distribuim pod-urile pe noduri diferite (anti-affinity soft default in chart).
# Pe 3 workeri vrem cate un master si cate un tserver pe fiecare.
masterAffinity: |
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: "yb-master"
          topologyKey: kubernetes.io/hostname

tserverAffinity: |
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
```

```
            matchLabels:
              app: "yb-tserver"
          topologyKey: kubernetes.io/hostname
```

Listing F.2: `10-yugabyte-values.yaml` — Valorile Helm pentru YugabyteDB.

Service de acces la YugabyteDB

Chart-ul oficial genereaz dou Service-uri *headless* (`yb-masters` i `yb-tservers`) pentru DNS-ul intern necesar protocolului Raft, dar nu un Service `ClusterIP` clasic pe care îl pot folosi consumatorii YSQL. Acest Service este adugat printr-un manifest separat.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: yb-tserver-service
  namespace: epimodel
  labels:
    app: yb-tserver
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  selector:
    app: yb-tserver
  ports:
    - name: tcp-yql-port
      port: 9042
      targetPort: 9042
    - name: tcp-ysql-port
      port: 5433
      targetPort: 5433
```

Listing F.3: `12-yugabyte-tserver-service.yaml` — `ClusterIP` `yb-tserver-service` (port 5433, YSQL).

Redis

Redis este deployat ca `Deployment` cu o singur replic i volum efemer (`emptyDir`). Cozile de mesaje nu sunt persistente între restarturi, ceea ce este acceptabil pentru job-uri de simulare care dureaz secunde sau minute. Pentru retenie durabil, structura poate fi convertit într-un `StatefulSet` cu `PersistentVolumeClaim` fr modificri la nivelul aplicaiei.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: epimodel
  labels:
    app.kubernetes.io/name: redis
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: redis
      port: 6379
      targetPort: 6379
  selector:
    app.kubernetes.io/name: redis
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
```

```
  namespace: epimodel
  labels:
    app.kubernetes.io/name: redis
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app.kubernetes.io/name: redis
  template:
    metadata:
      labels:
        app.kubernetes.io/name: redis
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 30
      volumes:
        - name: data
          emptyDir:
            sizeLimit: 512Mi
      containers:
        - name: redis
          image: redis:7-alpine
          imagePullPolicy: IfNotPresent
          # Fara persistenta  folosim emptyDir pentru a economisi un volum OpenStack
          # (quota tenant 10 vol, prioritizam Yugabyte RF=3). Cozile se pierd la
    restart Pod,
          # dar simularile dureaza secunde, nu zile.
          args:
            - redis-server
            - --save
            - ""
            - --appendonly
            - "no"
          ports:
            - name: redis
              containerPort: 6379
          volumeMounts:
            - name: data
              mountPath: /data
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 256Mi
          readinessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 3
          livenessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 15
            periodSeconds: 15
            timeoutSeconds: 5
```

Listing F.4: `20-redis.yaml` — Redis cu volum efemer.

Job-ul de migrare

Migraiile SQL sunt aplicate printr-un `Job` care folosete clientul `psql` (în loc de `golang-migrate`) pentru a evita dependena de *advisory locks* PostgreSQL, pe care YugabyteDB nu le implementeaz înc. Job-ul itereaz fiierele `*.up.sql` în ordine alfabetic i le aplic cu `ON_ERROR_STOP=1`.

```
# Job care aplica migraiile SQL pe YugabyteDB.
# Migraiile sunt încrcate dintr-un ConfigMap generat din ./db/migrations/.
#
# Înainte de a aplica acest Job, genereaz ConfigMap-ul:
#
#   kubectl create configmap db-migrations \
#     --namespace=epimodel \
#     --from-file=code/db/migrations/
#
# YugabyteDB nu suport (înc) advisory locks pe care golang-migrate le folosete,
# aa c rulm migraiile direct cu psql, în ordine alfabetic a fiierelor *.up.sql.
# Job-ul foloseste IF NOT EXISTS în migraii pentru idempoten.
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  namespace: epimodel
  labels:
    app.kubernetes.io/name: db-migrate
    app.kubernetes.io/part-of: epimodel
spec:
  backoffLimit: 3
  ttlSecondsAfterFinished: 600
  template:
    metadata:
      labels:
        app.kubernetes.io/name: db-migrate
        app.kubernetes.io/part-of: epimodel
    spec:
      restartPolicy: OnFailure
      initContainers:
        - name: wait-for-ysql
          image: postgres:16-alpine
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          command:
            - /bin/sh
            - -c
            - |
              until pg_isready -d "$DATABASE_URL"; do
                echo "waiting for YSQL ..."
                sleep 3
              done
              echo "YSQL is ready"
      containers:
        - name: migrate
          image: postgres:16-alpine
          imagePullPolicy: IfNotPresent
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
```

```
                  key: DATABASE_URL
          command:
            - /bin/sh
            - -c
            - |
              set -e
              echo "Applying migrations in order..."
              for f in $(ls /migrations/*.up.sql | sort); do
                echo ""
                echo "=== Applying $f ==="
                psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$f"
              done
              echo ""
              echo "All migrations applied successfully."
              psql "$DATABASE_URL" -c "\dt"
          volumeMounts:
            - name: migrations
              mountPath: /migrations
          resources:
            requests:
              cpu: 50m
              memory: 32Mi
            limits:
              cpu: 200m
              memory: 128Mi
      volumes:
        - name: migrations
          configMap:
            name: db-migrations
```

Listing F.5: `30-migrate-job.yaml` — Job care aplic migraiile prin `psql`.

API Go

API-ul ruleaz ca `Deployment` cu dou replici pentru toleran la pierderea unui pod. Configuraia non-sensibil provine din `ConfigMap`, iar parolele i secretele din `Secret`. Probele HTTP pe `/api/health` verific atât pod-ul, cât i conectivitatea efectiv ctre baza de date i Redis.

```
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: epimodel
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 8080
      targetPort: 8080
  selector:
    app.kubernetes.io/name: api
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: epimodel
  labels:
    app.kubernetes.io/name: api
```

```
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
        app.kubernetes.io/part-of: epimodel
    spec:
      serviceAccountName: api
      terminationGracePeriodSeconds: 30
      imagePullSecrets:
        - name: ghcr-pull
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app.kubernetes.io/name: api
                topologyKey: kubernetes.io/hostname
      containers:
        - name: api
          image: ghcr.io/masterprogramercpu/emedicine-api:latest
          imagePullPolicy: Always
          envFrom:
            - configMapRef:
                name: epimodel-config
            - secretRef:
                name: epimodel-secrets
          env:
            # Activeaza provisioning-ul K8s pentru runner-ele plug-in.
            - name: RUNNER_DEPLOY_ENABLED
              value: "true"
          ports:
            - name: http
              containerPort: 8080
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /api/health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
            timeoutSeconds: 3
          livenessProbe:
            httpGet:
```

```
              path: /api/health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 30
            timeoutSeconds: 5
```

Listing F.6: `40-api.yaml` — Deployment i Service pentru API-ul Go.

Runner Julia

Runner-ul Julia este `Deployment`-ul cu cele mai generoase limite de resurse (512 MiB request, 2 GiB limit) din cauza costului iniial de încrcare al pachetelor `DifferentialEquations.jl`. Runner-ele R i Python au structuri identice, dar cu limite mai mici.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: runner-julia
  namespace: epimodel
  labels:
    app.kubernetes.io/name: runner-julia
    app.kubernetes.io/component: runner
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app.kubernetes.io/name: runner-julia
  template:
    metadata:
      labels:
        app.kubernetes.io/name: runner-julia
        app.kubernetes.io/component: runner
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 60
      imagePullSecrets:
        - name: ghcr-pull
      containers:
        - name: runner-julia
          image: ghcr.io/masterprogramercpu/emedicine-runner-julia:latest
          imagePullPolicy: Always
          env:
            - name: REDIS_URL
              valueFrom:
                configMapKeyRef:
                  name: epimodel-config
                  key: REDIS_URL
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          resources:
            requests:
              cpu: 200m
              memory: 384Mi
            limits:
              cpu: "1"
              memory: 1Gi
```

Listing F.7: `52-runner-julia.yaml` — Deployment pentru runner-ul Julia.

Runner LSTM

Runner-ul LSTM este separat de runner-ul Python pentru simulri compartimentale. El consum exclusiv coada `epimodel:jobs:lstm`, antreneaz modelul recurent pe seria `infected` i persist separat valorile observate, potrivite i prezise.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: runner-lstm
  namespace: epimodel
  labels:
    app.kubernetes.io/name: runner-lstm
    app.kubernetes.io/component: runner
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app.kubernetes.io/name: runner-lstm
  template:
    metadata:
      labels:
        app.kubernetes.io/name: runner-lstm
        app.kubernetes.io/component: runner
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 60
      imagePullSecrets:
        - name: ghcr-pull
      containers:
        - name: runner-lstm
          image: ghcr.io/masterprogramercpu/emedicine-runner-lstm:latest
          imagePullPolicy: Always
          env:
            - name: REDIS_URL
              valueFrom:
                configMapKeyRef:
                  name: epimodel-config
                  key: REDIS_URL
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          resources:
            requests:
              cpu: 300m
              memory: 512Mi
            limits:
              cpu: "2"
              memory: 2Gi
```

Listing F.8: `53-runner-lstm.yaml` — Deployment pentru runner-ul LSTM.

Frontend admin

Consola super-admin ruleaz ca un `Deployment` separat fa de aplicaia principal, dei ambele folosesc acelai API. Aceast separare produce imagini Docker distincte i permite rutarea prin host-ul `admin.sir-workbench.ro`.

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-admin
  namespace: epimodel
  labels:
    app.kubernetes.io/name: frontend-admin
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 80
      targetPort: 80
  selector:
    app.kubernetes.io/name: frontend-admin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-admin
  namespace: epimodel
  labels:
    app.kubernetes.io/name: frontend-admin
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: frontend-admin
  template:
    metadata:
      labels:
        app.kubernetes.io/name: frontend-admin
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 30
      imagePullSecrets:
        - name: ghcr-pull
      containers:
        - name: frontend-admin
          image: ghcr.io/masterprogramercpu/emedicine-frontend-admin:latest
          imagePullPolicy: Always
          ports:
            - name: http
              containerPort: 80
          resources:
            requests:
              cpu: 50m
              memory: 32Mi
            limits:
              cpu: 200m
              memory: 128Mi
```

```
          readinessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 15
            periodSeconds: 30
```

Listing F.9: `61-frontend-admin.yaml` — Deployment i Service pentru consola super-admin.

Ingress de rutare

Ingress-ul nginx ruteaz cererile externe pe dou host-uri: `sir-workbench.ro` pentru aplicaia principal i `admin.sir-workbench.ro` pentru consola super-admin. Pe ambele host-uri, prefixul `/api` este trimis ctre API-ul Go; restul traficului ajunge la frontend-ul corespunztor. Într-un deployment cu DNS public, host-urile se înlocuiesc cu FQDN-uri reale i se adaug o seciune `tls:` cu certificate gestionate prin `cert-manager`.

```
# Ingress nginx pentru rutarea traficului HTTP extern.
# Presupune c nginx-ingress controller este deja instalat în cluster
# (https://kubernetes.github.io/ingress-nginx/deploy/).
#
# Pentru testare local, mapeaz în /etc/hosts:
#   127.0.0.1  sir-workbench.ro
#   127.0.0.1  admin.sir-workbench.ro
#
# Pentru producie, înlocuiete host-urile cu FQDN-uri reale i adaug o
# seciune `tls:` cu un Secret de tip kubernetes.io/tls (cert-manager etc.).
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: epimodel-ingress
  namespace: epimodel
  labels:
    app.kubernetes.io/part-of: epimodel
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "64m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "120"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "120"
spec:
  ingressClassName: nginx
  rules:
    # Host principal  aplicaia user-facing + API
    - host: sir-workbench.ro
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 8080
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
```

```
                port:
                  number: 80
    # Host dedicat pentru consola super-admin
    - host: admin.sir-workbench.ro
      http:
        paths:
          # API-ul rmâne accesibil i de pe sub-domain (apelurile XHR
          # din admin SPA folosesc /api/... relativ la admin.sir-workbench.ro).
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 8080
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-admin
                port:
                  number: 80
```

Listing F.10: `70-ingress.yaml` — Ingress nginx pentru rutare HTTP.

Alternativ pentru dezvoltare local

În paralel cu manifestele Kubernetes, repository-ul include i o configuraie Docker Compose care reproduce aceeai topologie pe o singur main, fr nevoia unui cluster. Aceasta este util pentru dezvoltare local i pentru verificri rapide ale fluxurilor end-to-end fr costul operaional al unui cluster K8s real.

```
services:
  yugabyte:
    image: yugabytedb/yugabyte:2.21.1.0-b271
    container_name: emedicine-yugabyte
    command:
      - bin/yugabyted
      - start
      - --background=false
      - --tserver_flags=ysql_num_shards_per_tserver=4
    ports:
      - "5433:5433"
      - "7000:7000"
      - "9000:9000"
    volumes:
      - yugabyte-data:/root/var
    healthcheck:
      test: ["CMD-SHELL", "bin/ysqlsh -h yugabyte -p 5433 -U yugabyte -c 'SELECT 1' >/dev/null 2>&1 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 30s

  redis:
    image: redis:7-alpine
    container_name: emedicine-redis
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
```

```
      timeout: 3s
      retries: 10

  migrate:
    image: migrate/migrate:v4.17.1
    container_name: emedicine-migrate
    depends_on:
      yugabyte:
        condition: service_healthy
    volumes:
      - ./db/migrations:/migrations
    entrypoint:
      - migrate
      - -path=/migrations
      - -database=postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable
      - up
    restart: "no"

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    container_name: emedicine-api
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      HTTP_ADDR: ":8080"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=
    disable"
      REDIS_URL: "redis://redis:6379/0"
      JWT_SECRET: "${JWT_SECRET:-dev-secret-change-me}"
      RUNNER_DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?
    sslmode=disable"
    ports:
      - "8080:8080"
    restart: unless-stopped

  runner-python:
    build:
      context: ./runners/python
      dockerfile: Dockerfile
    container_name: emedicine-runner-python
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=
    disable"
    restart: unless-stopped

  runner-lstm:
    build:
      context: ./runners/lstm
      dockerfile: Dockerfile
    container_name: emedicine-runner-lstm
    depends_on:
```

```
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  runner-r:
    build:
      context: ./runners/r
      dockerfile: Dockerfile
    container_name: emedicine-runner-r
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  runner-julia:
    build:
      context: ./runners/julia
      dockerfile: Dockerfile
    container_name: emedicine-runner-julia
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: emedicine-frontend
    depends_on:
      - api
    ports:
      - "5173:80"
    restart: unless-stopped

volumes:
  yugabyte-data:
```

Listing F.11: `docker-compose.yml` — Configuraia echivalent pentru mediul de dezvoltare local.

Anexa G

Documentaia API REST

API-ul REST este implementat în pachetul `internal/api` al backend-ului Go. Rutele sunt definite centralizat în routerul `chi`, iar handler-ele sunt separate pe domenii: autentificare, profil utilizator, studii, membri, dataseturi, simulri, benchmarkuri, super-admin i health check. Specificaia tabelar a endpointurilor se afl în Table 4.2; în aceast anex sunt incluse fiierele relevante pentru definirea rutelor i a formatului comun de rspuns.

```go
package router

import (
    "net/http"
    "time"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
    "github.com/go-chi/cors"

    "github.com/tudorgeana/emedicine-api/internal/api/handlers"
    authmw "github.com/tudorgeana/emedicine-api/internal/api/middleware"
    "github.com/tudorgeana/emedicine-api/internal/models"
)

type Deps struct {
    Health        *handlers.HealthHandler
    Auth          *handlers.AuthHandler
    Users         *handlers.UsersHandler
    Studies       *handlers.StudiesHandler
    Datasets      *handlers.DatasetsHandler
    Simulations   *handlers.SimulationsHandler
    Benchmarks    *handlers.BenchmarksHandler
    Admin         *handlers.AdminHandler
    Runners       *handlers.RunnersHandler
    ScriptDrafts *handlers.ScriptDraftsHandler
    JWT           *authmw.JWTMiddleware
    RBAC          *authmw.RBAC
}

func New(deps Deps) http.Handler {
    r := chi.NewRouter()

    r.Use(middleware.RequestID)
    r.Use(middleware.RealIP)
    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)
    r.Use(middleware.Timeout(60 * time.Second))
    r.Use(cors.Handler(cors.Options{
        AllowedOrigins:   []string{"*"},
        AllowedMethods:   []string{"GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"
    },
```

```
        AllowedHeaders:   []string{"Accept", "Authorization", "Content-Type"},
        AllowCredentials: false,
        MaxAge:           300,
    }))

    r.Route("/api", func(r chi.Router) {
        r.Get("/health", deps.Health.Get)

        r.Route("/auth", func(r chi.Router) {
            r.Post("/register", deps.Auth.Register)
            r.Post("/login", deps.Auth.Login)
        })

        // Toate rutele de mai jos cer autentificare.
        r.Group(func(r chi.Router) {
            r.Use(deps.JWT.Authenticate)

            // Profilul userului curent
            r.Get("/me", deps.Users.Me)

            // Studii  list / create sunt scope pe userul curent
            r.Route("/studies", func(r chi.Router) {
                r.Get("/", deps.Studies.List)
                r.Post("/", deps.Studies.Create)

                // Rute care lucreaza intr-un studiu specific
                r.Route("/{studyId}", func(r chi.Router) {
                    // Membership minim (orice rol)  citire
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership())

                        r.Get("/", deps.Studies.Get)
                        r.Get("/members", deps.Studies.ListMembers)

                        // Datasets  citire pentru toti membrii
                        r.Get("/datasets", deps.Datasets.List)
                        r.Get("/datasets/{id}", deps.Datasets.Get)

                        // Simulations  citire pentru toti membrii
                        r.Get("/simulations", deps.Simulations.List)
                        r.Get("/simulations/{id}", deps.Simulations.Get)
                        r.Get("/simulations/{id}/results", deps.Simulations.Results)
                        r.Get("/simulations/{id}/export", deps.Simulations.Export)

                        // Benchmarks  citire pentru toti membrii
                        r.Get("/benchmarks", deps.Benchmarks.Aggregate)

                        // Runners (plug-in-uri)  citire pentru toti membrii
                        r.Get("/runners", deps.Runners.List)
                        r.Get("/runners/{id}", deps.Runners.Get)

                        // Script drafts  fiecare membru isi vede doar draft-urile
                        // proprii (filtrare la nivel de repo dupa user_id).
                        r.Get("/script-drafts", deps.ScriptDrafts.List)
                        r.Get("/script-drafts/{id}", deps.ScriptDrafts.Get)
                        r.Post("/script-drafts", deps.ScriptDrafts.Create)
                        r.Patch("/script-drafts/{id}", deps.ScriptDrafts.Update)
                        r.Delete("/script-drafts/{id}", deps.ScriptDrafts.Delete)
                    })

                    // Runners  creare/stergere: doar super-admin.
                    // Lant: RequireStudyMembership (super-admin trece + seteaza
```

```
                    // study context) -> RequireSuperAdmin (filtreaza membrii normali).
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership())
                        r.Use(deps.RBAC.RequireSuperAdmin)
                        r.Post("/runners", deps.Runners.Create)
                        r.Delete("/runners/{id}", deps.Runners.Delete)
                    })

                    // Datasets  scriere: admin sau contributor
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership(models.RoleAdmin, models.RoleContributor))
                        r.Post("/datasets", deps.Datasets.Create)
                        r.Delete("/datasets/{id}", deps.Datasets.Delete)
                    })

                    // Simulations  scriere: admin sau researcher
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership(models.RoleAdmin, models.RoleResearcher))
                        r.Post("/simulations", deps.Simulations.Create)
                    })

                    // Admin studiu  doar rolul admin
                    r.Group(func(r chi.Router) {
                        r.Use(deps.RBAC.RequireStudyMembership(models.RoleAdmin))
                        r.Patch("/", deps.Studies.Update)
                        r.Delete("/", deps.Studies.Delete)
                        r.Post("/members", deps.Studies.AddMember)
                        r.Patch("/members/{userId}", deps.Studies.UpdateMemberRole)
                        r.Delete("/members/{userId}", deps.Studies.RemoveMember)
                    })
                })
            })

            // Endpoint-uri super admin
            r.Route("/admin", func(r chi.Router) {
                r.Use(deps.RBAC.RequireSuperAdmin)
                r.Get("/stats", deps.Admin.Stats)
                r.Get("/users", deps.Admin.ListUsers)
                r.Patch("/users/{userId}/super-admin", deps.Admin.SetSuperAdmin)
                r.Get("/studies", deps.Admin.ListStudies)
                r.Get("/breakdown", deps.Admin.Breakdown)
                r.Get("/recent-simulations", deps.Admin.RecentSimulations)
                r.Get("/recent-datasets", deps.Admin.RecentDatasets)
            })
        })
    })

    return r
}
```

Listing G.1: Routerul REST al API-ului Go.

```
package middleware

import (
    "context"
    "errors"
    "net/http"

    "github.com/go-chi/chi/v5"
```

```
    "github.com/google/uuid"

    "github.com/tudorgeana/emedicine-api/internal/db/repository"
    "github.com/tudorgeana/emedicine-api/internal/models"
)

const (
    studyIDKey   ctxKey = "study_id"
    studyRoleKey ctxKey = "study_role"
)

// RBAC contine dependentele necesare pentru middleware-urile de autorizare.
type RBAC struct {
    Users        *repository.UserRepository
    StudyMembers *repository.StudyMemberRepository
}

func NewRBAC(users *repository.UserRepository, members *repository.
    StudyMemberRepository) *RBAC {
    return &RBAC{Users: users, StudyMembers: members}
}

// RequireSuperAdmin permite trecerea doar daca userul curent are is_super_admin=true
    .
func (rb *RBAC) RequireSuperAdmin(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        uid, ok := UserIDFrom(r.Context())
        if !ok {
            writeAuthError(w, "missing user context")
            return
        }
        user, err := rb.Users.GetByID(r.Context(), uid)
        if err != nil || user == nil {
            writeAuthError(w, "user lookup failed")
            return
        }
        if !user.IsSuperAdmin {
            writeForbidden(w, "super admin required")
            return
        }
        next.ServeHTTP(w, r)
    })
}

// RequireStudyMembership verifica daca userul are unul din rolurile permise in
    studiul
// identificat de URL param "studyId". Super-adminul trece intotdeauna.
// Daca allowedRoles e gol, orice rol valid trece (doar membership-ul conteaza).
func (rb *RBAC) RequireStudyMembership(allowedRoles ...models.StudyRole) func(http.
    Handler) http.Handler {
    allowed := make(map[models.StudyRole]struct{}, len(allowedRoles))
    for _, r := range allowedRoles {
        allowed[r] = struct{}{}
    }

    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            uid, ok := UserIDFrom(r.Context())
            if !ok {
                writeAuthError(w, "missing user context")
                return
            }
```

```
67
68              studyIDStr := chi.URLParam(r, "studyId")
69              studyID, err := uuid.Parse(studyIDStr)
70              if err != nil {
71                  writeBadRequest(w, "invalid studyId")
72                  return
73              }
74
75              // Super-adminul are acces direct, fara verificare de membership.
76              user, err := rb.Users.GetByID(r.Context(), uid)
77              if err != nil {
78                  writeAuthError(w, "user lookup failed")
79                  return
80              }
81
82              ctx := context.WithValue(r.Context(), studyIDKey, studyID)
83
84              if user.IsSuperAdmin {
85                  ctx = context.WithValue(ctx, studyRoleKey, models.RoleAdmin)
86                  next.ServeHTTP(w, r.WithContext(ctx))
87                  return
88              }
89
90              role, err := rb.StudyMembers.GetRole(r.Context(), studyID, uid)
91              if errors.Is(err, repository.ErrNotFound) {
92                  writeForbidden(w, "not a member of this study")
93                  return
94              }
95              if err != nil {
96                  writeForbidden(w, "membership lookup failed")
97                  return
98              }
99
100             if len(allowed) > 0 {
101                 if _, ok := allowed[role]; !ok {
102                     writeForbidden(w, "insufficient role")
103                     return
104                 }
105             }
106
107             ctx = context.WithValue(ctx, studyRoleKey, role)
108             next.ServeHTTP(w, r.WithContext(ctx))
109         })
110     }
111 }
112
113 // StudyIDFrom returneaza ID-ul studiului curent din context.
114 func StudyIDFrom(ctx context.Context) (uuid.UUID, bool) {
115     v := ctx.Value(studyIDKey)
116     id, ok := v.(uuid.UUID)
117     return id, ok
118 }
119
120 // StudyRoleFrom returneaza rolul userului curent in studiul din context.
121 func StudyRoleFrom(ctx context.Context) (models.StudyRole, bool) {
122     v := ctx.Value(studyRoleKey)
123     r, ok := v.(models.StudyRole)
124     return r, ok
125 }
126
127 func writeForbidden(w http.ResponseWriter, msg string) {
128     w.Header().Set("Content-Type", "application/json")
```

```
    w.WriteHeader(http.StatusForbidden)
    _, _ = w.Write([]byte(`{"error":{"code":403,"message":"` + msg + `"}}`))
}

func writeBadRequest(w http.ResponseWriter, msg string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusBadRequest)
    _, _ = w.Write([]byte(`{"error":{"code":400,"message":"` + msg + `"}}`))
}
```

Listing G.2: Middleware-ul RBAC pentru studii i super-admin.

```
package handlers

import (
    "encoding/json"
    "net/http"
)

type envelope struct {
    Data  any          `json:"data,omitempty"`
    Error *errPayload `json:"error,omitempty"`
}

type errPayload struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
}

func writeJSON(w http.ResponseWriter, status int, data any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(envelope{Data: data})
}

func writeError(w http.ResponseWriter, status int, message string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(envelope{Error: &errPayload{Code: status, Message:
    message}})
}

func decodeJSON(r *http.Request, dst any) error {
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    return dec.Decode(dst)
}
```

Listing G.3: Formatul comun al rspunsurilor JSON.